

Information Modelling Framework Manual

Version 0.4.0 INTERNAL

This document is a draft and subject to change. It is not intended for distribution beyond the designated audience. Please read the draft notice on next page.

2026-01-12

Draft Notice

This version of the Information Modelling Framework (IMF) Manual is an internal release that presents results from work carried out in the DISC ID Management Project, the DISC Reasoning Projects, and ongoing EU-funded research projects. *This version incorporates these contributions as they currently stand, with full harmonisation across the framework to follow in subsequent releases.* The major contributions are listed below.

The results constitute major improvements to the IMF Framework, some of which are expected to be breaking and backwards incompatible with previous versions. For this reason, this version is released as an internal version for hearing rather than as a public release, and is distributed to a selected audience for the purpose of collecting feedback. Due to the scope and status of these changes, they have not yet been fully propagated to all parts of the framework—in particular the IMF metamodel implementations, including the IMF OWL ontology and SHACL shapes. These components will be updated in subsequent releases once feedback has been incorporated and the conceptual changes have stabilised. A list of future work is given below.

Request for Feedback Recipients of this internal version are encouraged to review the proposed changes and provide feedback. Feedback will be incorporated into subsequent revisions before a public release is made.

Summary of Major Changes

Revised Aspect Definition The formulation of aspects has been substantially reworked ([Chapter 3](#)). In particular, the Interest parameter has been given an updated and central role. Interest now serves as a key mechanism for a selection/filtering relation on elements. This change enables more precise contextualisation of model elements according to engineering discipline, allowing the same system to be viewed from different perspectives (functional, physical, spatial, etc.) while maintaining semantic coherence.

Revised Modality Definition The definition of *Modality* has been extended with the introduction of the *Pattern* modality ([Chapter 3](#)). While earlier IMF versions (including v0.3) distinguished only between *Actual* and *Intended*, v0.4 explicitly recognises *Pattern* as a first-class modality for expressing typical, reusable, or reference-level structural definitions that are not tied to a specific installation or design instance. This extension clarifies the status of templates, standard solutions, and reference models, and provides a cleaner separation between reusable engineering knowledge and project-specific intent or realization.

New Structure for Attribute Information This version introduces *Quality* as a new element type for collecting and relating groups of attributes ([Chapter 4](#)). This structural change relieves other element types (such as *Block* and *Terminal*) of the responsibility for managing attribute information, allows elements to focus on the identification of system elements and their relations, and enables better alignment with industrial data formats such as Asset Administration Shell (AAS), datasheets, and property lists. *Qualities* can represent measurements, properties, parameters, and other attribute collections that characterise system elements.

Visual Graph Notation This version includes an updated specification of a visual language for serialising IMF models ([Chapter 7](#)). The notation defines rendering conventions for elements (*Blocks*, *Terminals*, *Connectors*, *Qualities*) as nodes with distinct shapes, relations as edges, aspects through fill colours, and breakdown hierarchies using groups. The visual notation provides an intuitive way to communicate IMF models across engineering teams and supports model documentation and review processes.

IMF Metamodel Implementation This version includes a new chapter describing the concrete implementation of the IMF metamodel ([Chapter 8](#)). The chapter presents both the normative IMF OWL Ontology and SHACL Shapes (which define the semantic and validation layers) and the emerging LinkML implementation (which serves as a single authoring source for generating multiple artefacts). The LinkML approach provides multi-format generation, data transformation, and automated documentation generation capabilities.

IMF Model Serialisation Formats This version includes descriptions of the different serialisation and representation formats for IMF models ([Chapter 9](#)): RDF (the canonical graph-based exchange format), OWL (class-based representation for classification reasoning), SHACL (shapes-based validation for quality assurance), and AAS (experimental mapping to the Asset Administration Shell framework for digital twin integration). Each format serves distinct purposes in the IMF ecosystem, from basic data exchange to formal reasoning and validation.

IMF Workbench This version includes a description of the IMF Workbench ([Chapter 10](#)), providing detailed guidance on tooling support for creating, editing, transforming, and validating IMF models. The chapter covers installation procedures, data transformation between formats, validation and reasoning using SHACL and OWL, visualisation capabilities, and export to external formats. The IMF Workbench provides practitioners with a practical toolchain for working with IMF models in real projects.

Practical Use Case A detailed practical use case of IMF is provided ([Chapter 11](#)), demonstrating the framework through a simple gas cooling system example. The use case illustrates how IMF integrates functional, physical, and spatial system information across multiple aspects within a single model, demonstrates querying capabilities for answering practical engineering questions (such as tracing the source of a gas leak), and shows how to generate different model representations (RDF, OWL, IDO, and AAS) for various stakeholder needs.

Document Restructuring The IMF manual has been restructured into four parts that progressively develop the framework from foundational concepts through formal specification to practical application. Chapters are structured with the intention of possibly becoming independent specifications.

Future Work

The following areas are identified for development in subsequent releases:

Update Structural Specification and Visual Graph Notation The structural specification ([Chapter 6](#)) and the visual graph notation ([Chapter 7](#)) is not synchronised with the recent developments of the IMF fundamentals. The Structural Specification will be updated to fully align with the revised aspect definition and the introduction of Quality as a new element type. This includes refining the formal ontology to reflect the updated Interest parameter's role in element selection and filtering, and incorporating Quality elements into the structural definitions and evaluation semantics.

Update Metamodel Implementation The normative IMF OWL Ontology and SHACL Shapes ([Chapter 8](#)) will be updated to fully incorporate the conceptual changes introduced in this version, including the revised aspect parameters and Quality elements. The LinkML implementation will be matured to achieve feature parity with the normative artefacts with the aim to become the primary authoring source for generating OWL, SHACL, and other format artefacts.

Formalise Types The notion of types and templates, currently introduced informally in [Chapter 4](#), will be given a formal treatment in the structural specification. Anticipated amendments include precise semantics for type definitions, instantiation rules, template parameterisation, and the relationship

between types and classifiers. The formalisation will enable reasoning about conformance and support automated validation of type usage in IMF models.

Formalise Requirement–Solution Relations The framework will be extended with formal semantics for relating requirements to solution elements, enabling traceability between system requirements and design solutions. This will include defining relation types for requirement satisfaction, requirement decomposition, and requirement allocation to system elements, building on the aspect-based contextualisation mechanisms introduced in this version.

Formalise IMF–IDO Alignment The alignment between IMF and the Industrial Design Ontology (IDO) will be given a formal specification. This will include defining precise mapping rules between IMF constructs and IDO concepts, establishing consistency conditions for integrated IMF-IDO models, and providing transformation procedures for converting between IMF and IDO representations.

Formalise Identifier Resolution and Reference Contexts Based on continued conceptual development the next releases will introduce a systematic treatment of identifier resolution in IMF, including the formalisation of *reference contexts*, cross-context identifier mappings, and controlled links to external data resources. This will provide a principled basis for resolving identifiers across organisational, lifecycle, and system boundaries, and for relating IMF structural definitions to identifiers used in external systems such as PLM, ERP, asset registers, and digital twin platforms. The integration of identifier management concepts developed in the DISC ID Management Project will be consolidated into the IMF manual once the core language and aspect framework have stabilised.

Contents

1	Introduction to the IMF Manual	1
I	Fundamentals of IMF	4
2	Visual Terms Primer for the IMF Language	5
3	Conceptual Fundament of IMF	13
3.1	Introduction	13
3.2	A motivating example: a P&ID fragment	14
3.3	Systems Thinking: Establishing Common Ground	16
3.4	Appearances: how systems show up in engineering information	20
3.5	Aspects: the organising principle for appearances	22
3.6	Revisiting the P&ID through the lens of aspects	25
3.7	Aspect switching and cross-aspect relations	26
3.8	Structuring principles within an aspect	27
4	Core Concepts of the IMF Language	29
4.1	Introduction	30
4.2	Contextual parameters of structural definitions	30
4.3	Structural roles in IMF	32
4.4	Structural elements of the IMF language	35
4.5	Composition of structural definitions via relations	37
4.6	Refining structural definitions: classifiers	40
4.7	IMF models as collections of structural definitions	43
4.8	From P&ID notation to structural definitions	44
4.9	Cross-aspect relations	49
4.10	Outline of formal evaluation	51
	Glossary	54
II	IMF Languages	56
5	Language Definitions	57
5.1	Terminology summary (reference)	57
5.2	Overview of syntactic building blocks	59
5.3	Elements	59

5.4	Relations	60
5.5	Classifiers	63
5.6	Attributes (deferred)	63
5.7	Well-formedness conditions	63
5.8	Conformance and evolution	63
6	Structural Specification	65
6.1	Introduction	66
6.2	Vocabulary Structure	68
6.3	Generic relations	68
6.4	InformationArtefacts	69
6.5	InformationModel	70
6.6	Elements	70
6.7	Aspects	72
6.8	Aspect Elements	73
6.9	Relations	74
6.10	AspectElement relations	79
6.11	Attributes	80
7	Visual Graph Notation	81
7.1	Introduction	81
7.2	Nodes are Elements	82
7.3	Edges are relationships	83
7.4	Fill colour and Aspects	88
7.5	Fill colour and Medium	88
7.6	Grouping	88
7.7	References and metadata	88
III	IMF as a Framework	92
8	IMF Metamodel Implementation	93
8.1	Introduction	93
8.2	IMF OWL Ontology and SHACL Shapes	94
8.3	The LinkML Implementation	98
9	IMF Serialisation Formats	101
9.1	RDF: Canonical and Versatile Exchange Format	101
9.2	OWL: Schema-Level Semantics and Reasoning	105
9.3	SHACL: Validation and Quality Assurance	110
9.4	AAS: Industrial Exchange and Digital Twin Integration	114
10	IMF Workbench	120
10.1	Introduction	120
10.2	Installation	121
10.3	Data Transformation	121
10.4	Validation and Reasoning	123
10.5	Visualization	124
10.6	External Format Export	125

IV IMF in Practice	126
11 Use Case: Gas Cooling System	127
11.1 Purpose and Scope of the Use Case	128
11.2 Use Case Overview	131
11.3 Integrating Multiple Aspects in One IMF Model	131
11.4 Querying the IMF Model: Finding the Source of a Gas Leak	138
11.5 Using Query Results to Extract System Attributes	144
11.6 IMF RDF Graph Representation	145
11.7 IMF Model OWL Representation	147
11.8 IMF Model IDO Representation	149
11.9 IMF Model AAS Representation	149
11.10 Scope, Limitations, and Common Pitfalls	149

Chapter 1

Introduction to the IMF Manual

This manual is organized into four parts that progressively develop the Information Modelling Framework from foundational concepts through formal specification to practical application:

- **Part I: Fundamentals of IMF** establishes the conceptual and theoretical foundations.
- **Part II: IMF Languages** provides the formal language specification, including definitions, structural specification, and visual notation.
- **Part III: IMF as a Framework** addresses practical implementation through serialisation formats and tooling.
- **Part IV: IMF in Practice** demonstrates IMF application through an industrial use case.

Part I: Fundamentals of IMF

Part I establishes the conceptual and theoretical foundations of the Information Modelling Framework. It introduces readers to the motivation behind IMF, its core philosophical underpinnings, and the basic language constructs that enable systematic modelling of engineered systems.

Chapter 2: Visual Terms Primer for the IMF Language This chapter serves as a high-level conceptual gateway to the Information Modelling Framework. It establishes a common vocabulary through visual terms accompanied by diagrams, distilling complex architectural concepts into accessible visual representations.

Chapter 3: Conceptual Fundament of IMF This chapter develops the conceptual foundation of IMF through a practical motivating example based on a P&ID (Piping and Instrumentation Diagram) fragment. It begins by establishing common ground with systems engineering principles—boundaries, purpose, breakdown structures, and interaction topologies. The chapter then explains how real industrial systems are described through multiple, often overlapping representations that constitute distinct “appearances” of the same physical reality. The concept of aspects is introduced as the organising principle for these appearances, with contextual parameters including information domain, modality, and interest. The chapter also covers aspect switching, cross-aspect relations, and the fundamental structuring principles of breakdown and connectivity.

Chapter 4: Core Concepts of the IMF Language This chapter presents the syntactic building blocks of the IMF language. It explains how engineering meaning is translated into IMF syntax through structural definitions, which are typed intensional expressions evaluated under specific aspects. The chapter covers contextual parameters, primitive structural definitions (elements such as Blocks, Terminals, Connectors, and

Qualities), and the composition of structural definitions through relations (including `partOf`, `connectedTo`, and `hasTerminal`). It also addresses classifiers that link IMF elements to external reference data libraries, and discusses how IMF models function as coherent collections of structural definitions. A glossary of key terms is provided for reference.

Glossary of IMF Terms **Part I** ends with a glossary for quick reference.

Part II: IMF Languages

Part II provides the formal specification of the IMF language, including precise definitions, the structural specification, and the visual notation used to represent IMF models graphically.

Chapter 5: Language Definitions This chapter serves as a reference chapter that collects all definitions introduced in the IMF language. It provides concise, precise definitions of aspect parameters, element types (Block, Terminal, Connector, Quality, Breakdown), relations (`partOf`, `connectedTo`, `hasTerminal`, `proxy`, and others), classifier types, and associated modelling constraints. The chapter includes a syntax summary, well-formedness conditions, and rules for valid IMF model construction. It is designed as a practical lookup resource for modellers who need quick access to IMF vocabulary without delving into formal semantics.

Chapter 6: Structural Specification This chapter presents the formal IMF ontology and language specification. It describes the reserved vocabulary used to express facts about the target domain, the axioms that govern correct use of the vocabulary, and the logical foundations that underpin IMF models. The chapter covers identifiers, IMF models as collections of facts, types and templates for defining reusable patterns, and the formal rules that ensure models are semantically consistent. This chapter provides the rigorous logical foundation for IMF as a framework for reasoning about engineering meaning.

Chapter 7: Visual Graph Notation This chapter specifies the visual language for representing IMF models as graphs. It defines how elements are rendered as nodes (with different shapes for Blocks, Terminals, Connectors, and Qualities), how relations are rendered as edges, and how aspects are indicated through fill colours. The notation covers groups for representing breakdown hierarchies, references and metadata annotations, and conventions for types and templates. The chapter also describes available implementations of the visual notation and documents changes from previous versions.

Part III: IMF as a Framework

Part III addresses the practical implementation of IMF, covering the metamodel implementation, serialisation formats for data exchange, and the tooling ecosystem that supports IMF model creation, validation, and transformation.

Chapter 8: IMF Metamodel Implementation This chapter describes the concrete implementation of the IMF metamodel, bridging the formal structural specification and the user-facing serialisation formats. It presents two implementations: the normative IMF OWL Ontology and SHACL Shapes (which define the semantic and validation layer), and the emerging LinkML implementation (which serves as a single authoring source for generating multiple artefacts). The chapter explains why OWL and SHACL are used, details the ontology and shapes with examples, presents LinkML's capabilities for multi-format generation, data transformation, and documentation generation, and outlines the benefits and considerations of the LinkML approach. This chapter is essential for ontology engineers, semantic web practitioners, and tool developers who need to understand how the IMF language is implemented in concrete artefacts.

Chapter 9: IMF Serialisation Formats This chapter describes the different formats in which IMF models (as distinct from the metamodel) can be stored, exchanged, and processed. It covers four main serialisation formats: RDF (the canonical graph-based exchange format), OWL (class-based representation enabling classification reasoning), SHACL (shapes-based validation for quality assurance), and AAS (experimental

mapping to the Asset Administration Shell framework for industrial digital twin integration). Each format serves different purposes in the IMF ecosystem, from basic data exchange to formal reasoning and validation.

Chapter 10: IMF Workbench This chapter introduces the IMF Workbench, a tooling environment for working with IMF models. It covers installation procedures, data transformation capabilities for converting between formats, validation and reasoning features using SHACL and OWL, visualization tools for rendering IMF models graphically, and external format export functionality. The IMF Workbench provides and orchestrates tools for practitioners who need to create, edit, validate, and share IMF models in real projects.

Part IV: IMF in Practice

Part IV demonstrates the practical application of IMF through a detailed, realistic use case that illustrates how the framework can be applied to solve real engineering problems.

Chapter 11: Use Case: Gas Cooling System This chapter presents a comprehensive use case based on a gas cooling system. It demonstrates how IMF can integrate functional, physical, and spatial system information across multiple aspects within a single model. The chapter walks through the process of querying the IMF model to answer practical questions (such as tracing the source of a gas leak), extracting system attributes from query results, and generating different representations of the model (RDF, OWL, IDO, and AAS). The chapter concludes with important guidance on scope and limitations, and practical recommendations for successful IMF adoption in real projects.

Part I

Fundamentals of IMF

Chapter 2

Visual Terms Primer for the IMF Language

Abstract

This chapter serves as a high-level conceptual gateway to the Information Modelling Framework (IMF). Before diving into the conceptual fundamentals and core languages in later chapters, this primer establishes a common vocabulary accompanied with visualisations. It distils complex architectural concepts into "primitive" visual terms, allowing readers to build a mental model of how IMF describes and connects engineered systems.

Intended readers This primer is intended for all stakeholders interacting with IMF models, including:

- **System Architects and Engineers** who need to understand the fundamental building blocks of the framework.
- **Information Managers** looking for a clear way to communicate data requirements across different domains.
- **Software Developers** requiring a conceptual map before implementing IMF-compliant tooling.
- **Decision Makers** seeking a "lightweight" understanding of how IMF provides traceability across a system's lifecycle.

Terms

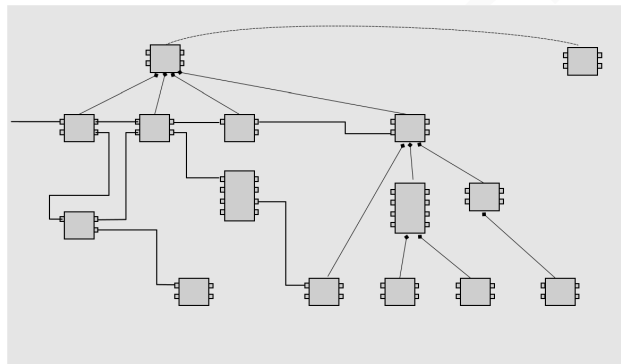
Context: Determines the interpretive framework or stance, the background and setting in which the meaning of something is understood.

Context parameters: Parameters that define the actual context.



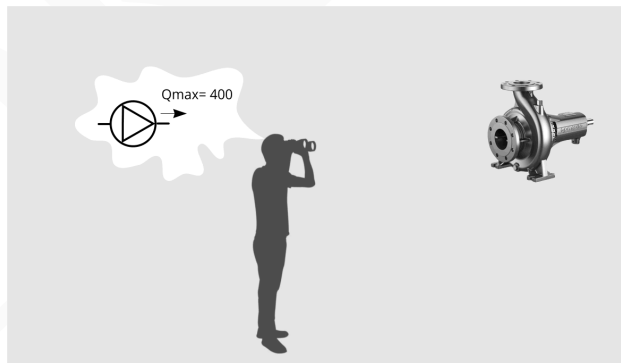
The diagram shows how choosing Category, Modality, and Interest sets the context for interpreting the meaning. In this example, the meaning is the capability intended for the pump with regard to process interest. The conventional diagrammatic view is shown alongside.

Structure: A composition of elements and relations.



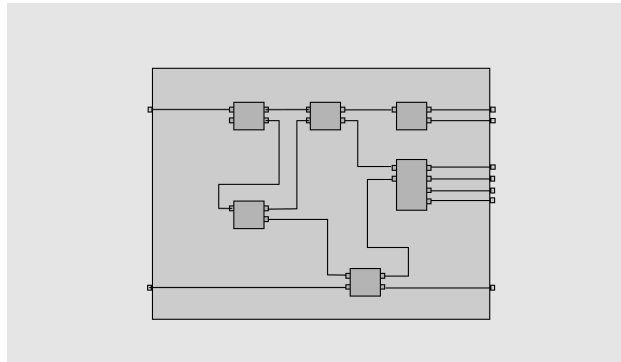
The diagram shows how elements (blocks, terminals, connectors) can be composed in a structure using relations.

Appearance: The way something shows up—what can be known, not what is.



The diagram shows how something is perceived and understood by an observer, rather than describing the thing's intrinsic nature.

System: A set of interconnected and interacting elements, each of which can be a system in itself.

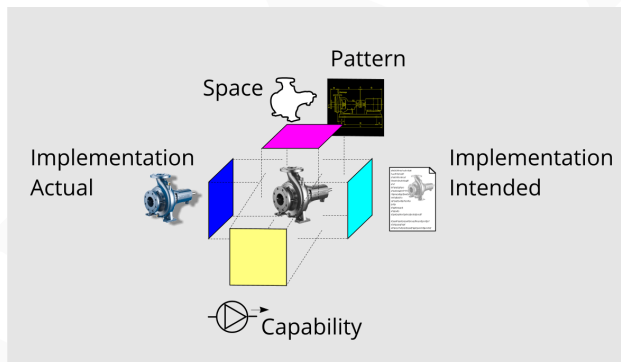


The diagram shows a system with inputs and outputs, where smaller systems are contained within the boundary, representing a system of systems.

Aspect: Category + Modality.

Category: Determines what kind of entity the system appears as for the purpose of interpretation (Capability, Implementation, Space).

Modality: Determines the status of that appearance (Intended, Actual, Pattern).



The diagram shows how aspects give rise to different information about the same thing. Pattern is special in that it represents information that is reused for multiple things of identical or similar kind. For practical purposes, some of the aspects have been given names:

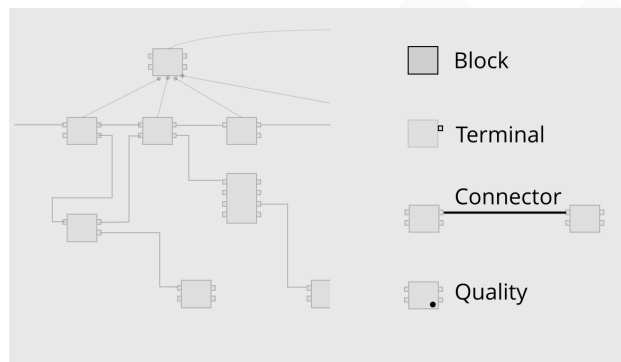
Modality \ Category	Capability	Implementation	Space
<i>Intended</i>	Function	Product	Location
<i>Actual</i>	Performance	Installed	Location As Built
<i>Pattern</i>			

Interest: A focus on a delimited or filtered part of the information within one aspect.



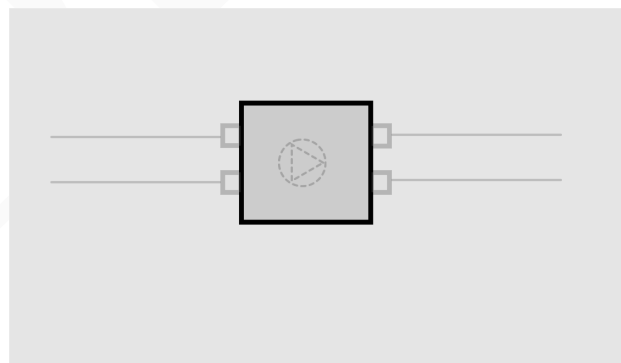
The diagram highlights that distinct interests dictate specific information needs. In the Installed aspect, for instance, the required information depends entirely on the observer's objective.

Elements: Fundamental units that constitute the basic structural definitions of a system appearance.



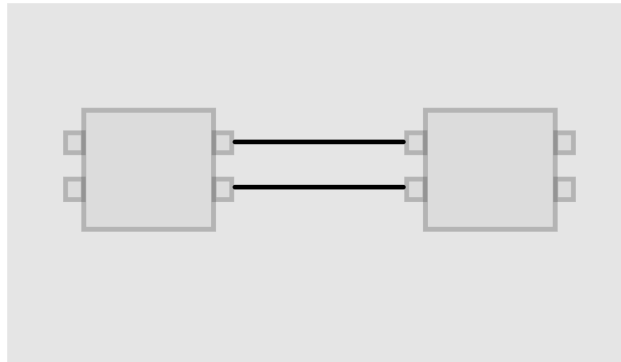
A larger structure is formed by the combination of these elements and their relational bindings. This illustration shows a high-level structure and its supporting legend; internal relational details, such as Terminal-to-Block bindings, are omitted for clarity at this level of abstraction.

Blocks: Elements expressing a system unit with a boundary.



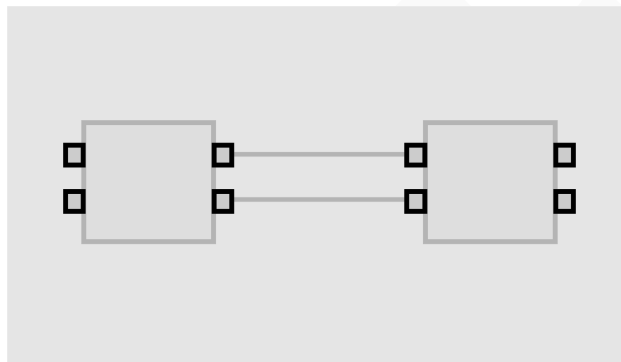
A Block is shown in context—with some Terminals and Connectors—as a boundary around a system, in this example a pump system. The Block carries the structural definition of this system and its boundaries. Note that the dotted pump symbol is provided for explanatory clarity only, and is not part of the structural definition.

Connectors: Elements expressing an interaction system.



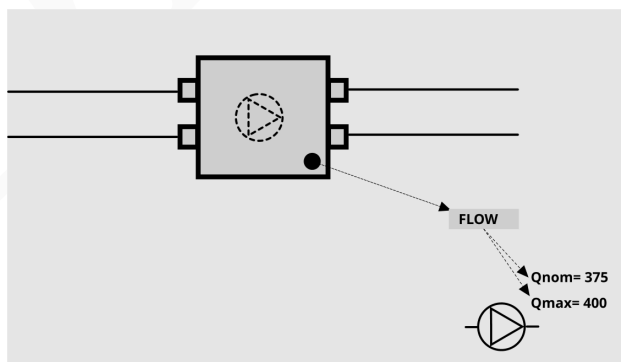
Two Connectors are shown together with two Blocks and some Terminals to illustrate how Connectors define the media flow between two systems.

Terminals: Elements expressing an interface condition.



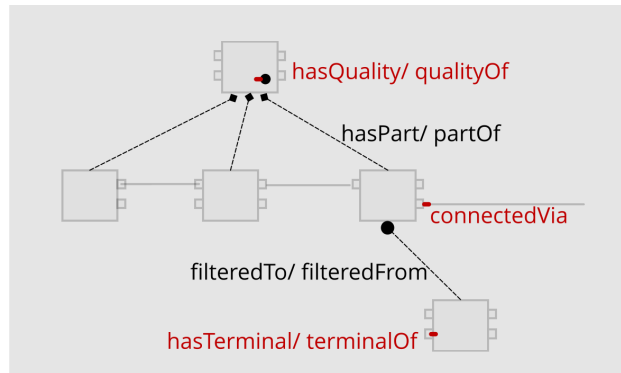
Terminals are shown together with Blocks and Connectors. A Terminal explicitly marks the point where an interaction traverses a boundary. It serves as the logical bridge through which a Connector interfaces with a Block.

Qualities: Elements expressing quantifiable properties that refine the interpretation of elements by means of descriptive or constraining information.



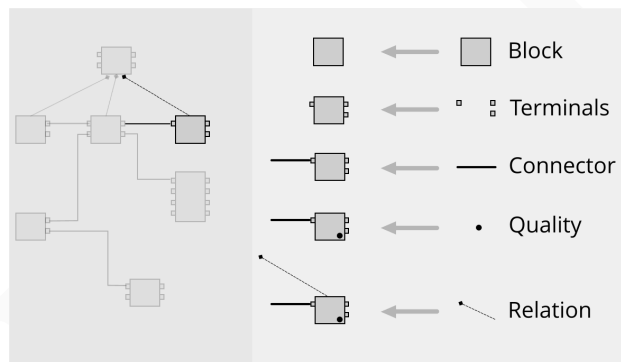
The figure demonstrates how the Quality serves as the mechanism for associating requirements, measurements, and parameters with system elements. In this instance, the Quality 'Flow' is associated with a Block; Qualities can also be applied to Terminals and Connectors.

Relations: Express dependencies that facilitate the composition of individual Elements into a unified system model.



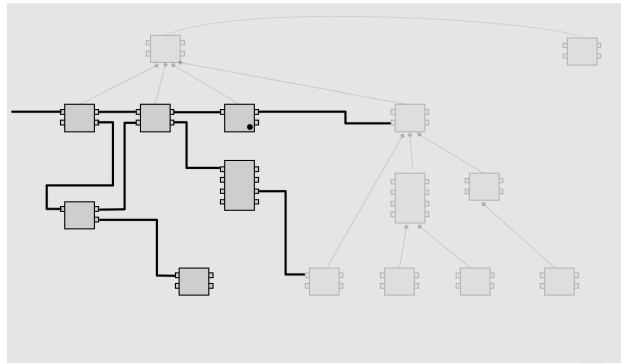
The relations in black are managed by the engineer user. The relations in red represent the underlying structural logic. While these are typically hidden from the engineers to reduce visual complexity, they are implicitly established whenever an engineer composes a structure using Terminals, Connectors, or Qualities.

Composition: Defining and relating elements to build a system appearance/model given a context/aspect.



The diagram shows how composition is the activity of building structural definitions of a system. The workflow progresses from defining Blocks and their Terminals to establishing connectivity via Connectors. The model is further refined by attaching Qualities and organising elements into a hierarchy using hasPart/ partOf relations.

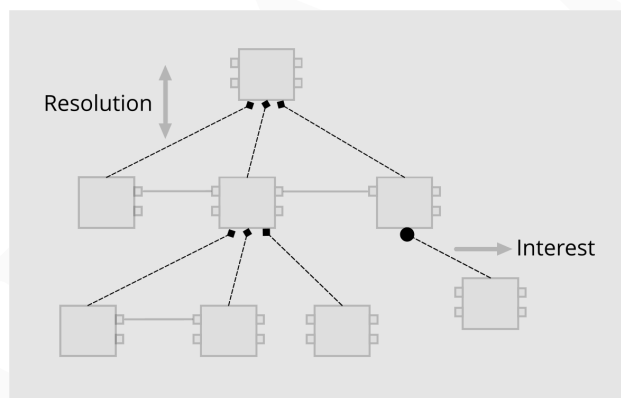
Topology: The structural definition of media connectivity between system Blocks.



The diagram shows how different systems are connected by media. This is captured through the coordinated use of Terminals and Connectors, where the Terminal specifies the interface condition of a system (Block), and the Connector specifies an system interaction that satisfies these conditions. The interaction is achieved via media exchange.

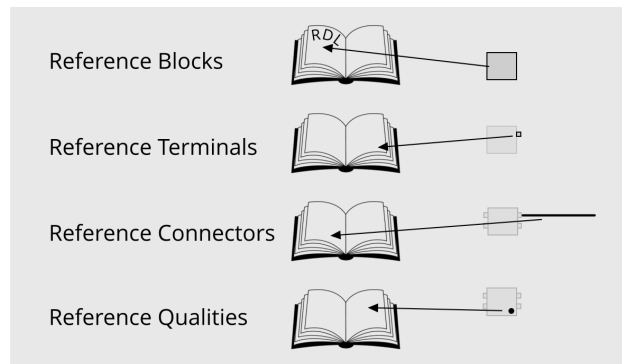
Hierarchy: Structural resolution and scope of interest.

1. Resolution—breaking down into finer resolution or aggregating up to a higher level.
2. Scope of Interest—applying filtering for Interest that persists through further breakdown.



This diagram demonstrates how descriptions are refined by increasing structural resolution. A system represented as a single unit at one level can be decomposed to reveal internal components, sub-functions, and sub-interfaces. It also shows how specific interests are introduced, for instance, transitioning from a general view to a specialised safety view.

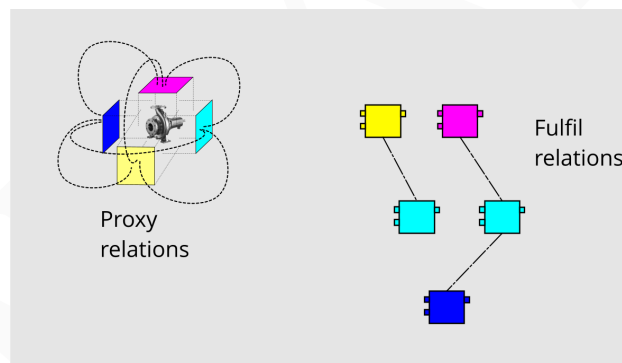
Classifier: A mechanism for associating elements with external reference data to ensure that the interpretation adheres to the constraints specified.



This diagram illustrates how a Classifier links an element to reference data libraries (RDL). This association assigns functional or structural roles to Blocks, specifies the media for Terminals and Connectors, or defines quantitative constraints, such as pressure or temperature, for Qualities. Furthermore, Classifiers can tag elements with specific terminology to support contextual specialisation, such as defining an Interest.

Cross-aspect relations: Define dependencies between distinct system appearances.

1. **Proxy:** States that different appearances of a 'thing' refer to the same object.
2. **Fulfil:** States a satisfaction relationship, where a definition in one aspect serves as a solution to a requirement defined in a different aspect.



The diagram demonstrates how the Proxy relation establishes direct correspondence between different appearances of the same system entity. The Fulfil relation defines a conditional link where one aspect satisfies the requirements of another, such as an 'Intended Implementation' fulfilling an 'Intended Capability.' The specific criteria for fulfilment are case-dependent and governed by engineering validation rules.

Chapter 3

Conceptual Fundament of IMF

Abstract

This chapter establishes the conceptual foundations of the Information Modelling Framework (IMF). It introduces the idea that engineering descriptions express *appearances* of systems rather than direct assertions about what exists, and that such appearances are always interpreted under explicit contextual conditions. To make these conditions precise, the chapter also introduces *aspects* as structured configurations of contextual parameters, clarifying the roles of modality, category, and interest in engineering reasoning. This framework explains how the same system may be described in multiple, non-equivalent but coherent ways across lifecycle stages, disciplines, and purposes. The concepts developed here provide the semantic grounding for the IMF language introduced in subsequent chapters and motivate its emphasis on explicit context, intensional structure, and incremental modelling.

Intended readers This chapter is written for engineers, architects, domain experts, and researchers who need a clear conceptual understanding of how IMF represents engineering meaning across contexts. It is intended for readers who work with complex technical systems and require a principled vocabulary for reasoning about system structure, appearances, and interpretation, independent of any specific data format or tool. The chapter assumes no prior familiarity with IMF, but it does assume experience with engineering modelling, systems thinking, or technical documentation such as P&IDs. It is suitable both as an entry point to IMF and as a conceptual reference for readers who later engage with the formal language definitions and implementations.

3.1 Introduction

This chapter introduces the conceptual foundations of the Information Modelling Framework (IMF). The purpose is to establish a precise semantic vocabulary for explaining what an IMF model expresses and how such expressions are to be interpreted. Throughout, we use a small and familiar engineering setting as a running reference: the piping and instrumentation diagram (P&ID) fragment in [Figure 3.1](#), showing a pump circulating a cooling medium through a heat exchanger. In this chapter, the P&ID is used as an example of an *engineering appearance*: what engineers take such an artefact to convey under shared disciplinary assumptions.

The foundational notion in IMF is that of a *system appearance*. A system appearance is a way a system is presented, understood, and articulated within a modelling context. IMF models are concerned with

such appearances: they make explicit the structural content that engineers attribute to a system in a given context, and they provide a basis for comparing, integrating, and interpreting that content across contexts and stakeholders.

A central role is played by the notion of an *aspect*. In IMF, an aspect is defined by a *modality* and a *category*. The category determines what kind of entity the system appears as for the purpose of interpretation (capability, implementation, or spatial entity), while the modality determines the status of that appearance (intended, actual, or pattern). Fixing an aspect makes the interpretive conditions explicit and supports a disciplined understanding of what it means for different modelling statements to concern the same appearance.

The conceptual framework developed in this chapter rests on three pillars:

1. **Systems thinking:** engineered systems are understood in terms of boundaries, decomposition, interfaces, and interaction.
2. **Appearances:** systems are encountered and communicated through representations and practices that convey structured content under shared assumptions.
3. **Aspects:** interpretive conditions are made explicit by fixing the modality and category under which an appearance is to be understood.

These foundations prepare for [Chapter 4](#), which introduces the IMF modelling constructs and explains how they are used to express and refine the structural content of system appearances.

Chapter roadmap. The remainder of this chapter develops the conceptual foundations of IMF in a structured progression. We begin by introducing a concrete P&ID fragment as a motivating example, which serves as a recurring reference point throughout the chapter. We then revisit core systems-engineering notions—such as system boundaries, decomposition, interfaces, and interaction—to establish shared intuitions about how engineers reason about systems. These intuitions are subsequently reframed in terms of *system appearances*: structured ways in which systems are presented, understood, and communicated through engineering artefacts. The chapter then introduces *aspects* as the mechanism by which the interpretive conditions of such appearances are made explicit, by fixing modality and category. Finally, the chapter shows how aspects support systematic comparison, switching, and structuring of appearances, preparing the ground for the formal modelling constructs introduced in the next chapter.

3.2 A motivating example: a P&ID fragment

To ground the discussion, we consider the piping and instrumentation diagram (P&ID) shown in [Figure 3.1](#). The diagram depicts a simple cooling system in which a circulation pump delivers cooling medium to one or more heat exchangers, with a tank providing buffer volume and instrumentation monitoring levels and temperatures. This example will be revisited throughout the chapter to illustrate how IMF interprets and structures such engineering appearances.

In the P&ID, physical objects and instruments are identified by tags that follow a project-wide convention. The initial letter **A** designates the installation area or asset. The number **40** identifies a tag group associated with a cooling service within that area. The functional part of the tag (**PA** for pumps, **HJ** for heat exchangers, **TA** for tanks) identifies the equipment class according to the project's tagging standard, and the following digits specify a unique item within that group. Thus, **A40PA103** refers to a specific circulation pump in the cooling system, while **A23HJ105** refers to a heat exchanger belonging to a neighbouring system that interacts with the cooling service across a defined boundary. Instrumentation tags such as **TIC 002**,

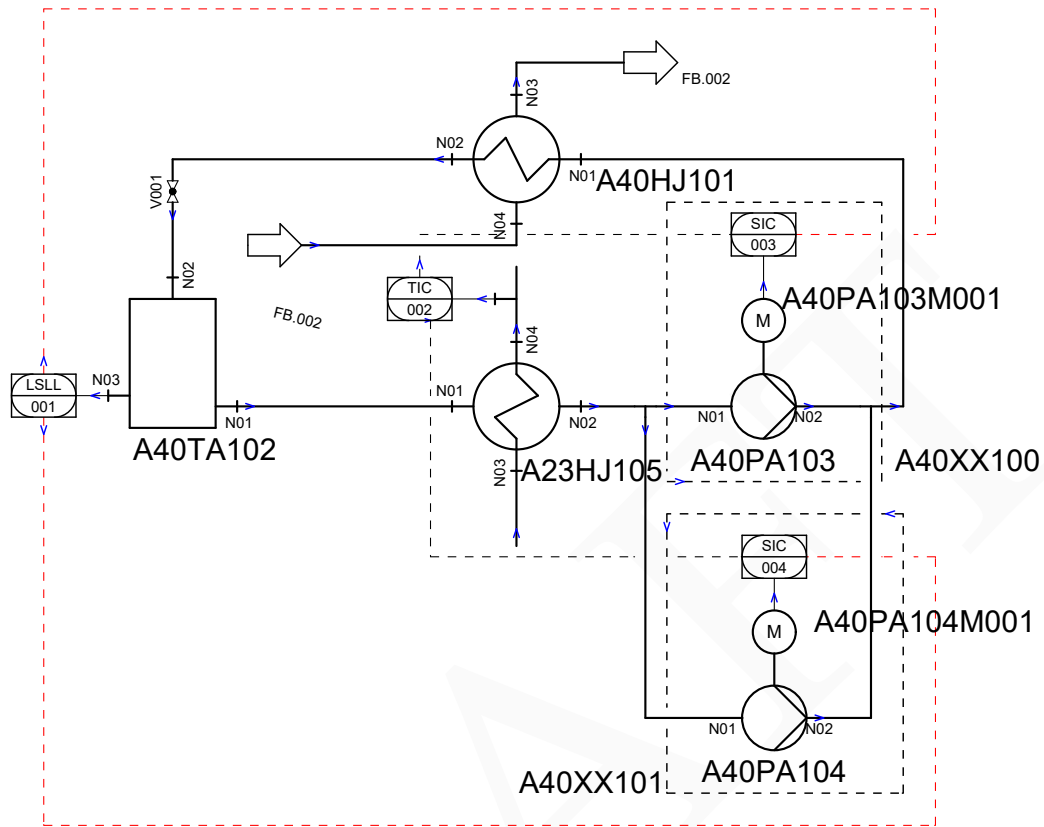


Figure 3.1: Simplified P&ID fragment: a circulation pump delivering cooling medium to a heat exchanger. (Illustrative only.)

LSLL 001, and SIC 003 follow established ISA conventions indicating the measured variable and the instrument's functional role.

The diagram presents the cooling system as a set of interconnected subsystems, each with clearly identified interfaces:

- **Circulation subsystem:** the pumps **A40PA103** and **A40PA104**, which provide the driving force for the cooling medium.
- **Heat-exchange interfaces:** the heat exchangers **A23HJ105** and **A40HJ101**, which connect the cooling service to neighbouring systems.
- **Storage and buffering:** the tank **A40TA102**, which provides surge volume and stable suction conditions.

Interfaces between these subsystems are indicated by nozzles (such as **N01–N04**), which mark the points where the cooling medium crosses subsystem boundaries. The process lines connecting them express intended flow paths, flow direction, and the order in which subsystems interact.

Although the P&ID is a single diagram, it implicitly combines contributions from several engineering disciplines:

- **Process engineering**, through the functional storyline in which pumps circulate a medium to remove heat via the heat exchangers.
- **Instrumentation and control**, through measurement and control functions such as level monitoring (LSLL 001) and temperature control (TIC 002).
- **Mechanical and piping engineering**, through the explicit representation of physical connectivity, nozzles, and pipeline segments that realise the cooling service.

Additional assumptions from electrical, safety, and automation disciplines are also embedded, for example through instrumentation conventions and interface points to control and shutdown logic. These disciplinary contributions coexist in the diagram without being explicitly separated.

The P&ID fragment therefore does not merely depict a cooling loop. It is a composite engineering representation that blends functional intent, physical realisation, control logic, and system boundary information. Each symbol, tag, and connection is an entry point into a larger body of engineering understanding. When IMF speaks of *system appearances*, diagrams such as this P&ID are precisely the kinds of appearances that are meant: structured and purposeful representations that foreground certain system features while leaving others implicit, depending on the context in which they are interpreted.

In the sections that follow, we introduce a precise vocabulary for these contextual conditions and show how they govern the interpretation of system appearances. IMF organises such conditions into *aspects*, which make explicit the assumptions under which a diagram like [Figure 3.1](#) is read and understood.

3.3 Systems Thinking: Establishing Common Ground

Before turning to the notions of *appearance* and *aspect*, it is useful to recall a set of shared systems-engineering intuitions that form common ground for most readers. Engineers typically understand a *system* as a grouping of objects that interact to achieve a purpose. The Systems Engineering standard ISO/IEC/IEEE 15288 defines a system as “a combination of interacting elements organized to achieve one or more stated purposes.” This definition captures a widely shared engineering intuition: a system is understood in terms of elements, boundaries, interactions, and purpose. At the same time, it does not commit to a single, uniquely determined system structure.

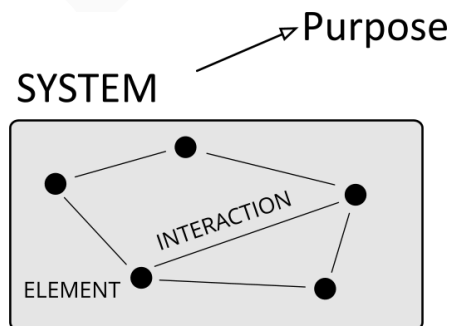


Figure 3.2: The system concept from ISO/IEC/IEEE 15288, illustrated as a group of elements interacting within a boundary to achieve a purpose.

These systems-engineering intuitions are directly visible in the P&ID fragment introduced earlier. At first glance, one might be tempted to identify the “cooling system” shown in the diagram with *System 40*, since many component tags begin with **A40**. This interpretation, however, would conflate a naming convention with a systems-engineering concept. The prefix **A40** denotes a tag group associated with a cooling service in the project’s code manual; it may encompass thousands of tagged items across the asset, without implying a coherent internal system structure. What the P&ID presents is not “System 40” as a tag group, but a specific cooling system in the engineering sense: a coherent arrangement of interacting subsystems selected for a particular purpose and level of description.

3.3.1 Purpose and transformation

A central intuition in systems thinking is that of *purpose*. In engineered systems, purpose is commonly understood in terms of a transformation: a change in the state or flow of a medium. In the P&ID fragment, pump **A40PA103** raises pressure and drives the cooling medium toward the heat exchangers; heat exchangers **A23HJ105** and **A40HJ101** transfer heat between media; and tank **A40TA102** stabilises system behaviour by shaping the dynamics of the fluid level.

Although a P&ID focuses on connectivity and instrumentation, its functional purpose is implicit in the way components are arranged. A process engineer reading the diagram immediately recognises which flow paths realise which transformations, even though the functional model is not represented explicitly. In this sense, functional intent and physical representation are already interwoven in the diagram.

3.3.2 System elements and breakdown

Another recurring intuition is that a system may itself be understood as composed of subsystems. A system element can therefore be regarded as a system in its own right, with its own boundary, internal structure, and purpose. This recursive organisation is commonly referred to as a *system breakdown*.

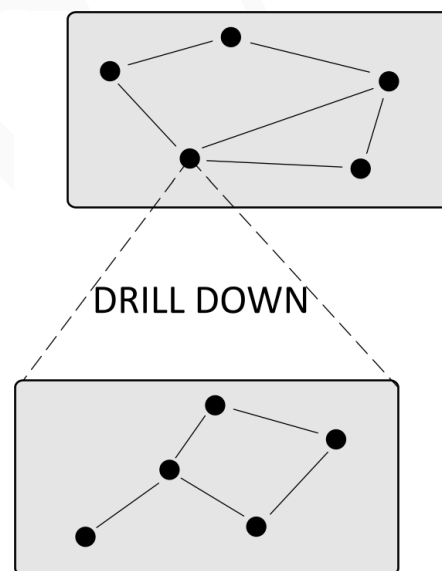


Figure 3.3: Illustration of a system breakdown. System elements may themselves be regarded as systems with their own elements.

The P&ID fragment illustrates this clearly. The cooling system can be understood as comprising:

- a circulation subsystem consisting of pumps **A40PA103/A40PA104** and tank **A40TA102**,
- two heat-exchange subsystems, **A23HJ105** and **A40HJ101**.

Each of these subsystems could, in turn, be broken down further. A pump may be decomposed into motor, impeller, shaft, bearings, and housing; a heat exchanger into tube bundles and shells; a tank into internal structures and mounted instrumentation. None of these decompositions appears in the P&ID, yet all are legitimate within their respective engineering contexts.

Different stakeholders apply different breakdown principles. A pump vendor decomposes **A40PA103** into mechanical and hydraulic components; a process engineer treats it as a single unit contributing to a flow function; a maintenance engineer focuses on replaceable subcomponents; an automation engineer decomposes it into control signals and shutdown functions. These are different ways of structuring the same system for different engineering purposes.

3.3.3 System interaction and topology

System behaviour emerges from interactions between system elements. These interactions are often described in terms of flows of media, such as fluid, heat, electrical power, information signals, or force. In the cooling system fragment, cooling medium flows from tank **A40TA102** through pump **A40PA103** via nozzles **N01** and **N02**, and onward to the heat exchangers. The lines in the P&ID encode this connectivity, expressing a flow topology without committing to physical geometry or layout.

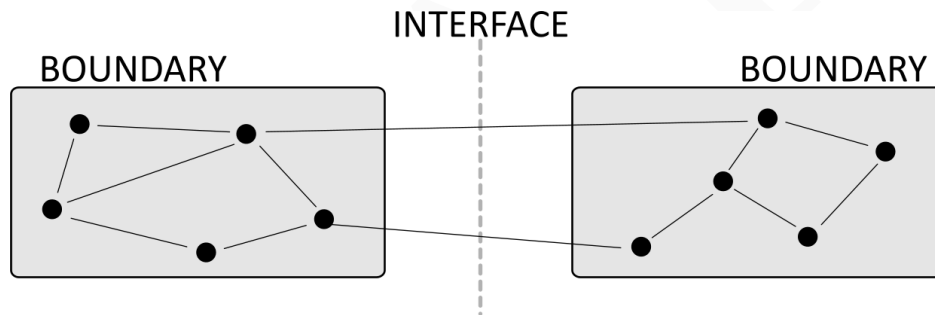


Figure 3.4: Illustration of interactions within and across system boundaries.

Other interaction structures exist in parallel. An instrumentation topology is visible through level switches, controllers, and signal paths. A mechanical interaction structure exists through couplings and supports, though it is largely omitted from the P&ID. Each topology captures a particular connectivity structure that is meaningful within a given engineering discipline and lifecycle stage.

3.3.4 Systems as structuring principles

Systems thinking provides powerful principles for structuring technical information. Breakdown structures allow engineers to navigate from large systems to smaller components; interaction topologies explain how system-level behaviour arises from local interactions; boundaries make it possible to reason about inputs and outputs. These principles scale across levels of detail: the same patterns recur whether one is reasoning about a pump, a process unit, or an entire facility.

The P&ID fragment is a small but clear example of this scalability. It exhibits boundaries, interfaces, breakdowns, and interaction topologies that mirror those found at much larger scales, enabling engineers to reason effectively about complex systems using representations of manageable size.

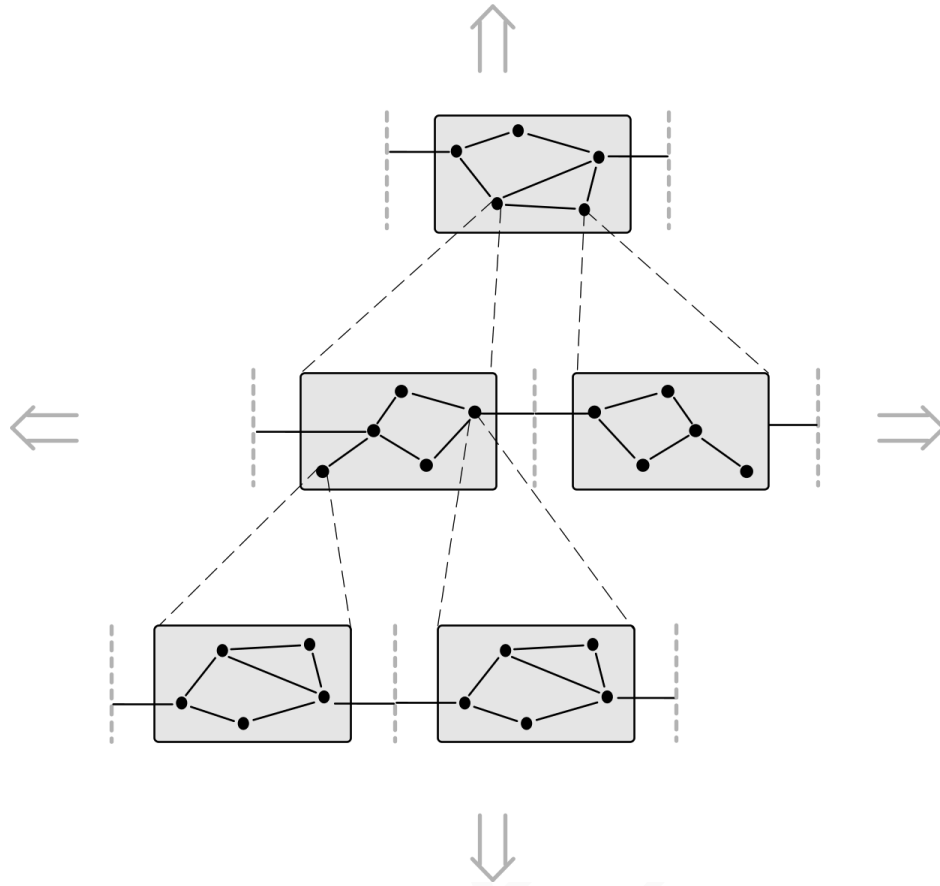


Figure 3.5: Illustration of breakdown and interaction structure across scales.

3.3.5 From systems to appearances

The observations above lead to an important conclusion. Engineering practice does not present us with a single, all-encompassing description of a system. Instead, we encounter technical information in many forms: drawings, documents, datasets, models, and diagrams, each produced for a specific purpose, by a specific discipline, and at a specific stage in the lifecycle of the asset.

The P&ID fragment demonstrates this clearly. It presents one configuration of a cooling system, articulated in terms relevant to piping, instrumentation, and process operation. The same physical objects appear very differently in a 3D model, a maintenance register, a control logic diagram, or a bill of materials. Each representation foregrounds certain relationships while leaving others implicit.

None of these representations can be treated as an authoritative description of “the” system. Each is a *purpose-driven appearance*, expressing structural assumptions under which its content is intended to be understood. The system itself is not given directly; it is accessed through such appearances.

This motivates a shift in perspective. Rather than assuming a single system structure to be modelled once and for all, IMF takes appearances as its primary objects of study. The guiding question is not “What is the system?”, but “What structure does this piece of technical information express, and under what conditions is that structure intended to hold?” To answer this question systematically, IMF introduces the notion of an

appearance and the mechanism by which appearances are organised and related: *aspects*. We now turn to these notions.

3.4 Appearances: how systems show up in engineering information

The shift from systems as assumed to be uniquely given to systems as they are encountered in engineering information calls for a more precise account of what an *appearance* is. In IMF, an appearance is not a mere simplification or partial view of a system. It is a structured, meaning-bearing representation produced for a particular engineering purpose, using the conventions and abstractions of a particular discipline, at a particular stage in the lifecycle of an asset. An appearance carries explicit and implicit assumptions about how its content is to be interpreted.

This idea is illustrated in Figure 3.6, which places a pump at the centre and shows four distinct appearances of that same pump in different kinds of technical information. The circled labels ①–④ identify the appearances, each surrounded by a coloured region indicating a different context of meaning.

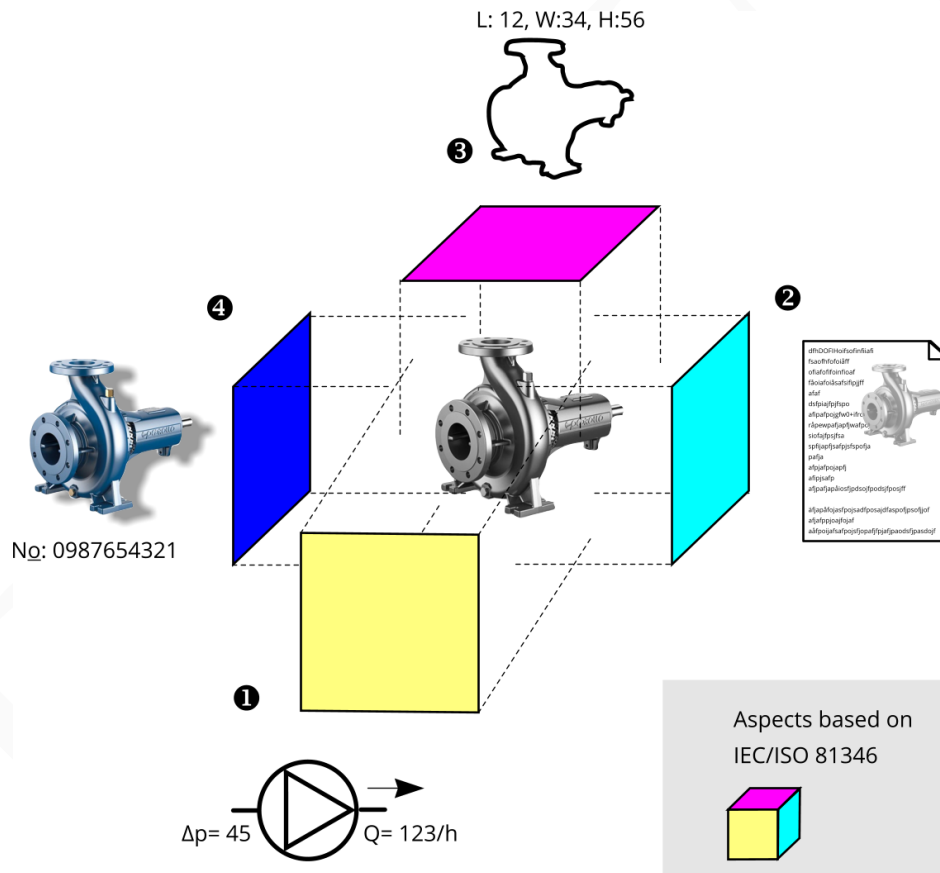


Figure 3.6: A pump with four different appearances under different aspects: ① a functional symbol in a process diagram, ② a product specification, ③ a logical or spatial location used in design, and ④ an installed item with implementation data.

Appearance ①: the pump as a functional symbol. In diagrams such as PFDs or P&IDs, the pump appears as a standardised symbol expressing a *functional contribution* within a flow path. Only features relevant to the functional storyline are shown, such as suction and discharge points, medium type, flow direction, and sometimes control logic. Many physical characteristics are deliberately suppressed. This appearance expresses how the pump participates in a function that the system is intended to realise.

Appearance ②: the pump as a specification. In datasheets, procurement documents, and vendor catalogues, the pump appears as a *product specification*. This appearance emphasises performance data, operating envelopes, materials, standards compliance, and procurement identifiers. The pump is treated not as a functional actor in a system, but as a deliverable with defined and guaranteed properties.

Appearance ③: the pump as a logical location. In many design activities, the pump appears as a component situated in a logical or spatial context. This appearance brings forward constraints related to placement, access, area classification, maintenance clearances, and safety considerations. It reflects design intent and spatial reasoning rather than functional behaviour or procurement characteristics.

Appearance ④: the pump as installed equipment. When the pump is installed and operating in the facility, it appears as an *asset* with a serial number, operating state, condition monitoring data, and maintenance history. This appearance expresses facts about the actual implementation of the pump in the running plant and is central to operations and maintenance.

All four appearances refer to the *same* pump, yet they differ substantially in structure, purpose, and assumptions. Here, “the same pump” does not denote an entity given independently of its representations, but a stable engineering referent whose identity is maintained across appearances. None of these appearances is more authoritative than the others; each is indispensable in its own context. What relates them is that they are all appearances of the same system, understood as a system with internal structure and external interactions.

3.4.1 The logic of appearances

When IMF is described as a logic, this refers to a specific claim about engineering information. Engineering knowledge is distributed across many forms—diagrams, documents, databases, 3D models, and simulation results—each capturing only part of what engineers intend to express about a system. Everything engineers know about a system is known through such appearances.

IMF takes this as its starting point. Appearances are the primary constituents of engineering meaning, and each appearance expresses a partial but purposeful understanding of a system. IMF therefore asks:

What structure underlies an engineer’s intention when a system is described? How is that structure composed, how does it connect, and how is it classified?

To answer these questions, IMF is guided by three principles:

1. **Systems are understood as systems.** The pump at the centre of [Figure 3.6](#) is not merely a symbol or a line item; it is understood as a system with parts, interfaces, behaviours, and constraints.
2. **Systems are known through their appearances.** The same pump shows up differently depending on discipline, purpose, and lifecycle stage. These differences are systematic and context-dependent.
3. **Appearances are interpreted relative to aspects.** Each appearance is associated with explicit conditions that determine how its structural content is to be interpreted. IMF formalises these conditions as *aspects*.

Together, these principles constitute the logic of IMF. IMF is a logic of how systems appear in engineering practice and how those appearances can be related in a principled way.

3.4.2 Appearances in the P&ID fragment

The P&ID fragment discussed earlier provides a concrete counterpart to the abstract example above. Pump **A40PA103** appears in the P&ID as a functional participant in a cooling loop. In a datasheet, it appears as a procured item with specified characteristics. In a 3D model, it appears as a spatially located component with access and mounting constraints. In a maintenance system, it appears as an installed asset with an operational history. Each appearance expresses a different slice of engineering understanding, and none can be reduced to the others.

This plurality is not a deficiency to be eliminated; it is a fundamental feature of engineering practice. IMF accepts the coexistence of multiple appearances and provides a language for describing them explicitly and relating them systematically.

In the next section, we introduce the mechanism IMF uses to organise and distinguish appearances: *aspects*. Aspects make explicit the contextual conditions under which an appearance expresses system structure.

3.5 Aspects: the organising principle for appearances

Engineering practice presents the same engineered system through many forms of technical information: PFDs, P&IDs, datasheets, 3D models, hazard analyses, control logic diagrams, and maintenance records. These are not alternative visualisations of a single authoritative model. They are distinct *appearances*, each governed by conventions and assumptions that determine how its content is to be interpreted.

While the notion of appearances captures the diversity of how systems are represented in engineering practice, it does not by itself provide a formal way to distinguish the interpretive conditions under which these representations are meant to be understood. Without such a mechanism, differences in lifecycle stage, disciplinary perspective, or organisational purpose risk being conflated, even when the underlying structural content is similar. Engineering artefacts may therefore appear comparable or compatible while in fact being intended to express different kinds of meaning.

IMF addresses this by introducing the notion of an *aspect*. An aspect makes the interpretive conditions of an appearance explicit and machine-processable. It does not add structure to an appearance, nor does it introduce an additional representation layer. Rather, an aspect specifies the conditions under which the structural content expressed by an appearance is evaluated. By making these conditions explicit, IMF enables contextual meaning to be stated, compared, and reasoned about, rather than remaining implicit in disciplinary practice.

In the following sections, aspects are introduced in detail and shown to provide the organising principle by which appearances are distinguished and related within IMF.

3.5.1 Definition of an aspect

An *aspect* is a semantic abstraction that fixes the conditions under which an appearance is interpreted. An aspect does not introduce structure into an appearance; it determines how whatever structure is present is to be read. In IMF, an aspect is determined by two parameters:

- **Modality:** whether the appearance is read as describing what is *actual* in the realised asset, what is *intended* as design intent or requirements, or what is *pattern* knowledge intended for reuse.

- **Category:** what kind of entity the system appears as for the purpose of interpretation: *capability*, *implementation*, or *space*.

These parameters suffice to distinguish major interpretive contexts in which the same system may appear across lifecycle stages and engineering disciplines. Multiple aspects may coexist without contradiction: they are alternative interpretive conditions under which a system is understood through different appearances.

Interest. Engineering work is also shaped by concerns such as process, maintenance, safety, procurement, and operations. IMF captures such concerns using *interest* as a contextual parameter of description and articulation: interest governs which distinctions are made explicit and which breakdowns or interaction structures are brought into focus. Interest is introduced later, together with IMF language constructs, in [Section 4.2](#).

Why aspects are needed

Similar-looking engineering representations can carry different meanings. A P&ID, for instance, may be used as a design document expressing intended constraints, as an operator-facing depiction of an installed plant, or as a reusable template expressing pattern knowledge. From a distance, these drawings may look nearly identical; what differs is the meaning assigned to them under a chosen aspect.

Aspects allow IMF to embrace the diversity of engineering information without collapsing it into a single view. By making modality and category explicit, IMF prevents unintended mixing of appearances that belong to different lifecycle situations or express different kinds of engineering organisation. Aspects therefore provide the semantic foundation on which IMF can later define a disciplined language for describing appearances and relating them.

3.5.2 Aspect parameters in more detail

Modality. Modality captures the status of an appearance: what sort of engineering claim or constraint the appearance is intended to convey. IMF distinguishes three modalities:

- **Actual** — the appearance is read as describing the realised asset or its current configuration, as established by evidence or operational information.
- **Intended** — the appearance is read as describing design intent, requirements, or planned configurations, which may or may not be realised.
- **Pattern** — the appearance is read as expressing typical or reusable engineering knowledge, independent of a particular installation.

Even when two diagrams look identical, their meaning differs depending on modality: a pattern expresses reusable knowledge; an intended diagram constrains the space of admissible solutions; and an actual diagram documents realised state. In IMF, this divergence is captured by explicitly fixing modality in the aspect.

Category. The category specifies what kind of engineering organisation the appearance is primarily meant to express. IMF reserves three categories:

- **Capability** — functional behaviour, transformations, and what the system does or is intended to do.
- **Implementation** — physical realisation: equipment, components, materials, and their properties.
- **Space** — spatial or logical placement: layout, access, routing, and location-dependent constraints.

In practice, engineering representations often blend categories. A P&ID, for example, is a piping diagram that also conveys functional flow intent and may presuppose spatial constraints. IMF requires a *primary* category choice for interpretation, not to deny such blending, but to make explicit which category governs evaluation in a given context. The same representation can therefore support multiple legitimate readings, provided the aspect is made explicit.

3.5.3 Canonical aspects

Aspects in IMF are defined semantically, in terms of the contextual parameters *Modality* and *Category*. Certain combinations of these parameters recur across engineering practice and are useful as named reference points. The names introduced below are semantic anchors for explanation; they are not modelling prescriptions.

Function aspect (Intended Capability). Under the *Function aspect*, a system appears as a capability constraint: the appearance expresses intended or required behaviour, what transformations the system should support, and what functional roles it is expected to fulfil.

Performance aspect (Actual Capability). Under the *Performance aspect*, a system appears as capability in terms of realised or observed behaviour. The modality shifts the interpretation from constraint to realised function, separating function as requirement from function as behaviour.

Product aspect (Intended Implementation). Under the *Product aspect*, a system appears as an intended physical realisation. The appearance emphasises structure, composition, materials, and technical characteristics as specified or required.

Installed aspect (Actual Implementation). Under the *Installed aspect*, a system appears as a realised physical system: what is actually installed and operated, including serialised equipment, configuration state, and maintenance-relevant properties.

Location aspect (Intended Space). Under the *Location aspect*, a system appears as an intended placement in space. This includes logical or physical location, access constraints, spatial relationships, and layout assumptions as part of design intent.

As-built location aspect (Actual Space). Under the *As-built location aspect*, a system appears as an actual spatial placement, as captured in as-built documentation, verified layout models, or 3D scans.

These six canonical aspects cover common forms of system appearance encountered across disciplines and lifecycle stages. IMF does not require that every appearance be assigned one of these names; they provide stable reference points for interpretation and explanation.

Relation to ISO/IEC 81346 (RDS). The term *aspect* is also used in the reference designation system (RDS) defined in ISO/IEC 81346, which distinguishes Function, Product, and Location aspects for organising reference designations. These ideas were an early inspiration for IMF, but the purposes differ.

RDS aspects are naming perspectives, concerned with designation: how objects are labelled and organised in documentation. IMF aspects are interpretive conditions, concerned with appearance: how engineering information is to be interpreted semantically.

There is nevertheless a correspondence: RDS Function aligns with IMF *Capability*, RDS Product with *Implementation*, and RDS Location with *Space*. IMF generalises this scheme by making *modality* explicit, thereby distinguishing intended and actual appearances that RDS leaves implicit.

On simulation and digital twins. Simulation-based appearances and digital twins raise important questions about modality. IMF does not introduce a separate simulation modality. Instead, simulated behaviour is interpreted within existing aspects (e.g. Intended or Actual Capability), with its epistemic status expressed through contextual parameters available to the modeller. Whether simulation warrants its own modality is left open for future revisions of the framework.

3.6 Revisiting the P&ID through the lens of aspects

The P&ID fragment shown in [Figure 3.1](#) can now be reconsidered with the concept of *aspects* in place. What initially appears as a single engineering diagram can be understood as supporting multiple, distinct *appearances*, depending on the aspect under which it is interpreted. The graphical form remains the same; what changes is the meaning assigned to it.

Under the *Function aspect* (Intended–Capability), the P&ID is interpreted as expressing constraints on system capability. Pump **A40PA103** appears as a contributor to an intended flow transformation; tank **A40TA102** appears as a condition shaping system dynamics; and the heat exchangers appear as intended points of thermal interaction. Read in this way, the diagram functions as a *design constraint*: it specifies how the system is required to behave, not how it is physically realised.

Under the *Product aspect* (Intended–Implementation), the same diagram is read as expressing intended physical structure. Pumps, heat exchangers, and tanks appear as equipment units with defined boundaries; nozzles **N01–N04** appear as physical interfaces; and the lines between them express intended physical connectivity. In this aspect, the diagram constrains how the system is to be realised, rather than how it is to behave functionally.

Under the *Installed aspect* (Actual–Implementation), a diagram of the same form may be read as documenting the realised facility. The appearance now supports factual claims about what equipment is installed, how it is physically connected, and which interfaces exist in the built asset. Although the graphical form may be identical to that of an intended reading, the engineering meaning is fundamentally different: the diagram is descriptive rather than normative.

Under the *Performance aspect* (Actual–Capability), the P&ID supports a different kind of interpretation again. The appearance is read in terms of realised or observed behaviour: flows, states, and transformations that are measured, estimated, or inferred from operation. In this aspect, the diagram provides the structural context within which operational data are interpreted, even though the measurements themselves are not part of the diagram.

These readings illustrate a central principle of IMF:

The visual or notational form of a representation does not determine its engineering meaning. Meaning arises from how that form is interpreted under a fixed aspect.

A single P&ID fragment can therefore sustain multiple, equally valid appearances: functional, product-oriented, installed, and performance-related. IMF preserves the familiar diagrammatic form used by engineers, while providing a precise semantic account of how that form is to be read in each case.

By separating representational form from interpretive conditions, IMF allows multiple appearances of the same system to coexist without conflict. This prepares the ground for distinguishing, in later chapters,

between relations that link appearances across aspects and relations that structure appearances within a single aspect.

3.7 Aspect switching and cross-aspect relations

An appearance is always defined *relative to an aspect*. When engineers reason about a system in functional terms, then as a realised installation, and later in terms of observed performance, the system under discussion does not change. What changes is the *aspect* under which it is interpreted.

IMF refers to such transitions between appearances as *aspect switching*. Aspect switching is not a transformation of the system, nor a refinement of its structure. It is a change in the interpretive conditions under which the system is understood, yielding a different appearance of the same system.

Aspect switching as a change of interpretive context. Consider the pump A40PA103. Under the Function aspect (Intended–Capability), it appears as a condition constraining intended system behaviour. Under the Installed aspect (Actual–Implementation), it appears as a physical asset with a specific realised configuration. Under the Performance aspect (Actual–Capability), it appears as a system capability characterised by observed behaviour.

These appearances differ because they are interpreted under different aspects, not because they refer to different systems. Aspect switching therefore relates distinct appearances of the same system across different interpretive contexts.

Uniqueness of appearance within an aspect. A fundamental semantic constraint follows directly from the definition of an aspect:

For a given aspect, a system has exactly one appearance.

An aspect determines the contextual conditions—modality and category—under which interpretation takes place. Once these conditions are determined, there is no ambiguity about how the system appears in that context. Multiplicity arises only *across* aspects, never *within* a single one.

This constraint is essential for disciplined reasoning about engineering information. It ensures that changes in meaning are governed by explicit changes of context, rather than remaining implicit within representations.

Relating appearances across aspects. Engineering work routinely involves moving between aspects: from intended function to realised installation, from installed structure to observed performance. IMF does not attempt to collapse these appearances into a single representation. Instead, it treats their relationships as *cross-aspect relations*: semantic links that make explicit that different appearances correspond to the same system.

Such relations do not merge structures or reconcile differences. They preserve the integrity of each appearance while stating their correspondence. In this way, IMF supports coherent navigation between engineering viewpoints without erasing the contextual conditions that give each appearance its meaning.

Aspect switching therefore provides the conceptual bridge between the plurality of appearances encountered in engineering practice. It explains how engineers move coherently between functional, physical, and behavioural interpretations of the same system—by changing aspect, not by changing the system itself.

3.8 Structuring principles within an aspect

While aspect switching relates different appearances of the *same* system across interpretive contexts, the structuring principles discussed in this section operate *within* a single aspect. They organise how *different systems*—subsystems, assemblies, interfaces, and interaction carriers—are related inside one appearance.

Two such principles recur throughout engineering practice: *breakdown* and *connectivity*. In IMF, neither is treated as a property of a system in itself. Both are modelling structures imposed on a system *as it appears under a given aspect*. They depend on the interpretive conditions fixed by that aspect and are therefore context-sensitive by design.

Breakdown as aspect-constrained resolution

Breakdown captures the familiar act of introducing finer structural resolution: distinguishing subsystems, components, or roles that were previously treated as a unit.

Breakdown is always constrained by the aspect under which the system is interpreted. Under the *Function aspect* (Intended–Capability), a pump may be broken down into functional contributions such as suction, discharge, and control behaviour. Under the *Installed aspect* (Actual–Implementation), the same pump may be broken down into casing, impeller, motor, and bearings. Both breakdowns are correct, but they belong to different aspects and must not be intermixed.

Within a single aspect, breakdown provides a coherent notion of resolution: the system may be described at different levels of detail, but all such descriptions are governed by the same interpretive conditions.

Connectivity as aspect-constrained topology

Connectivity captures how systems interact: through flows, couplings, or other forms of relationship.

As with breakdown, connectivity is aspect-constrained. The same P&ID fragment supports different connectivity structures depending on the category governing interpretation. Under a *Capability* category, the topology is functional: cooling medium flows from the tank to the pump to the heat exchangers, establishing a transformation chain. Under an *Implementation* category, the topology is physical: pipes, nozzles, and equipment interfaces define how components are connected. Under a *Space* category, connectivity may be read in terms of placement or adjacency, such as co-location within a package or module.

Breakdown and connectivity are orthogonal structuring dimensions. Breakdown answers which systems are distinguished at a given level of resolution; connectivity answers how those systems interact. Both are interpreted relative to the same aspect and together constitute the structural content of an appearance.

Micro-walkthrough: breakdown and connectivity in a P&ID

The cooling loop in the P&ID fragment illustrates breakdown and connectivity side by side. The circulation subsystem—tank, pumps, and associated interfaces—forms a breakdown structure within the appearance: a higher-level cooling system resolved into interacting subsystems. At the same time, the lines between nozzles express a connectivity structure that orders these subsystems into a circulation path.

These two structures coexist without conflict. The breakdown specifies which subsystems are distinguished at this level of description; the connectivity specifies how they interact. Both are read under the same aspect and together define the organisation of the appearance.

Operational readings under different aspects

A P&ID primarily supports *Intended* appearances: it constrains how a system is supposed to function or be realised. When the same structural organisation is interpreted under the *Performance aspect* (Actual–Capability), it can serve as a carrier for operational data derived from the realised system. Flow paths may then be associated with observed rates, temperatures, or pressures, and functional units with measured behaviour.

In this way, the same structural form—interpreted under a different aspect—supports articulation of actual system behaviour without requiring a different diagrammatic representation.

Keeping the dimensions separate

The distinctions introduced so far can be summarised as follows:

Aspect switching relates different appearances of the *same* system across interpretive contexts.

Breakdown and connectivity structure appearances of *different systems* within a single context.

Within an aspect, engineers introduce resolution through breakdown and express interaction through connectivity. Across aspects, they change how the same system appears. By keeping these dimensions separate, IMF preserves the plurality of legitimate engineering structures while preventing them from being conflated.

This conceptual separation prepares the ground for the next chapter, where breakdown and connectivity are given precise expression as IMF language constructs and relations.

Summary and transition

Summary of Fundamental Concepts. This chapter has established the semantic foundations of the Information Modelling Framework. We have introduced the notion of a *system appearance* as the primary bearer of engineering meaning, and argued that systems are understood in practice only through such appearances. Appearances are always interpreted under explicit *aspects*, which fix the modality and category governing their meaning. Multiple appearances of the same system may coexist without contradiction, each corresponding to a different aspect.

Within a given aspect, appearances are structured using familiar systems-engineering principles. *Breakdown* introduces structural resolution by distinguishing subsystems, while *connectivity* describes how systems interact through flows or couplings. These principles are aspect-dependent modelling structures, not intrinsic properties of systems. Together, appearance, aspect, breakdown, and connectivity provide a coherent conceptual account of how engineering information is organised, interpreted, and related across disciplines and lifecycle stages.

Transition to Chapter 4. The concepts introduced in this chapter explain *what IMF models are about* and *under what conditions their content is to be interpreted*. They do not yet explain how such content is expressed in a model. Chapter 4 introduces the core constructs of the IMF language—elements, relations, and classifiers—and shows how they are used to express structural definitions of appearances in a precise and disciplined way. The chapter makes explicit how the semantic notions developed here are realised as modelling constructs, thereby connecting engineering meaning to concrete IMF syntax.

Chapter 4

Core Concepts of the IMF Language

Abstract

This chapter introduces the core language constructs of the Information Modelling Framework (IMF). It presents elements, relations, classifiers, and contextual parameters as the basic syntactic and conceptual building blocks used to express structural definitions explicitly. The chapter explains how these constructs are combined to form IMF models, how structural definitions are composed and refined, and how contextual parameters govern their interpretation. Emphasis is placed on the intensional character of IMF models, their support for incremental and partial descriptions, and the separation between structural articulation and realization. Together, these concepts establish the bridge between the conceptual foundations of IMF and the formal language definitions presented in later chapters.

Draft Notice *This chapter presents the core modelling constructs of IMF and their intended conceptual reading. While the overall structure and terminology are considered stable, several parts of the chapter are intentionally provisional. In particular, the treatment of cross-aspect relations (including `proxy`) and the outline of formal interpretation are incomplete and will be refined in subsequent releases. Furthermore, compositional relations are restricted in this version: notably, `partOf` is defined only for blocks and not for connectors. Support for compositional structuring of connectors is planned for next version.*

The role of Interest has been substantially revised in this version and is not backwards compatible. This change is deliberate and reflects a clarified separation between interpretive conditions (aspects) and articulation mechanisms.

This chapter establishes the conceptual contract that later formal and implementation-oriented chapters build upon; however, full formal semantics and complete tooling alignment are deferred.

Intended readers This chapter is written for practitioners and designers who need to work directly with IMF models. This includes system engineers, information architects, method developers, and advanced users who require a precise understanding of IMF elements, relations, classifiers, and contextual parameters. The chapter assumes familiarity with the conceptual foundations of IMF ([Chapter 3](#)) and is intended to prepare the reader for using IMF as a modelling language, whether manually, through visual notation, or via tooling. While the presentation remains conceptually motivated, readers are expected to be comfortable with formal distinctions and structured modelling concepts.

4.1 Introduction

This chapter introduces the core constructs of the IMF language and explains how the conceptual notions developed in [Chapter 3](#) are operationalised as concrete modelling statements. While the preceding chapter established a semantic vocabulary—covering system appearances, aspects, modality, category, and interest—this chapter shows how those notions are expressed explicitly and systematically using IMF language constructs.

The central move made in IMF is to distinguish clearly between *semantic notions* and the *language constructs* used to articulate them. Concepts such as appearance and aspect are not themselves modelling primitives. Instead, IMF introduces a small set of syntactic constructs—elements, relations, classifiers, and contextual parameters—through which the structural content of an appearance is made explicit. In particular, IMF elements serve as the primary carriers of structural content. Each element introduces a *structural definition*: a definition-like specification of how a system appearance is to be understood, constrained, and related to other definitions under a fixed aspect.

This chapter therefore marks the transition from conceptual foundations to language-level articulation. Aspects, defined in terms of modality and category, determine the interpretive conditions under which elements are evaluated. Interest constrains how that structure is articulated. Elements introduce structural definitions; relations compose and connect them; classifiers refine their interpretation without asserting realisation. Together, these constructs provide a disciplined and compositional way of expressing system appearances in a form suitable for incremental modelling, comparison across artefacts, and integration across stakeholders and data sources.

Chapter structure. The chapter proceeds in a layered fashion. It begins by revisiting the contextual parameters that govern interpretation, establishing how modality, category, and interest function at the language level. It then introduces the primitive element types used to express structural definitions, followed by the relations through which these definitions are composed into larger structures. Subsequent sections introduce classifiers and qualities as mechanisms for refinement, before characterising IMF models as collections of interrelated structural definitions. A running P&ID example is used selectively to illustrate the intended reading of the constructs, without serving as a normative encoding of P&ID semantics.

Integrative perspective. Although the chapter is organised into focused subsections, the constructs it introduces are not independent. An IMF model is always read as a whole: elements introduce structural definitions, relations establish organisation and interaction, classifiers constrain interpretation, and contextual parameters fix the conditions under which the entire configuration is evaluated. The meaning of any individual statement is therefore inseparable from the network of constructs in which it appears. This integrative view underpins the design of IMF and prepares for the formal language definitions given in [Part II](#).

4.2 Contextual parameters of structural definitions

Structural definitions in IMF are not interpreted in isolation. They express engineering meaning in a definition-like form: they specify how something is to be understood and structured, rather than asserting what exists in the world. A structural definition becomes meaningful only when interpreted relative to explicit *contextual parameters* that determine how it is to be read and evaluated.

This section introduces the contextual parameters used in IMF, clarifies their distinct roles, and explains how they steer the interpretation of structural definitions. The aim is to make explicit forms of contextualisation

that are usually left implicit in technical information—such as P&IDs, drawings, and datasheets—and to show how IMF brings these conditions under explicit control.

Structural definitions and context

An IMF element introduces a *structural definition*: a specification of boundaries, interfaces, interactions, or constraints that engineers take to be meaningful when reasoning about a system. Such a definition does not determine a single interpretation. Instead, it determines a space of admissible interpretations, which is narrowed and fixed by context.

Different contexts may therefore yield different, yet equally valid, interpretations of the same structural definition. For example, the same pump may be understood as a functional capability, a physical piece of equipment, or a spatially located object, depending on the interpretive conditions under which its structural definition is evaluated.

It is important to distinguish structural definitions from several closely related notions:

- A structural definition is *not an instance* or data record; it does not assert that a particular object exists or has been observed.
- It is *not an ontological class*; it does not define a category of things by necessary and sufficient conditions.
- It is *not a complete system model*; it may be partial, provisional, and incrementally refined.

Rather, a structural definition captures how engineers intend a system, subsystem, or interaction to be understood within a given modelling situation. Contextual parameters answer the question:

Under which conditions, and in which sense, is this structural definition to be interpreted?

By making these conditions explicit, IMF allows engineering meaning to be handled systematically, compared across artefacts, and related across disciplines and lifecycle stages without collapsing distinct interpretations into a single representation.

Aspect parameters: category and modality

The most fundamental contextual parameters in IMF are those that define an *aspect*. As established in Chapter 3, an aspect fixes the conditions under which a system appearance is interpreted.

In the IMF language, these semantic conditions are exposed explicitly as parameters associated with structural definitions. An aspect is determined by two parameters:

- **Category**, specifying the *kind of entity* the system is taken to be for the purpose of interpretation, reflecting a particular engineering specialisation (e.g. *Capability*, *Implementation*, or *Space*);
- **Modality**, specifying the status of the appearance (e.g. *Intended*, *Actual*, or *Pattern*).

Together, these parameters constrain:

- what kind of entity a structural definition may evaluate to,
- how boundaries, relations, and descriptive conditions are interpreted,
- which semantic rules govern evaluation, comparison, and correspondence.

Aspect parameters therefore govern *appearance*. They determine what kind of thing a system is taken to be as an object of interpretation, independently of how that appearance is articulated structurally through elements, relations, and classifiers.

Interest

Distinct from aspect parameters is the notion of *interest*. Interest captures the engineering motivation or focus that guides *how* an appearance is articulated—namely, which distinctions are introduced, emphasised, or suppressed when describing a system under a fixed aspect.

Typical interests include functional analysis, maintenance planning, safety assessment, procurement, and spatial organisation. Interest influences:

- which decompositions are introduced or omitted,
- which relations are foregrounded,
- which constraints and descriptive details are made explicit.

In IMF, *articulation* refers to the selective structural expression of an appearance: the choice of what structure to make explicit, at what granularity, and for what purpose. Articulation does not introduce a new appearance, nor does it change the conditions under which meaning is interpreted. Rather, it reflects a modelling decision about relevance and focus.

Crucially, interest does not change how a system appears. Two modellers working under the same aspect but with different interests may legitimately produce different structural definitions without contradiction. For this reason, IMF treats interest as a contextual parameter of *articulation*, not as part of the aspect itself.

Interest plays a strategic role in IMF. It provides a principled mechanism for selective modelling, allowing the same appearance to be articulated differently for different engineering purposes while remaining comparable and semantically coherent. This role is realised structurally through relations such as `filteredTo`, which express restricted articulations motivated by interest without introducing new system structure.

This separation is essential: *aspect governs interpretation; interest governs modelling focus*. It allows multiple, parallel articulations of the same appearance to coexist within a single modelling context, supporting incremental modelling, discipline-specific views, and lifecycle traceability.

Explicit versus implicit context

Traditional technical information rely heavily on implicit context. Category, modality, and interest are typically inferred from drawing conventions, document status, and disciplinary practice rather than being stated explicitly.

IMF makes these contextual parameters explicit. By doing so, it provides a disciplined basis for explaining, comparing, and integrating information artefacts whose meaning would otherwise remain implicit or ambiguous.

Preparation for formal evaluation

Explicit contextual parameters prepare IMF models for formal evaluation. Aspect parameters constrain the evaluation space in which structural definitions are interpreted and determine what counts as a valid evaluation. Interest further constrains how such definitions are articulated and refined, without altering their mode of appearance. An outline of this evaluation process is given in [Section 4.10](#).

4.3 Structural roles in IMF

IMF expresses engineering meaning through three orthogonal but cooperating construct families: *elements*, *relations*, and *classifiers*. Together, these constructs provide the means to introduce, compose, and constrain

structural definitions that constitute a system appearance.

At the conceptual level, IMF distinguishes between *structural roles* and *structural elements*. A *structural role* characterises the function an entity plays in the articulation of an appearance—such as acting as a system boundary, an interface condition, an interaction carrier, or a descriptive constraint. A *structural element* is the concrete language construct used to realise such a role in an IMF model.

In IMF, this distinction is deliberately explicit:

- **Structural roles** capture modelling intent at an abstract level (e.g. system unit, interface, interaction, qualification).
- **Structural elements** (Block, Terminal, Connector, Quality) are the syntactic carriers through which these roles are instantiated in a model.

This separation allows IMF to maintain a small and stable core language while supporting different modelling practices and interpretations under varying aspects and interests. The semantic role of an element is therefore determined not only by its kind, but also by the contextual parameters under which it is declared and the relations in which it participates.

Within this framework:

- *Elements* introduce primitive structural definitions by instantiating structural roles.
- *Relations* compose these definitions, expressing decomposition, connectivity, articulation, and correspondence.
- *Classifiers* constrain interpretation by associating elements with terms from reference vocabularies.

These three construct families operate jointly. Elements provide the carriers of structural meaning, relations organise them into coherent configurations, and classifiers refine how those configurations are interpreted under a given aspect.

4.3.1 Elements: what appears

Elements are the primary carriers of structural definitions. An element introduces a unit that may participate in relations and be refined by classifiers.

Elements answer the question:

What kinds of things appear in this engineering description?

In particular:

- Blocks and Connectors introduce *system-bearing* appearances,
- Terminals introduce *interface conditions*,
- Qualities introduce *descriptive or constraining content*.

Elements are intensional: they do not assert existence or realisation. Their interpretation depends entirely on contextual parameters.

4.3.2 Relations: how appearances are organised

Relations compose elements into structured definitions. They express how elements are connected, refined, restricted, or related, either within a single appearance or across distinct appearances, under explicitly fixed aspects.

Within a fixed aspect, relations organise the internal structure of an appearance by expressing decomposition, connectivity, and articulation. Across aspects, relations make explicit how different appearances of what is understood to be the same system correspond or stand in normative dependency.

Relations answer the question:

How are the elements contributing to an appearance organised and related?

Different relations serve distinct organisational roles:

- `partOf` refines structural resolution,
- terminal–connector relations express topology and interaction,
- `filteredTo` restricts articulation without changing appearance,
- cross-aspect relations relate appearances of the same system.

Relations do not introduce new appearances or appearance categories. They organise and constrain how structural definitions introduced by elements jointly contribute to an appearance under interpretation.

4.3.3 Classifiers: how appearances are to be understood

Classifiers refine interpretation by associating elements with terms from reference vocabularies. They constrain the admissible *reading* of a structural definition under evaluation, without introducing new structure or asserting realisation.

Classifiers answer the question:

In what role, category, or sense is this element to be understood when interpreted?

A classifier:

- restricts interpretation under evaluation;
- does not introduce or modify structural relations;
- does not assert ontological type membership or existence.

Classifiers do not carry content themselves. Instead, they fix the *interpretive frame* under which an element's structural definition is evaluated.

Classifiers work in tandem with *qualities*. A classifier specifies *what kind of descriptive or constraining information is relevant*, while qualities carry the corresponding values or conditions.

For example, a classifier may indicate that a block is to be understood as a *pump*, or that a connector represents a *fluid stream*, thereby constraining how its terminals, relations, and associated qualities are interpreted during evaluation.

Note (non-normative): In later versions of IMF, classifier values may also be used to impose well-formedness constraints on admissible structural configurations (e.g. restricting the number or kinds of terminals associated with a block). Such constraints are not part of the core syntax in v0.4 and are treated as a matter of semantic validation rather than structural definition.

4.3.4 Orthogonality and cooperation

The three construct families are intentionally orthogonal:

Construct	Primary responsibility
Elements	Introduce units of engineering meaning (what appears)
Relations	Organise and compose those units (how they relate)
Classifiers	Constrain interpretation of those units (how they are understood)

No single construct is sufficient on its own:

- elements without relations lack organisation,
- relations without elements have no semantic anchors,
- classifiers without structure have no effect.

Engineering meaning in IMF emerges only from their combination, evaluated under explicit contextual parameters.

4.3.5 Preparation for formal evaluation

This separation of roles is not merely conceptual. It prepares IMF models for formal evaluation in later chapters:

- elements determine the carriers of interpretation,
- relations determine structural constraints,
- classifiers contribute interpretive premises and restrictions.

Evaluation operates over their combination, under fixed context, to derive semantic consequences. By keeping these roles distinct, IMF enables rigorous interpretation without collapsing engineering meaning into either pure structure or pure classification.

Summary. Elements determine *what appears*, relations determine *how it is organised*, and classifiers determine *how it is understood*. Their disciplined separation is a foundational design principle of IMF.

4.4 Structural elements of the IMF language

IMF provides a small number of structural elements. Each such element introduces a distinct kind of *structural definition* that contributes to how a system appears relative to a given aspect.

It is important to emphasise that IMF elements are *intensional* constructs. They do not denote concrete objects or instances. Rather, they express structural definitions whose extension depends on a given configuration of contextual parameters—most notably the aspect—relative to which they are interpreted.

This section introduces the core IMF elements and explains their conceptual roles. Together, these elements provide the minimal vocabulary required to articulate appearances of system boundaries, interactions, interfaces, and qualifications in a disciplined and aspect-aware manner.

4.4.1 System-bearing and structural elements

IMF elements fall into two broad categories, reflecting their role in structuring:

Category	Elements
System-bearing elements	Block, Connector
Structural elements	Terminal, Quality

System-bearing elements introduce appearances of systems or interaction systems relative to a given aspect. Structural elements refine how such systems are connected, interfaced, or qualified. This distinction concerns how system appearances are articulated, not how systems exist in the physical world.

4.4.2 Blocks

A *Block* introduces a structural definition of a system unit with a boundary. Relative to a given aspect, it expresses that something can meaningfully be treated as a unit for engineering reasoning.

What the boundary represents—functional responsibility, physical extent, spatial enclosure, or another organising principle—depends entirely on the configuration of contextual parameters that fixes the aspect. Blocks therefore serve as the primary system-bearing elements in IMF and act as anchors for further articulation.

4.4.3 Connectors

A *Connector* introduces the structural definition of an interaction system. It expresses continuity, transfer, or coupling between system units, and foregrounds interaction rather than containment.

Like blocks, connectors are system-bearing elements. They are bounded systems in their own right, but their defining modelling role is not to establish a system boundary, but to mediate interaction across boundaries introduced elsewhere.

The distinction between a block and a connector is therefore not one of ontological status, but of modelling focus. A block introduces a system unit whose boundary is primary for reasoning; a connector introduces an interaction system whose internal continuity or flow is primary.

At one level of abstraction, a flow of medium may be treated as a single connector. At another level, the same interaction may be articulated as a block with its own internal structure and terminals. IMF supports both choices and allows elements to shift role as modelling detail increases, without privileging either representation.

4.4.4 Terminals

A *Terminal* introduces the structural definition of an interface condition. It expresses how a system-bearing element may participate in an interaction.

Terminals articulate *interfaces* by making explicit where and under which conditions interactions cross a system boundary. They are always interpreted relative to a hosting system-bearing element, yet their meaning is not reducible to that host. Instead, terminals express interface semantics that become significant only when composed with connectors and interpreted relative to an aspect.

4.4.5 Qualities

A *Quality* introduces the structural definition of descriptive or constraining information. It is the mechanism through which properties, requirements, measurements, or parameters are associated with other elements.

Qualities refine the interpretation of existing structural definitions without introducing new system structure. A quality may represent a single parameter, a bundle of properties, or a reference to an external

information artefact such as a datasheet or submodel. Its semantic role depends on the aspect relative to which it is declared: the same quality may function as a requirement in one aspect and as a measured value in another.

4.4.6 Attributes

Attributes are a syntactic form used to associate named values with elements. Semantically, attributes are interpreted as qualities. The distinction is introduced for notational, visual, and tooling purposes and does not introduce a separate semantic category.

4.4.7 Why these elements are primitive

Blocks, connectors, terminals, and qualities are treated as primitive because they introduce the basic distinctions upon which all further articulation depends. They are not defined in terms of other IMF constructs.

By themselves, these elements do not yet constitute a system appearance. Appearances emerge only when elements are composed through relations and interpreted relative to a given aspect. Nevertheless, without these primitive elements there would be no stable units of meaning to compose.

4.5 Composition of structural definitions via relations

Structural elements introduce primitive structural definitions that can be composed through relations. Relations determine how elements are connected, refined, restricted, and qualified, thereby shaping how a system appearance is articulated under a given aspect.

In IMF, relations always relate *elements*. A structural definition is constituted by the closure of its elements and relations, interpreted under contextual parameters.

This section introduces the principal relations used in IMF. Together, they express the core organisational distinctions engineers routinely make when reasoning about systems: interaction, resolution, articulatory restriction, and qualification.

4.5.1 Topology: connectivity via terminals and connectors

Topological structure in IMF is expressed through the coordinated use of *Terminals* and *Connectors*. This enforces an interface-driven account of interaction.

A *Terminal* specifies an interface condition of a block. A *Connector* specifies an interaction system that satisfies such interface conditions at the chosen level of abstraction.

Interface mediation. Connectivity is always mediated explicitly. Blocks never connect directly to one another. Instead, interaction is expressed through terminals that are satisfied by a connector.

The core relations involved are:

$$hasTerminal(b, t), \quad hasConnector(t, c), \quad connectedVia(b, c).$$

Conceptually:

$$b \xrightarrow{hasTerminal} t \xrightarrow{hasConnector} c, \quad b \xrightarrow{connectedVia} c.$$

Connectivity between blocks. Two blocks b_1 and b_2 are connected under a given context if and only if there exist terminals t_1, t_2 and a connector c such that:

$$hasTerminal(b_1, t_1), \quad hasTerminal(b_2, t_2), \quad hasConnector(t_1, c), \quad hasConnector(t_2, c).$$

This formulation ensures that interaction is always articulated through explicit interface conditions on both sides.

Topology and abstraction. Whether an interaction is modelled as a connector or as a block is an abstraction decision. At coarser resolution, an interaction may be treated as a single connector. At finer resolution, the same interaction may be articulated as a block with internal structure. IMF supports both choices within the same aspect, enabling incremental modelling without invalidating existing structure.

4.5.2 Structural resolution: refinement via *partOf*

Structural definitions are often refined by introducing finer internal structure. IMF captures this form of refinement through the relation:

$$partOf(Child, Parent).$$

Intended reading. The intended reading of *partOf* is resolution-based:

At a finer level of structural resolution, the child element becomes explicit; at a coarser level, the parent element suffices.

Introducing a *partOf* relation makes additional internal structure explicit. Abstracting away from such relations yields a coarser description.

Engineering hierarchy. The relation *partOf* operates on structural definitions, not on systems as physical entities. It does not express physical containment or ontological mereology. Instead, it captures refinement of engineering articulation under a fixed aspect.

Structural hierarchies introduced via *partOf* are therefore purpose-driven and revisable, in line with engineering practice.

4.5.3 Articulatory restriction: *filteredTo*

IMF provides a relation for expressing contextual restriction without introducing new structure:

$$filteredTo(Parent, Child).$$

Intended reading. *filteredTo(Parent, Child)* is read as:

Child is a filtered articulation of *Parent*.

The argument order is intentional: the child is derived by filtering the parent.

Semantic contract. The relation *filteredTo(p,c)* states that:

- p and c express the same system appearance,
- they are interpreted under the same modality and category,
- c may expose fewer terminals than p ,

- c may carry additional qualities or stricter constraints,
- no new structural decomposition is introduced,
- no change in abstraction level is implied.

Relation to interest. Interest explains *why* a restricted articulation is useful in a given modelling situation. The relation *filteredTo* expresses *that* such restriction has occurred within the structural definition.

Separation from other relations. The role of *filteredTo* is clarified by contrast:

Relation	Changes structure	Changes abstraction	Changes constraints
<i>partOf</i>	Yes	Often	Possibly
<i>filteredTo</i>	No	No	Yes
<i>proxy</i>	No	No	No (cross-aspect)

4.5.4 Cross-aspect correspondence: proxy

Distinct elements may express the same underlying system under different aspects. IMF captures this correspondence using the relation:

$$proxy(x, y).$$

Intended reading. The relation *proxy*(x, y) states that x and y are aspect-dependent appearances of the same system. Neither refines nor restricts the other. The relation is symmetric and does not introduce structure.

Role of proxy. The *proxy* relation enables navigation across aspects while preserving the integrity of each appearance. It supports disciplined movement between functional, physical, and behavioural interpretations without collapsing them into a single representation.

4.5.5 Qualitative refinement: hasQuality

Structural definitions are refined through descriptive or constraining information using qualities and the relation:

$$hasQuality(x, q),$$

where x is an element and q is a quality.

Role of qualities. A quality refines the interpretation of an element without introducing new system structure. Qualities may express requirements, properties, measurements, or constraints, depending on the aspect under which they are interpreted.

Aspect-indexed interpretation. The meaning of a quality is governed by the aspect of its declaration. The same quality may function as a requirement under an intended aspect and as an observed value under an actual aspect, without changing its syntactic role.

4.5.6 Orthogonality of relational dimensions

The relations introduced above operate along distinct, orthogonal dimensions:

- **Topology** (*hasTerminal*, *hasConnector*, *connectedVia*) expresses interaction.
- **Structural resolution** (*partOf*) expresses refinement.
- **Articulatory restriction** (*filteredTo*) expresses contextual narrowing.
- **Cross-aspect correspondence** (*proxy*) links appearances.
- **Qualification** (*hasQuality*) refines meaning.

These relations may coexist freely within a single structural definition. Together, they provide a disciplined language for expressing how systems are structured, connected, restricted, and qualified in engineering practice.

4.5.7 Summary

Relations compose primitive structural definitions into coherent expressions of engineering meaning. They articulate interaction, resolution, restriction, correspondence, and qualification as independent but composable dimensions.

The next section introduces classifiers, which further refine structural definitions by associating elements with terms from reference vocabularies without asserting realization.

4.6 Refining structural definitions: classifiers

Structural definitions are composed from elements and relations. To further constrain how these definitions are to be understood—without introducing new structure or asserting realization—IMF uses *classifiers*.

A classifier associates an element with a term drawn from a reference vocabulary. It refines engineering meaning by constraining interpretation, while preserving the intensional and non-assertive character of structural definitions.

4.6.1 Classifiers as interpretive refinement

Formally, a classifier statement has the form:

$$\text{classifier}(x, v),$$

where x is an IMF element and v is a term drawn from a reference vocabulary.

Informally, a classifier specifies how an element is intended to be understood within a structural definition. Typical uses include

- assigning an intended role or category to a block,
- characterising the kind of interaction expressed by a connector,
- indicating the interface semantics of a terminal,
- identifying the nature of information carried by a quality.

In all cases, the classifier value is used *as a term*. It is not treated as an ontological type assertion and does not imply membership in an extensional class. Classifiers constrain interpretation; they do not assert existence or realisation.

4.6.2 Classifiers are not ontological typing

It is important to distinguish classifiers in IMF from typing in ontologies or data models.

In particular:

- a classifier does not change the structural role of an element,
- it does not introduce subsumption or inheritance relations,
- it does not assert that an element instantiates a class in reality.

Instead, classifiers associate elements with externally defined terms in order to support shared understanding, integration, and interpretation under specific semantic frameworks.

This separation allows IMF to remain neutral with respect to ontological commitments while still benefiting from established vocabularies and standards.

4.6.3 Relation between classifiers and qualities

Classifiers and qualities play complementary but distinct roles in IMF.

- A *classifier* specifies *what kind of meaning* is being expressed.
- A *quality* carries the corresponding descriptive or constraining content.

For example, a terminal may be classified as expressing an *InterfaceCondition*, while an attached quality collects the concrete constraints associated with that condition, such as allowable pressure range or maximum temperature.

This separation allows IMF to distinguish clearly between:

- categorisation or qualification of meaning (classifier),
- content-bearing information (quality).

Together, classifiers and qualities enable precise refinement of structural definitions without entangling structure with instance data.

4.6.4 Reference vocabularies and semantic neutrality

Classifier values are typically drawn from reference data libraries, controlled vocabularies, or standardised terminologies. IMF assumes that such vocabularies provide stable terms with shared engineering understanding.

From the perspective of the IMF language:

- a classifier value is treated as a symbol with agreed meaning,
- using it as a classifier does not import external axioms or instance data,
- multiple vocabularies may be used side by side within the same model.

IMF therefore remains neutral with respect to the internal semantics of reference vocabularies. The meaning of a classifier arises from its role within a structural definition and its interaction with elements, relations, and contextual parameters.

4.6.5 Aspect- and interest-dependent interpretation

Like all IMF constructs, classifiers are interpreted relative to contextual parameters.

The same classifier term may play different roles depending on:

- the modality under which the element is interpreted,
- the category determining the kind of appearance,
- the interest constraining articulation.

For example, a classifier indicating a performance characteristic may function as:

- a requirement under an intended aspect,
- an observed property under an actual aspect,
- a typical constraint under a pattern aspect.

Interest does not change the appearance expressed by a classifier. Instead, it constrains how that appearance is articulated and used within the structural definition.

4.6.6 Admissible classifiers and syntactic discipline

Different kinds of elements admit different kinds of classifiers. This admissibility is part of the syntactic discipline of the IMF language and ensures meaningful use without overconstraining interpretation.

For example:

- blocks may be classified by functional role or system category,
- connectors by medium or interaction type,
- terminals by interface semantics,
- qualities by the nature of the information they contain.

These constraints regulate *where* classifier terms may be applied, not *what they mean*. They support consistency and tool support while preserving the openness required for incremental modelling.

4.6.7 Contribution to formal evaluation

Although classifiers are introduced here informally, their design aligns directly with later formal evaluation.

In such evaluation, classifier statements contribute constraints that shape admissibility, compatibility, and semantic consequences of structural definitions. They function as premises for checking consistency and for deriving formal interpretations, without fixing realisation.

At this stage, the essential point is:

Classifiers constrain engineering meaning without asserting existence or fixing realisation.

They allow structural definitions to remain expressive, flexible, and reusable across contexts, while supporting rigorous interpretation when required.

4.6.8 Summary

Classifiers refine structural definitions by associating elements with terms from reference vocabularies. They constrain interpretation without introducing structure or asserting realisation.

Together with elements, relations, and qualities, classifiers complete the core expressive resources of the IMF language for articulating engineering meaning in a disciplined and aspect-aware manner.

4.7 IMF models as collections of structural definitions

An IMF model is a collection of interrelated structural definitions that are intended to be interpreted together as a coherent whole. The elements, relations, and classifiers in a model are not independent assertions, but mutually constraining definitions articulated under compatible contextual parameters and a shared modelling purpose. An IMF model may span multiple levels of detail, contextualisation, and maturity, and is designed to support incremental extension rather than exhaustive or final description.

4.7.1 Structural definitions as semantic units

The fundamental semantic units of an IMF model are its elements. Each element expresses a structural definition: a definition-like specification of how something is to be understood relative to a given configuration of contextual parameters (an aspect).

An IMF model may therefore be characterised as a collection of structural definitions, linked by relations and refined by classifiers.

Relations do not relate systems or appearances directly. They relate elements and thereby articulate how individual structural definitions depend on, refine, or contextualise one another. As a result, the meaning of any single element is inseparable from the network of relations and classifiers in which it participates.

4.7.2 Incremental modelling and partial context

IMF is designed to support incremental modelling. In many engineering situations, not all contextual parameters are known, decided, or agreed upon when modelling begins. An IMF model therefore allows elements to be introduced under partially specified contexts.

In practice:

- contextual parameters such as modality, category, or interest may be omitted initially,
- parameters may be marked as tbd (to be determined),
- different elements within the same model may be associated with contexts of differing specificity.

This reflects engineering practice directly. Early models often express tentative structure, open questions, or alternative interpretations. IMF allows such partial structural definitions to coexist and to be refined over time as additional information becomes available.

4.7.3 Open-world interpretation

IMF adopts an open-world interpretation. The absence of information is treated as underspecification rather than as negation or error.

An IMF model may therefore contain:

- alternative structural definitions awaiting resolution,

- provisional classifications subject to later refinement,
- partial hierarchies or topologies that will be completed over time.

As modelling progresses, additional relations, classifiers, or contextual assignments refine the model. Each such addition restricts admissible interpretations of the model as a whole, while preserving the validity of earlier structural definitions.

4.7.4 Multiple systems and multiple appearances

An IMF model may simultaneously describe multiple systems and multiple appearances. Structural definitions belonging to different systems may coexist within the same model, distinguished by their relational structure and, where appropriate, related explicitly through cross-aspect relations.

There is no requirement that an IMF model correspond to a single system or a single viewpoint. Instead, the model provides a shared space in which multiple structural definitions—possibly produced by different disciplines or at different times—can be articulated and related explicitly.

4.7.5 Relation to formal evaluation

Later chapters introduce a formal evaluation of structural definitions, in which contextual parameters determine how definitions are interpreted and what they evaluate to. The incremental and open-ended nature of IMF models is compatible with that evaluation.

In particular:

- missing or tbd parameters correspond to partially defined evaluations,
- refining contextual parameters corresponds to restricting the space of admissible interpretations,
- adding relations or classifiers corresponds to strengthening a structural definition.

From this perspective, an IMF model evolves by progressively refining a collection of structural definitions through additional articulation and contextualisation.

4.7.6 Summary

An IMF model is an evolving collection of structural definitions of engineering meaning. These definitions may be partial, provisional, or underspecified. Relations and classifiers intertwine them, and contextual parameters guide their interpretation when available.

IMF models are intentionally open-ended. They capture those structural definitions relevant to a given purpose and can be incrementally extended, refined, or related to other models over time, without forcing premature closure or ontological commitment.

4.8 From P&ID notation to structural definitions

We now revisit the P&ID fragment introduced in [Section 3.2](#) and show how its implicit engineering meaning is reconstructed explicitly using the core constructs of the IMF language.

4.8.1 Syntax and naming conventions for the P&ID example

This subsection summarises the naming and declaration conventions used consistently throughout the IMF examples in this chapter, in particular those derived from the P&ID fragment introduced in [Section 3.2](#). The

conventions are pragmatic and reader-oriented, while remaining aligned with the semantic foundations established in [Chapter 3](#).

Identifiers versus declarations. IMF makes a strict distinction between:

- *Identifiers*, which support human recognition and reference,
- *Declarations*, which fix the structural role and contextual interpretation of an element.

Identifiers are chosen for readability and recognisability and do not encode semantic categories. All semantic commitments are made explicitly in declarations.

Identifiers derived from P&ID artefacts. When examples are based on a P&ID, identifiers reuse the engineering tags shown in the diagram wherever possible:

TA102, PA103, HJ105.

These tags already encode equipment class and identity according to engineering convention. Repeating that information in the identifier (e.g. Tank_TA102) is therefore avoided.

Identifiers may include a short mnemonic suffix if this improves readability (e.g. PA103_In), but must not encode element kind, aspect, or interest.

Declaration syntax. Elements are introduced using explicit declarations that fix:

- the *kind of structural definition* introduced, and
- the *configuration of contextual parameters* under which it is interpreted.

Throughout this chapter, we use the following illustrative surface form:

$$x : \text{ElementKind}[m, c ; i]$$

where:

- *ElementKind* is one of Block, Connector, Terminal, or Quality;
- *m* is the **Modality**;
- *c* is the **Category**;
- *i* is the **Interest** (optional)

Here, **Modality** and **Category** together determine the *aspect* under which the element is interpreted.

Reading guide (semicolon intuition). The declaration may be read using the following heuristic:

Left of “;” — *what kind of meaning is expressed* (aspect).

Right of “;” — *for what purpose that meaning is articulated* (interest).

For example:

PA103 : Block[Intended, Capability]

should be read as:

“PA103 expresses a block-level structural definition, interpreted as intended capability.”

Element kind is not encoded in the identifier. Whether an identifier denotes a block, connector, terminal, or quality is determined entirely by its declaration. This keeps identifiers stable across modelling contexts and avoids coupling IMF syntax to project-specific naming conventions.

Terminals and aspect sensitivity. Terminals represent interface conditions and are declared like other elements, but their availability is aspect-dependent. For example:

- Nozzles are meaningful under an *Implementation* category,
- Functional entry or exit points may be introduced under a *Capability* category,
- The same underlying system may therefore give rise to distinct terminal sets under different aspects.

This variability is intentional and does not affect identifier syntax.

Interest-specific articulation and filtering. Interest parameters motivate restricted articulations of the same system appearance. Such restrictions are expressed structurally using the relation `filteredTo`, not by modifying identifiers or element kinds.

For example:

```
PA103_Maint : Block[Intended, Implementation ; Maintenance]
```

together with:

```
filteredTo(PA103, PA103_Maint)
```

expresses that the maintenance articulation is a filtered version of the same appearance, motivated by a specific interest, rather than a different system or a different aspect.

Summary of alignment rules.

- Identifiers support recognition; declarations carry semantics.
- Element kind is encoded only in declarations.
- Aspect parameters (Category and Modality) are explicit and visible in declarations.
- Interest motivates articulation but does not change appearance.
- Structural restriction is expressed via relations, not naming.

4.8.2 Identifying system-bearing structural definitions

We begin by identifying the system-bearing structural definitions that are implicit in the P&ID fragment shown in [Section 3.2](#). At the level of abstraction suggested by the diagram, the primary system units of interest are the circulation pump, the buffer tank, and the heat exchangers that interface with neighbouring systems.

Using the identifier conventions introduced above, these are declared as block-level structural definitions. For example:

```
A40PA103 : Block[Intended, Capability]
```

introduces a structural definition of the circulation pump as a system unit, interpreted as an intended capability. Likewise, the tank and one of the heat exchangers are introduced as:

```
A40TA102 : Block[Intended, Capability],    A23HJ105 : Block[Intended, Capability].
```

At this stage, no internal structure, interfaces, or constraints are made explicit. Each block expresses only that the corresponding artefact is treated as a meaningful system unit under the chosen aspect.

4.8.3 Making interaction explicit: terminals and connectors

The P&ID fragment does not merely list system units; it shows how they interact through a circulating cooling medium. In IMF, such interaction is not expressed directly between blocks. Instead, interaction is articulated through terminals and connectors, which make interface conditions and interaction systems explicit.

We therefore introduce terminal-level structural definitions that express the interface conditions under which system units participate in an interaction. For example:

$$PA103_In : Terminal[Intended, Capability], \quad TA102_Out : Terminal[Intended, Capability].$$

Each terminal is associated with exactly one hosting block:

$$hasTerminal(A40PA103, PA103_In), \quad hasTerminal(A40TA102, TA102_Out).$$

The circulating medium itself is introduced as a connector-level structural definition:

$$CoolingLoop : Connector[Intended, Capability].$$

The connector expresses the interaction system responsible for transporting the medium, without committing to any particular physical realisation.

Terminals participate in the interaction by being associated with the connector:

$$hasConnector(PA103_In, CoolingLoop), \quad hasConnector(TA102_Out, CoolingLoop).$$

From these relations, the block–connector participation is obtained *derivationally*. In particular, the relation

$$connectedVia(A40PA103, CoolingLoop)$$

is understood as holding whenever there exists a terminal t such that:

$$hasTerminal(A40PA103, t) \wedge hasConnector(t, CoolingLoop).$$

An analogous statement holds for A40TA102.

Through these declarations and relations, IMF makes explicit what the P&ID conveys implicitly: that the pump and the tank interact by participating—via explicitly defined interface conditions—in a common interaction system.

4.8.4 Adding descriptive refinement through classifiers

The P&ID also conveys descriptive information that constrains how the interaction is to be understood. For example, the circulating medium is not arbitrary; it is a cooling medium associated with a particular service.

In IMF, such refinement is expressed using classifiers. For example:

$$classifier(CoolingLoop, CoolingMedium),$$

associates the connector with a reference term indicating the nature of the interaction it represents.

Similarly, classifiers may be used to refine the interpretation of blocks or terminals, for example by indicating functional role, interface type, or service classification. At this level, classifier values are used strictly as terms; they do not assert physical realisation or instance membership.

4.8.5 Contextual variation without structural change

A notable feature of the P&ID fragment introduced in [Section 3.2](#) is that the same diagram may be read under different lifecycle assumptions. An operator may read it as a representation of an installed system, while a designer may read it as an intended configuration.

IMF captures this by allowing the same identifiers and relations to be declared under different aspects. For example, an implementation-oriented articulation of the pump may be introduced as:

```
A40PA103_Imp1 : Block[Actual, Implementation].
```

This declaration does not replace the capability-oriented one. Instead, it introduces a distinct structural definition that may later be related to the intended-capability articulation through correspondence relations introduced elsewhere in the manual.

In this way, IMF preserves the visual and structural continuity of the P&ID while making the underlying contextual assumptions explicit.

4.8.6 Restricted articulation for maintenance interest

The P&ID fragment in [Section 3.2](#) is primarily process-oriented: it foregrounds flow paths, operating units, and control instrumentation. In maintenance planning, however, engineers are often interested in a more restricted articulation of the same system appearance—one that suppresses irrelevant interfaces and foregrounds service-relevant information.

IMF captures this situation by introducing a filtered articulation of an existing structural definition, motivated by a maintenance interest.

For example, starting from the intended-capability articulation of the pump:

```
A40PA103 : Block[Intended, Capability],
```

a maintenance-oriented articulation may be introduced as:

```
A40PA103_Maint : Block[Intended, Capability ; Maintenance].
```

The relationship between these two articulations is then stated explicitly:

```
filteredTo(A40PA103, A40PA103_Maint).
```

This statement expresses that `A40PA103_Maint` is not a different system, nor a different appearance. It is a restricted articulation of the same appearance, under the same aspect, motivated by a different engineering purpose.

In the maintenance articulation:

- some terminals present in the process-oriented articulation may be omitted if they are irrelevant for servicing tasks,
- additional qualities may be attached, such as maintenance intervals, access requirements, or replacement constraints,
- no new internal structure is introduced unless explicitly added through separate relations.

Crucially, the restriction is expressed structurally via *filteredTo*, not implicitly through naming conventions or diagrammatic omission. This allows multiple interest-driven articulations of the same system appearance to coexist in a single IMF model, while remaining explicitly related and semantically disciplined.

4.8.7 What the reconstruction achieves

By reconstructing the P&ID fragment as a collection of structural definitions, relations, and classifiers, IMF achieves three things simultaneously.

First, it separates system units, interaction systems, and interface conditions that are visually conflated in the diagram. Second, it makes explicit the contextual parameters under which the diagram is interpreted. Third, it provides a basis for incremental refinement, formal evaluation, and integration with other engineering information.

Importantly, this reconstruction does not replace the P&ID. It makes explicit the structural assumptions that engineers already rely on when reading such diagrams, and renders them available for systematic reasoning within the IMF framework.

4.9 Cross-aspect relations

The relations introduced so far operate *within* a fixed aspect. They compose and refine structural definitions that jointly contribute to a single system appearance under a given configuration of contextual parameters.

Engineering practice, however, routinely requires relating structural definitions interpreted under *different aspects*. Design intent must be related to realised systems; functional descriptions must be related to physical implementations; requirements must be related to observed behaviour. Such relations do not refine a single appearance, but connect *distinct appearances* of what is understood to be the same underlying system.

IMF captures this need through *cross-aspect relations*.

4.9.1 Role of cross-aspect relations

Cross-aspect relations differ fundamentally from the relations introduced in [Section 4.5](#):

- they do not introduce additional structural resolution,
- they do not restrict articulation by interest,
- they do not express topology or connectivity.

Instead, they relate *structural definitions declared under different aspects*. They make explicit how different appearances are understood to correspond, without collapsing those appearances into a single description.

From the perspective of the IMF language, cross-aspect relations are relations between elements. From the perspective of engineering interpretation, they articulate how appearances are connected across lifecycle stages, domains, or modalities.

4.9.2 Correspondence: preview of proxy

The most basic cross-aspect relation in IMF is proxy:

$proxy(x, y).$

Intended reading. $proxy(x, y)$ states that the structural definitions introduced by x and y are alternative appearances of what is taken to be the same underlying system, evaluated under different aspects.

Each element retains its own declaration and contextual parameters. The relation establishes correspondence, not identity, derivation, or equivalence.

Admissible scope (v0.4). In IMF v0.4, proxy is defined only between *system-bearing elements*. That is, both arguments of proxy must be declared as either Block or Connector, and must share the same element kind. Terminals and qualities are excluded from the domain of proxy. They are treated as aspect-dependent refinements of an appearance rather than as independent cross-aspect correspondence targets.

Example (from the P&ID). Let

$A40PA103_Cap : Block[Intended, Capability]$

denote the functional appearance of the circulation pump, and

$A40PA103_Impl : Block[Actual, Implementation]$

denote the installed pump in the physical system.

The correspondence between these appearances is expressed as:

$proxy(A40PA103_Cap, A40PA103_Impl).$

This statement records the modeller's judgement that the two blocks concern the same system, seen under different conditions of appearance. It does not assert that one appearance is derived from the other, nor that their internal structure is equivalent.

What proxy does not assert. A proxy relation does *not* express:

- structural refinement or decomposition,
- fulfilment of requirements,
- equivalence of constraints, interfaces, or qualities,
- temporal, causal, or lifecycle dependency.

4.9.3 Normative dependency: preview of fulfils

Beyond correspondence, engineering reasoning often requires expressing *normative dependency* across aspects. Typical examples include:

- an implementation fulfilling a functional requirement,
- an installed system fulfilling a design specification,
- observed behaviour fulfilling an intended capability.

IMF anticipates such dependencies using relations of the form:

$fulfils(x, y).$

Intended reading. $fulfils(x, y)$ expresses that the interpretation of x , under its aspect, satisfies the constraints expressed by y , under its aspect.

Unlike proxy, fulfils is directional and normative. It expresses a dependency between appearances, not merely correspondence.

Deferred treatment. The precise semantics of fulfilments—including the role of classifiers, qualities, and evaluation—are developed in later chapters. Here, the relation is introduced only to situate its role within the overall structure of IMF.

4.9.4 Separation of concerns

By treating cross-aspect relations explicitly, IMF maintains a clear separation between:

- composition of structure within an aspect,
- restriction of articulation by interest,
- correspondence and dependency across aspects.

This separation supports lifecycle traceability, comparison, and verification without collapsing distinct appearances into a single, overloaded representation.

4.10 Outline of formal evaluation

Having introduced the core constructs of the IMF language, the remaining question is how these constructs acquire precise and stable meaning. Formal evaluation provides this bridge by defining how structural definitions are interpreted under an explicit configuration of contextual parameters.

Formal semantics are required in IMF not to replace engineering judgement, but to support systematic consistency checking, comparison across aspects, traceability across lifecycle stages, and reliable tool interoperability. Without a principled account of evaluation, IMF models would remain informal descriptions rather than analysable and integrable representations.

This section gives a principled outline of that evaluation. It does not introduce formal machinery, but fixes the conceptual commitments that any formal semantics of IMF must respect.

4.10.1 Structural definitions as intensional objects

In IMF, every element declaration introduces a *structural definition*. A structural definition specifies how something is to be understood, structured, or constrained, rather than asserting what exists.

Such definitions are inherently intensional and context-dependent. Their interpretation depends on:

- the aspect under which they are evaluated,
- the relations they enter into with other elements,
- the classifiers and qualities that refine them.

An IMF model therefore constitutes a network of mutually constraining structural definitions, whose joint interpretation yields a system appearance under a given aspect.

4.10.2 Evaluation relative to contextual parameters

The evaluation of a structural definition always takes place relative to an explicit configuration of contextual parameters. As introduced in [Chapter 3](#), these parameters govern how a definition is read, not what structure it introduces.

At a minimum, they include:

- **Modality**, constraining the status of the result (Intended, Actual, Pattern);

- **Category**, constraining the semantic category of the result (Capability, Implementation, Space);
- **Interest**, constraining which distinctions are relevant for articulation.

Fixing these parameters restricts the space of admissible interpretations, but does not add or remove elements from the model.

4.10.3 Declaration, composition, and evaluation

Informally, the interpretation of an IMF model proceeds in three stages:

1. **Declaration.** Element declarations introduce primitive structural definitions of system units, interactions, interfaces, or descriptive conditions.
2. **Composition.** Relations and classifiers compose these definitions into a structured configuration expressing boundaries, connectivity, hierarchy, articulation, and constraints.
3. **Evaluation.** Under a fixed aspect, the composed configuration is interpreted, yielding for example:
 - a concept description (in Intended or Pattern modalities),
 - a reference to an actual system or configuration (in Actual modality),
 - or a hybrid object combining constraints and references.

Adding, removing, or refining IMF statements changes the intensional content of the configuration and therefore changes the result of evaluation.

4.10.4 Primitive and composed structural definitions

From a semantic perspective, IMF distinguishes between:

- **Primitive structural definitions**, introduced directly by element declarations;
- **Composed structural definitions**, formed through relations and classifier attachments.

Primitive definitions carry engineering information in isolation. Composed definitions determine engineering meaning through their interaction. This distinction underlies the inductive structure of IMF models and their later formalisation.

4.10.5 Ontology as a parameter of evaluation

Formal evaluation in IMF is *parametric in the ontology used to formalise semantic concepts*. IMF does not fix a single ontological foundation a priori.

For example, under an IDO-based interpretation, evaluated structural definitions give rise to axioms expressed using `ido:System` and related concepts. Other ontologies may be used in principle, provided they support the required semantic distinctions.

This design choice preserves IMF's role as a structural language rather than an ontology.

4.10.6 Relation to later formalisation

The outline above anticipates a formal account in which:

- structural definitions are represented as typed intensional expressions,
- contextual parameters are treated as arguments to evaluation functions,

- relations correspond to intensional operators,
- classifiers contribute constraints in the target ontology.

Later chapters develop this account formally. The purpose of the present section is to fix the conceptual ground on which that development rests.

4.10.7 Chapter summary

This chapter has introduced the core language constructs of IMF and clarified how they jointly express system appearances under explicit interpretive conditions. Elements introduce structural definitions, relations organise and constrain them, classifiers refine their interpretation, and IMF models collect these constructs into coherent, incrementally extensible configurations.

Together, these constructs provide a disciplined language for making explicit the structural assumptions that engineers routinely rely on when describing, comparing, and reasoning about systems across contexts.

Glossary of IMF Terms

This chapter consolidates and stabilises the terminology introduced in Part I of the IMF manual. The entries provide concise, normative definitions of core IMF concepts and are intended as a reference for readers of both Part I and Part II. The glossary defines conceptual terms; syntactic definitions are given in Part II.

Appearance A structured semantic outcome that expresses how a system is understood under fixed interpretive conditions. An appearance conveys engineering meaning relative to an aspect, rather than asserting what exists independently of context.

Aspect A configuration of contextual parameters that fixes the conditions under which structural definitions are interpreted. In IMF, an aspect is determined by a *Modality* and a *Category*. The parameter *Interest* operates at the same conceptual level but constrains articulation rather than interpretation. An aspect governs interpretation but does not introduce structure.

Aspect switching A change in interpretive conditions whereby the same system is considered under a different aspect, yielding a different appearance. Aspect switching preserves system identity while altering only the conditions under which structure is interpreted.

Attribute A syntactic form used to associate named values with elements. Attributes are interpreted semantically as *qualities* and do not constitute a separate semantic category in IMF. The distinction reflects a separation between surface syntax and semantic role.

Block A system-bearing structural element that introduces a boundary relevant for engineering reasoning under a given aspect. A block contributes an appearance of a system unit; what the boundary represents—functional, physical, spatial, or otherwise—depends on the contextual parameters of interpretation.

Category A contextual parameter that specifies the semantic category under which an appearance is interpreted. IMF distinguishes the categories *Capability*, *Implementation*, and *Space*. Category governs how structure is interpreted, not which structural relations are available.

Classifier A statement that associates an element with a term from a reference vocabulary. Classifiers constrain admissible interpretations of a structural definition without asserting realisation or ontological membership.

Composed structural definition A structural definition formed by combining primitive structural definitions through relations and classifiers. Composed definitions determine appearances through the interaction of their parts, rather than through isolated declarations.

Connector A system-bearing structural element that introduces an interaction system, expressing continuity, transfer, or coupling. A connector mediates interaction between terminals and foregrounds interaction rather than boundary.

Contextual parameters Parameters that govern interpretation of structural definitions. In IMF, these are *Modality*, *Category*, and *Interest*. Together they determine an aspect and its articulation constraints. Contextual parameters constrain evaluation but do not introduce structure.

Cross-aspect relation A relation that connects distinct appearances of the same system across different aspects, without merging or collapsing those appearances into a single description.

Declaration A statement that assigns an element kind and contextual parameters to an identifier. Declarations fix the semantic role and interpretation of an element; identifiers themselves carry no semantic commitments.

Element A primitive carrier of a structural definition. Elements are *intensional* constructs, meaning that they specify how something is to be understood rather than denoting an extensional object. The core IMF elements are *Block*, *Connector*, *Terminal*, and *Quality*.

Evaluation The interpretation of a network of structural definitions relative to a fixed aspect. Evaluation yields an appearance, such as a concept description or a reference to an actual system, depending on modality.

filteredTo An intra-aspect relation expressing restricted articulation of the same appearance. `filteredTo(base, filtered)` states that the second element is a filtered articulation of the first under the same aspect, possibly exposing fewer interfaces or adding constraints, without introducing new structure.

Fulfils A directional cross-aspect relation expressing normative dependency. `fulfils(x,y)` states that the interpretation of x satisfies the constraints expressed by y , each under their respective aspects.

Identifier A pragmatic name used to refer to an IMF element. Identifiers support recognition and traceability but carry no semantic commitments.

Interest A contextual parameter that constrains how a structural definition is articulated and which distinctions are made explicit. Interest motivates filtered articulations but does not change appearance.

Modality A contextual parameter that specifies the status of an appearance. IMF distinguishes the modalities *Intended*, *Actual*, and *Pattern*.

Primitive structural definition A structural definition introduced directly by an element declaration, independent of relations and classifiers.

proxy A cross-aspect relation expressing correspondence between appearances. `proxy(x,y)` states that x and y are alternative appearances of what is taken to be the same system, without asserting equivalence, derivability, or satisfaction.

Quality A structural element that introduces descriptive or constraining information. Qualities refine interpretation by adding constraints or descriptors, without introducing new system structure.

Relation A construct that composes structural definitions by expressing connectivity, hierarchy, articulation, or correspondence. Relations relate elements, which in turn contribute to appearances of systems.

Structural definition An intensional specification of how something is to be understood, structured, or constrained when interpreted relative to a given configuration of contextual parameters (an aspect). Structural definitions are the semantic units of IMF models.

Terminal A structural element that introduces an interface condition under which a system-bearing element participates in an interaction. Terminals articulate interfaces and mediate connectivity between blocks and connectors.

Part II

IMF Languages

Chapter 5

Language Definitions

Abstract

This chapter defines the core syntax and well-formedness conditions of the IMF language as of version 0.4. It serves as a reference specification and as the syntactic anchor for later chapters that introduce concrete serialisations and implementations. The chapter specifies what constitutes a well-formed IMF model, in terms of element declarations, relations, classifiers, and contextual parameters. The chapter fixes the abstract syntax and admissible combinations of IMF constructs. It does not prescribe a concrete surface syntax or encoding. It does not define formal semantics. Conceptual motivation and interpretive principles are provided in [Part I](#), while concrete RDF-, OWL-, and SHACL-based realisations are introduced in subsequent chapters.

Draft Notice *The definitions in this chapter are **normative** for IMF v0.4 with respect to language syntax and well-formedness, unless explicitly stated otherwise. Concrete realisations of the IMF language in RDF, OWL, and SHACL, introduced in subsequent chapters, is not fully aligned with this specification in version 0.4. Full alignment across the framework is planned for version 0.5.*

Intended readers This chapter is written for language designers, tool developers, data engineers, and advanced practitioners who require a precise and implementation-independent specification of the IMF language. It assumes familiarity with the conceptual foundations of IMF presented in [Part I](#) and is intended to be read as a reference rather than a narrative. Readers are expected to be comfortable with formal definitions, well-formedness conditions, and abstract syntax, but not with any specific serialisation technology. This chapter provides the normative syntactic basis against which IMF serialisations, validation rules, and tooling are to be aligned in version 0.4.

5.1 Terminology summary (reference)

This section provides a compact reference summary of the terms used normatively in this chapter. The entries below do not restate conceptual motivation or formal semantics; they fix terminology and point to the corresponding discussions in [Part I](#) where needed.

Aspect A configuration of contextual parameters under which a structural definition is interpreted. In v0.4, an aspect is determined by **Modality** and **Category**, with optional **Interest** constraining articulation. (See [Chapter 3](#).)

Modality A contextual parameter indicating the status of interpretation, such as *Intended*, *Actual*, or *Pattern*. Modality affects how a structural definition is evaluated, not its syntactic form. (See [Section 4.2](#).)

Category A contextual parameter constraining the kind of semantic entity involved, such as *Capability*, *Implementation*, or *Space*. Category governs interpretation, not structural composition. (Replaces the earlier term *Information Domain*.)

Interest An optional contextual parameter constraining how a structural definition is articulated for a particular purpose (e.g. process, maintenance, safety). Interest does not change appearance and is reflected structurally via relations such as `filteredTo`. (See [Section 4.5](#) in Part I.)

Structural definition An intensional specification of how something is to be understood under a given aspect, rather than an assertion about what exists. Structural definitions are introduced by element declarations and composed via relations and classifiers.

Element A syntactic construct introducing a primitive structural definition. In v0.4, the element kinds are `Block`, `Connector`, `Terminal`, and `Quality`. (See [Section 4.4](#).)

Block A system-bearing element introducing a bounded structural definition of a system unit under an aspect. Blocks may participate in decomposition (`partOf`), interface articulation (`hasTerminal`), and cross-aspect correspondence.

Connector A system-bearing element introducing a structural definition of an interaction system (e.g. flow, signal, transfer). Connectors mediate interaction between blocks via terminals.

Terminal A structural element introducing an interface condition under which a system-bearing element participates in an interaction. Terminals are associated with blocks and connected to connectors.

Quality A structural element introducing descriptive or constraining information associated with exactly one other element. Qualities refine interpretation without introducing new system structure.

Relation A syntactic construct relating two elements and composing their structural definitions. Relations operate either within a fixed aspect (e.g. `partOf`, `connectedVia`, `filteredTo`) or across aspects (e.g. `proxy`, `fulfils`). (See [Section 4.5](#).)

Classifier A statement of the form `classifier(e, v)` associating an element with a term from a reference vocabulary. Classifiers constrain interpretation but do not assert ontological typing or realization. (See [Section 4.6](#).)

IMF model A collection of element declarations, relations, and classifiers that jointly express a set of structural definitions. IMF models are intentionally partial, open-ended, and incrementally extensible. (See [Section 4.7](#).)

Cross-aspect relation A relation connecting structural definitions interpreted under different aspects, expressing correspondence or normative dependency without collapsing appearances. (See [Section 4.9](#).)

Evaluation The interpretation of a composed IMF model under a fixed aspect, yielding semantic consequences (e.g. concept descriptions or references) without treating elements as extensional objects directly. (See [Section 4.10](#).)

5.2 Overview of syntactic building blocks

An IMF model is constructed from a small number of syntactic categories. The table below provides an orienting overview.

Category	Constructs
Contextual parameters	Modality, Category, Interest
Elements	Block, Connector, Terminal, Quality
Relations	partOf, hasTerminal, hasConnector, connectedVia, filteredTo, proxy, fulfils
Classifiers	classifier(element, term)
Attributes	<i>Not defined in v0.4</i>

The remaining sections define these constructs precisely.

5.3 Elements

Elements introduce primitive structural definitions. Every element declaration has the following abstract form:

$$e : \text{ElementKind}[m, c ; i]$$

where:

- e is an identifier,
- $\text{ElementKind} \in \{\text{Block}, \text{Connector}, \text{Terminal}, \text{Quality}\}$,
- m is a Modality,
- c is a Category,
- i is an optional Interest.

5.3.1 Block

A Block is a system-bearing element.

- May participate in `partOf` relations.
- May have associated `Terminal` elements via `hasTerminal`.
- May participate in `filteredTo`.
- May participate in cross-aspect relations.
- May have associated `Quality` elements.

5.3.2 Terminal

A Terminal introduces an interface condition.

- Must be associated with exactly one Block.
- May be related to at most one Connector via `hasConnector`.
- May not participate in `partOf`.
- May have associated Quality elements.

5.3.3 Connector

A Connector is a system-bearing element representing interaction.

- Mediates interaction between terminals.
- Is related to terminals via `hasConnector` (inverse: `isConnectorOf`).
- May have associated Quality elements.
- Does not participate in `partOf` in v0.4.

Note (non-normative): Participation of connectors in compositional relations is planned for future versions.

5.3.4 Quality

A Quality introduces descriptive or constraining information.

- Must be associated with exactly one other element.
- Does not participate in relations.
- Does not introduce system structure.

5.4 Relations

Relations compose structural definitions. All relations relate elements.

5.4.1 Intra-aspect relations

`partOf(child, parent)`

- Domain: Block
- Range: Block
- Inverse: `hasPart`
- Aspect constraint: same (m, c)

`hasTerminal(block, terminal)`

- Domain: Block
- Range: Terminal

`hasConnector(terminal, connector)`

- Domain: Terminal
- Range: Connector
- Inverse: `isConnectorOf`

`filteredTo(base, filtered)`

- Domain: Element
- Range: Element
- Inverse: `filteredFrom`
- Aspect constraint: same (m, c)

5.4.2 Derived relations

The following relations are *derived* relations. They are not primitive IMF relations, but are defined in terms of more basic relations for convenience and readability. *Note (non-normative)*: The relation `connectedTo` is intentionally defined at multiple structural levels. This reflects engineering practice, where connectivity may be discussed at the level of interfaces or at the level of system units, while preserving a single underlying structural interpretation.

`connectedTo(t_1, t_2)`

- Domain: Terminal
- Range: Terminal

`connectedTo(t_1, t_2)` holds iff there exists a connector c such that:

$$\text{hasConnector}(t_1, c) \wedge \text{hasConnector}(t_2, c).$$

This relation is symmetric and derived. It expresses that two interface conditions participate in the same interaction system.

`connectedVia(b, c)`

- Domain: Block
- Range: Connector

`connectedVia(b, c)` holds iff there exists a terminal t such that:

$$\text{hasTerminal}(b, t) \wedge \text{hasConnector}(t, c).$$

That is, a block participates in an interaction mediated by a connector via one of its terminals.

`connectedTo(b_1, b_2)`

- Domain: Block
- Range: Block

`connectedTo(b_1, b_2)` holds iff there exist terminals t_1, t_2 and a connector c such that:

$$\text{hasTerminal}(b_1, t_1) \wedge \text{hasTerminal}(b_2, t_2) \wedge \text{connectedTo}(t_1, t_2).$$

This relation is symmetric and derived. It abstracts from terminal-level detail and expresses that two blocks participate in a common interaction system.

5.4.3 Cross-aspect relations

`proxy(x, y)`

- Domain: Block $\cup$ Connector
- Range: Block $\cup$ Connector
- Aspect constraint: x and y must be declared under different aspects (different (m, c))
- Declaration constraint: either both x and y are declared as Block, or both are declared as Connector

`fulfils(x, y)`

- Domain: Element
- Range: Element
- Aspect constraint: different aspects

5.4.4 Informal reading of core relations (non-normative)

This subsection provides brief informal readings of core relations for orientation only. These readings are explanatory and non-normative; they do not introduce semantic commitments beyond those established conceptually in Part I.

`partOf` expresses structural decomposition, introducing finer-grained structural definitions relative to a coarser one, under a fixed aspect.

`hasTerminal` associates a block with an interface condition through which it may participate in interactions.

`hasConnector` associates a terminal with an interaction carrier that satisfies its interface condition.

`connectedVia` expresses that a block participates in an interaction mediated by a connector, via one of its terminals.

`connectedTo` expresses that two blocks participate in a common interaction, derived from their respective `connectedVia` relations. This relation is symmetric and derived.

`filteredTo` expresses restricted articulation of the same system appearance without introducing structural decomposition.

`proxy` expresses correspondence between structural definitions interpreted under different aspects, without asserting structural equivalence or derivation.

`fulfils` expresses a normative dependency across aspects, where one appearance is intended to satisfy the constraints expressed by another.

Note (non-normative): Later versions of IMF may refine or extend the admissible use of certain relations (notably `filteredTo` and `fulfils`) as their formal semantics are developed.

5.5 Classifiers

Classifiers associate elements with reference terms.

`classifier( $e, v$ )`

- e is an element.
- v is a term from a reference vocabulary.

Classifiers constrain interpretation. They do not assert realisation, instantiation, or ontological typing.

5.6 Attributes (deferred)

The syntax and semantics of attributes are not defined in IMF v0.4.

Attributes will be introduced in a future version as a surface form for attaching structured values that evaluate to Quality elements.

5.7 Well-formedness conditions

An IMF model is well-formed if:

- every identifier is declared exactly once;
- all relations respect declared domains and ranges;
- aspect constraints of relations are satisfied;
- each terminal is associated with exactly one system-bearing element;
- qualities are associated with at least one element.
- `proxy( $x, y$ )` is permitted only when x and y are both `Block` or both `Connector`, and are declared under different aspects.

5.8 Conformance and evolution

An IMF model conforms to version 0.4 if it satisfies all syntactic and well-formedness constraints defined in this chapter.

Future versions may:

- extend admissible relations,

- introduce attributes,
- refine well-formedness conditions,
- tighten semantic constraints.

Such changes will be documented explicitly in subsequent releases.

Chapter 6

Structural Specification

Abstract

This specification gives a formal description of the Information Modelling Framework (IMF) language. Its purpose is to provide an abstract and implementation independent specification of the IMF language, and with that give an introduction to the language and provide a common basis for practical and theoretical applications of IMF, such as specifying logical formalisations, serialization formats, and software applications and services that use IMF.

Draft Notice *This chapter has not yet been updated to reflect recent developments in [Part I: Fundamentals of IMF](#). See the Draft Notice on page [i](#) for more details.*

Intended readers This chapter is written for tool developers, data engineers, language designers, and researchers who require a rigorous, implementation-independent specification of IMF. It assumes familiarity with the IMF conceptual foundations ([Chapter 3](#)), core language constructs ([Chapter 4](#)), and definitions ([Chapter 5](#)). Readers should be comfortable with formal notation and standard semantic web technologies. This chapter provides the authoritative foundation for implementing IMF serialisations, reasoning engines, and validation tools.

Notation

The IMF vocabulary is written using the following typing:

- Classes are written using UpperCamelCase and appear as unary predicates in examples.
- Relations are written using lowerCamelCase and appear as binary predicates in examples.
- Individuals are written using lowerCamelCase and appear as constants in examples.

The diagrams that illustrate how classes, relations and individuals in the IMF vocabulary are related are informative only (and not normative). They indicate subclass relationships between classes, membership relationships between individuals and classes, and the domain and range of relations.

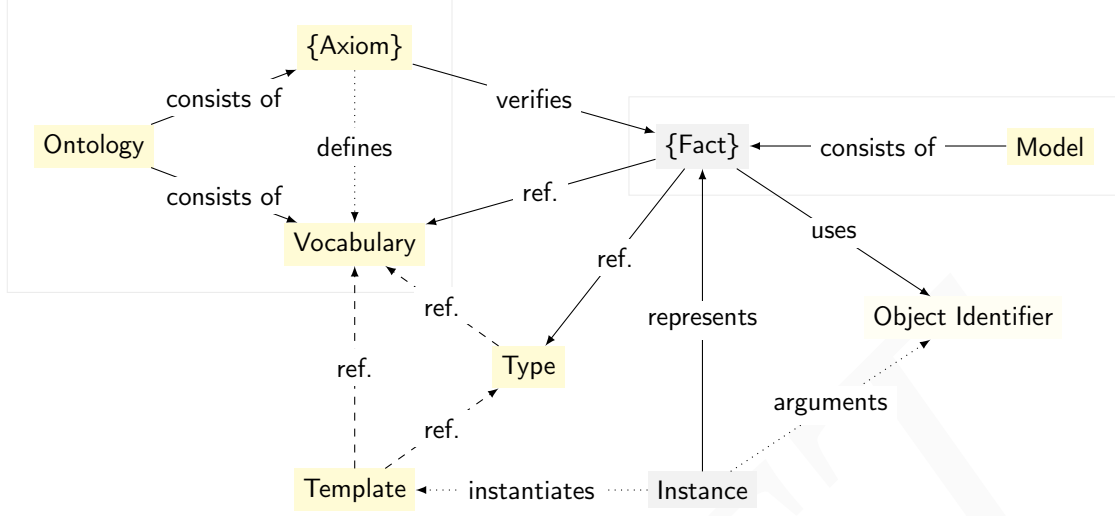


Figure 6.1: The parts of the IMF language and how they are related.

6.1 Introduction

The IMF language is split into the following main parts: ontology, models, types and templates. Closely related to the IMF language are also (object) identifiers. A high-level description of these parts are described below and informally displayed in [Figure 6.1](#).

6.1.1 IMF Ontology

The ontology comprises

- A vocabulary, i.e., a reserved set of terms, that is used to express facts about the target domain.
- A set of rules over the vocabulary, called axioms, that describe the intended use of the vocabulary and hence how the vocabulary is used to express correct and meaningful facts.

Example 6.1. The IMF ontology contains vocabulary terms such as *Block*, *Terminal*, *partOf* and *connectedTo*, and axioms such as “A *Terminal* can only be *partOf* other *Terminals*” and “*Blocks* are not *Terminals*”, which can be formalised as follows with these logic formulas:

$$\begin{aligned} \forall x, y (Terminal(x) \wedge partOf(x, y) \rightarrow Terminal(y)) \\ \forall x (Block(x) \rightarrow \neg Terminal(x)) \end{aligned}$$

6.1.2 Identifiers

In the context of the IMF language specification, an object identifier is a name for an object or entity that is described using the IMF language.

A goal of IMF is to be able to clearly identify distinct descriptions of assets, and to integrate data that is typically distributed across multiple systems and diagrams using multiple identifiers.

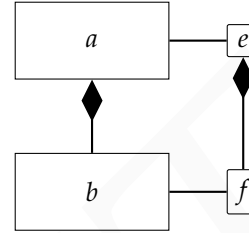
Example 6.2. Examples of identifiers are tags, system identifiers and database keys.

In our simple formal examples, we use $a, b, c, \dots$ to represent identifiers.

6.1.3 IMF Models

An IMF model is a language construct for grouping and organising facts expressed using the IMF vocabulary, i.e., it is essentially just a set of facts, a dataset. A fact is expressed using the vocabulary and identifiers.

Example 6.3. *The following is an IMF model:*

$$\{ \text{Block}(a), \text{connectedTo}(a, e), \text{Terminal}(e), \\ \text{Block}(b), \text{connectedTo}(b, f), \text{Terminal}(f), \\ \text{partOf}(b, a), \\ \text{partOf}(f, e) \}$$


The model states that a and b are Blocks, and e and f are Terminals. a is connectedTo e , b is connectedTo f , b is partOf a , and f is partOf e . This model complies with the axioms in [Remark 6.1](#).

6.1.4 IMF Types and IMF Templates

Informally, *types* are used by SMEs to express typical configurations of model elements, in order to support standardisation, reuse, and consistency and uniformity of information models. A type will typically represent a concept that is standard throughout an engineering discipline.

A *template* is used to consistently create a specific configuration of model elements, in order to simplify model construction and maintenance. The configuration that a template represents need not—but can—be standard, and may be created to support modelling consistency and uniformity within only a specific engineering project.

Hence, types and templates are similar, but they differ in their scope; types are expected to serve a wider use than templates, while templates will on the other hand typically represent more detailed configurations, often orchestrating and configuring types for particular modelling use cases. Instantiating types and templates from carefully curated libraries is a primary way of efficiently creating and maintaining information models of high quality.

This informal understanding of types and templates gives rise to the following more formal definition of IMF types and IMF templates.

An *IMF type* is represented by an expression over the IMF vocabulary that defines a class of all the objects that adhere to the configuration specified by the IMF Type. These objects are called instances of the type or type instances. An *IMF template* is a language construct for representing reusable modelling patterns over the IMF vocabulary. A template instance is a replica of the template's modelling pattern that follows the conditions that may be specified by the template. A template instance represents a set of facts over the vocabulary. Hence, a template instance represents a small IMF model.

There are hence clear similarities between an IMF type and an IMF template. The difference is that an IMF type *describes* a class of objects using the IMF vocabulary, while an IMF template is used to *generate* a set of facts over the IMF vocabulary. An IMF Template may refer to an IMF Type and hence generate facts that are instances of IMF Types.

Example 6.4. *A very simple (and artificial) IMF type is Block-with-Terminal-Type:*

$$\text{Block-with-Terminal-Type}(x) \rightarrow (\text{Block}(x) \wedge \exists y \text{Terminal}(y) \wedge \text{connectedTo}(x, y))$$

This IMF Type specifies the class of all Blocks that have a Terminal.

Example 6.5. A very simple IMF template is *Block-with-Terminal-Template*:

$$\text{Block-with-Terminal-Template}(x, y) \mapsto \{ \text{Block}(x), \text{Terminal}(y), \text{connectedTo}(x, y) \}$$

The x and y are the parameters of the template, and it represents the pattern of a *Block* with a *Terminal* connected to it.

An instance of the template, $\text{Block-with-Terminal-Template}(a, b)$, expands to the model:

$$\{ \text{Block}(a), \text{Terminal}(b), \text{connectedTo}(a, b) \}$$

Note that the object a is an instance of the IMF type *Block-with-Terminal-Type* from [Remark 6.4](#).

6.2 Vocabulary Structure

A vocabulary is a reserved set of terms which is split into *classes*, *relations* and *individuals*.

The single top-most class in the IMF vocabulary's class hierarchy is *Entity*; everything is a *Entity*. *Entity* is a very generic concept that is used by the ontology to state generic axioms that hold for all objects described by the ontology. *Entity* should not be directly instantiated, rather instantiate an appropriate subclass of *Entity*.

The ontology contains a limited number of individuals that represent reserved words in the vocabulary, such as names for *Aspects*. In this specification these individuals are presented as part of the presentation of the main class they are member of.

All relations are binary, i.e., all relationships relate two objects. As a high-level and informal categorisation of relations, we distinguish between:

- *Object* and *datatype* relations. Object relations relate two objects, while datatype relations associate a data value, such as a string, a number or a date, to an object.

For object relations the categorisation also includes:

- *Hierarchical* vs. *associative* relations. Hierarchical relations specify an ordering or ranking, while associative relations (or non-hierarchical relations) do not specify a clear ordering and often relate elements of similar rank (as specified by a hierarchical relation).
- *Internal* vs. *external* relations. Internal relations are defined with a domain and range that is specified by IMF classes, while *external relations* do not and will usually relate to objects from external reference data libraries.
- *Logical* vs. *annotation* relations. Logical relations have formal logical meaning which must be considered by logical reasoners, while annotations are typically used for metadata such as creation date and may be ignored by logical reasoners.

Unless otherwise noted, all relations are object type, associative, internal and logical.

6.3 Generic relations

Membership and equality relationships are generic relations that can be used to describe and reuse abstractions and categorisations. They are applicable to all *Entity*-s.

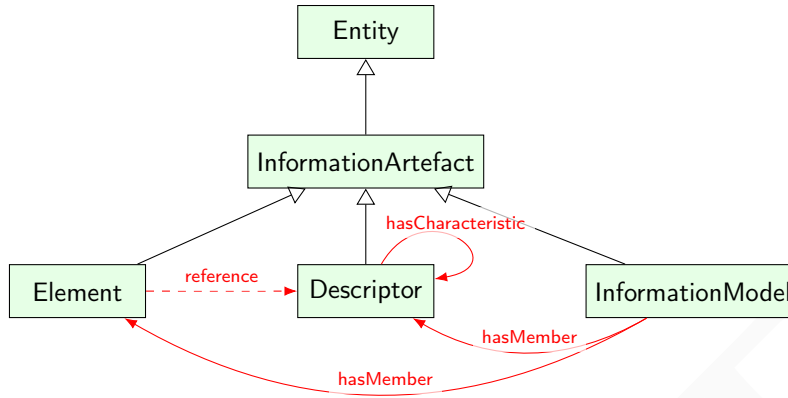


Figure 6.2: InformationArtefact and subclasses.

6.3.1 Equality

`sameAs` is a relation between Entity-s, such that `sameAs(a, b)` expresses that the Entity-s *a* and *b* are equal, i.e., *a* and *b* are different names/identifiers for the same Entity.

Axioms.

- (1) `sameAs` is a relation between Entity-s.
- (2) `sameAs` is reflexive, symmetric, and transitive.

6.3.2 Membership

`memberOf` is a relation between Entity-s, such that `memberOf(a, b)` expresses that the Entity *a* is a member of the Entity *b*, and hence that *b* is a collection or group for *a*. The inverse relation of `memberOf` is `hasMember`.

Axioms.

- (3) `memberOf` is a relation between Entity-s.
- (4) `hasMember` is the inverse of `memberOf`.

6.4 InformationArtefacts

Entity has a single subclass InformationArtefact that is the common class for the core categories of the ontology: Element, Descriptor and InformationModel (Section 6.4). These subclasses are pairwise disjoint.

Element (Section 6.6) is the core class of the ontology. All classes and relations in the ontology exist to describe Elements—either directly or indirectly.

A Descriptor is an InformationArtefact that serves to describe or identify a feature of an Entity.

A reference is a generic relation that relates a resource to a Descriptor.

Descriptors are associated with other Descriptors with the reference relation `hasCharacteristic`.

Axioms.

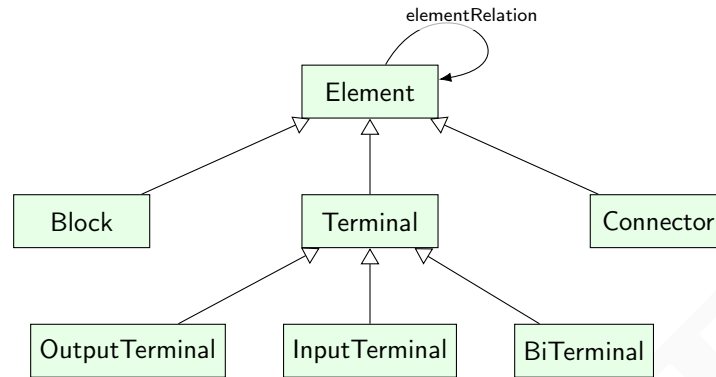


Figure 6.3: Element and its subclasses.

- (1) InformationArtefact is a subclass of Entity.
- (2) Element, Descriptor, and InformationModel are subclasses of InformationArtefact.
- (3) Element, Descriptor, and InformationModel are pairwise disjoint.
- (4) reference is a relation to a Descriptor.
- (5) hasCharacteristic is a subrelation of reference that relates Descriptors.

6.5 InformationModel

Elements and Descriptors may be collected into and organised by InformationModels, hence an InformationModel represents a set of such InformationArtefacts. Elements and Descriptor are associated with an InformationModel using the memberOf/hasMember relations.

6.6 Elements

Figure 6.3 displays the class Element and its subclasses. Figure 7.1 shows the graphical language for describing Elements.

6.6.1 Element

An Element represents the specification of an asset. All assets specifications are represented by an Element. An Element is described through its relations (Section 6.9) to other Elements, its Aspect (Section 6.7), and it may be assigned Attributes (Section 6.11).

An Element is either a Block, a Connector or a Terminal. Individuals rarely instantiate the class Element directly, rather they are stated to be an instance of a subclass of Element. (Element can be considered an “abstract class”.)

The permissive relationships for an Element are controlled by its type of subclass, and its Aspect.

An elementRelation is a generic relation that relates Elements.

Axioms.

- (6) An Element is either a Block, a Connector, or a Terminal.
- (7) Block, Connector, and Terminal are pairwise disjoint.
- (8) elementRelation is a relation between Elements.

6.6.2 Block

A Block represents an abstraction of any entity at any abstraction level. A Block also represents a boundary that separates what is internal and external to the Block. Elements that are part of the Block are internal to the Block and are not directly accessible to Elements outside (the boundary of) the Block. When a Block represents a system, it can be natural to consider the Block as a processing black box that transforms input to output, where the transformation process is detailed by the internals of the Block.

6.6.3 Connector

A Connector represents a topological connection between two Blocks, and it will specify conditions on this connection, such as the medium that is transferred between the Blocks.

A Connector can be considered as a Block with infinitely small boundary, i.e., a Block where there is no transformation of the input to the output at the current abstraction level of the Connector. A Connector may be further described by its parts.

6.6.4 Terminal

A Terminal represents a point on a Block's boundary where medium may pass, and it will specify conditions on the medium that may pass through it. A Terminal describes the permissible input and/or output of a Block. Any interaction with a Block goes through a Terminal of the Block.

A Terminal depends on a Block, hence a Terminal cannot exist without a Block.

Blocks may interact by transferring media between them ([Section 6.9.4](#)). A Terminal may refer to direction that specifies if the Terminal is set up to send or receive media. A Terminal that is specified to only receive input is called an InputTerminal. A Terminal that is specified to only send output is called an OutputTerminal. A Terminal that may both receive input and give output is called a BiTerminal.

Axioms.

- (9) direction is a subrelation of reference that is functional relation and associates a Terminal to a transfer direction.
- (10) An InputTerminal is a Terminal with direction input.
- (11) An OutputTerminal is a Terminal with direction output.
- (12) A BiTerminal is a Terminal with direction bidirectional.

6.6.5 Classifiers

classifier is a reference from an Element, such that $\text{classifier}(a, b)$ expresses that the Element a is classified by the Descriptor b . The relation is typically used to categorise Elements according to identifiers from standard reference data libraries or fixed enumerations for the purpose of connecting to and reusing well-known and shared identifiers and definitions.

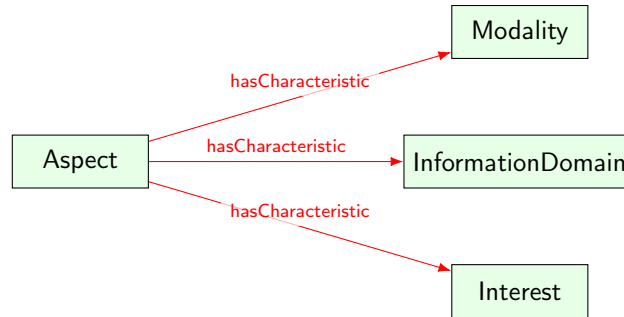


Figure 6.4: Aspect.

Subrelations of classifier define for more specific classification relations. We distinguish between *primaryClassifiers* that assign *essential* descriptions or features to Elements, and *auxClassifiers* (auxiliary) that assign *inessential* descriptions or features to Elements.

An Element should have at least one *primaryClassifier* that identifies the logically necessary types of an Element. Additional classifiers of Elements may be assigned using *auxClassifier*.

Axioms.

- (13) classifier is a subrelation of reference that relates an Element to a Descriptor.
- (14) *primaryClassifier* and *auxClassifier* are subrelations of classifier.

6.7 Aspects

Elements can be described from a specific point of view or aspect, in the IMF language this is represented by Aspects. An Aspect is a Descriptor specified by one Modality, one InformationDomain, and one optional Interest—all of which are also Descriptors.

Modality specifies the mode of existence of what is described. There are two predefined Modality-s: intended and actual.

Interest specifies the modelling interest or context from which the Element is described. There are two predefined Interests: *projectLifecycle* and *productLifecycle*

Aspects are associated with an InformationDomain that specifies what type of entities are in described by the Aspect. There are three predefined InformationDomains: *activity*, *implementation* and *space*.

The various combinations of Modality, InformationDomain and Interest give rise to different Aspects; these are listed in [Figure 6.5](#).

Axioms.

- (15) Aspect, Modality, Interest and InformationDomain are subclasses of Descriptor.
- (16) intended and actual are Modality-s.
- (17) *projectLifecycle* and *productLifecycle* are Interests.
- (18) *activity*, *space* and *implementation* are InformationDomains.

Modality	Interest	EntityType	Aspect
intended		activity	intendedActivity (functionAspect)
intended		space	intendedSpace (locationAspect)
intended		implementation	intendedImplementation (productAspect)
actual		implementation	actualImplementation (installedAspect)
intended	projectLifecycle	activity	intendedProjectActivity
intended	projectLifecycle	space	intendedProjectSpace
intended	projectLifecycle	implementation	intendedProjectImplementation
intended	productLifecycle	activity	intendedProductActivity
intended	productLifecycle	space	intendedProductSpace
intended	productLifecycle	implementation	intendedProductImplementation

Figure 6.5: Aspects

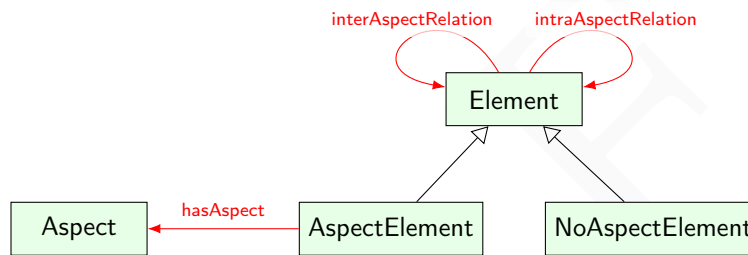


Figure 6.6: Aspects and Elements.

- (19) An Aspect is associated by one Modality, optionally one Interest and one InformationDomain via the hasCharacteristic relation.
- (20) intendedActivity, intendedSpace, intendedImplementation, intendedProjectActivity, intendedProjectSpace, intendedProjectImplementation, intendedProductActivity, intendedProductSpace and intendedProductImplementation are Aspects and are specified as in [Figure 6.5](#).

6.8 Aspect Elements

An Element may have at most one Aspect. An Element that has an Aspect is called an AspectElement.

It is convenient to be able to identify Elements that carry descriptions that are not of a single Aspect and where the Element's description is not analysed into separate Aspects. Such Elements are called NoAspectElements and have explicitly no Aspect.

Note that an Element where its Aspect is not known is not a NoAspectElement. Such an Element is neither an AspectElement nor a NoAspectElement, simply an Element.

Aspects are used to control the permissive relationships an Element may have, in part by way of the relations intraAspectRelation and interAspectRelation. An intraAspectRelation relates Elements that have the same value assigned as its Aspect, while an interAspectRelation relates Elements that have different values.

Axioms.

- (21) hasAspect is a subrelation of reference that is a functional relation from Element to Aspect.
- (22) An AspectElement is an Element with an Aspect assigned to it by the relation hasAspect.
- (23) A NoAspectElement is an Element with no Aspect.
- (24) intraAspectRelation and interAspectRelation are subrelations of elementRelation.
- (25) If two Elements are related by an intraAspectRelation they must either share the same Aspect or have no aspect.
- (26) If two Elements are related by an interAspectRelation they cannot share the same Aspect.

6.9 Relations

The IMF ontology contains a specific set of relations for describing Elements. The main relational structures are specialization, topology, partonomy, requirement–solution relationship, and relations between AspectElement that describe different aspects of the same entity. All these relations are subrelations of elementRelation, either directly or indirectly.

6.9.1 Specialization

specializationOf is a hierarchical relation and a subrelation of intraAspectRelation, such that specializationOf(a, b) expresses that the Element a is a specialization of the Element b . If a is a specializationOf b , then a inherits *all* the facts expressed for b .

specializationOf is used when specialized Elements are created from one or more generic Elements, or if the shared facts for multiple Elements are to be refactored into new generalized Element(s). Using specializationOf avoids repetition of facts and makes models more compact, possibly at the expense of added cognitive complexity to models.

Axioms.

- (27) specializationOf is subrelation of intraAspectRelation.
- (28) specializationOf is reflexive, anti-symmetric, and transitive.
- (29) If a is a specializationOf b , then a inherits all facts expressed for b .
- (30) specializes is the inverse relation of specializationOf.

6.9.2 Topology

Topological relations describe how Elements are connected and interact. Connected Elements are described on a similar level of abstraction.

The defining topology relation is connectedTo. connectedTo is an associative relation between Elements, such that connectedTo(a, b) expresses that a is topologically connected to b . adjacentTo is a subrelation of connectedTo, such that adjacentTo(a, b) expresses that a is directly topologically connected to b . That means that there is no element topologically “in between” a and b .

Two “shorthand” relations are also defined: hasTerminal is a subrelation of adjacentTo between Blocks and Terminals, while hasConnector is a subrelation of adjacentTo between Terminals and Connectors.

The difference between these topological relations is that connectedTo allows for expressing the topology of Elements between *any* kind of Elements, e.g., but not limited to, two Blocks, or a Block and a Connector,

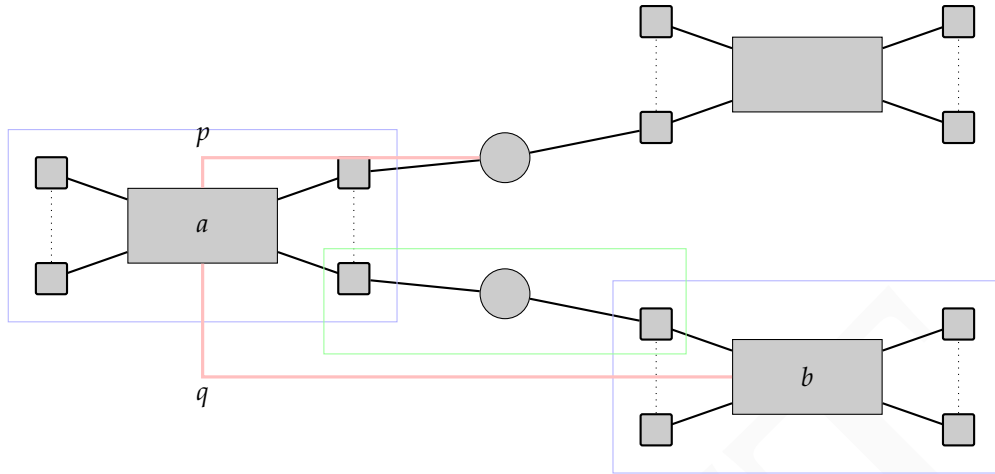


Figure 6.7: Pattern for topologically connected Elements: Blocks are through their Terminals (boxed in blue) connected via Connectors (boxed in green). A `connectedTo` relationship between a Block and a Connector (the edge labelled p) entails the existence of a Terminal between them. A `connectedTo` relationship between two Blocks (the edge labelled p) entails the existence of a pattern of the form of the green boxed pattern between them.

while the use of `adjacentTo` and its subrelations is more restricted. However, a `connectedTo` relationship is required to be realizable by a chain of `adjacentTo` relationships. This is to force the following reoccurring pattern in all topological chains of Elements:

$$\dots \rightarrow \text{Block} \rightarrow \text{Terminal} \rightarrow \text{Connector} \rightarrow \text{Terminal} \rightarrow \text{Block} \rightarrow \dots$$

Note however that each Element in the chain is not necessarily explicitly expressed, but may be left implicit (Figure 6.7). This can be useful in the case of representing incomplete information which may occur early in the modelling process or when the information to be modelled is at a very high abstraction level where, e.g., the conditions for Terminals and Connectors are not known, but it is clear which Blocks are connected.

The following axioms are defined for topological relations (Figure 6.8):

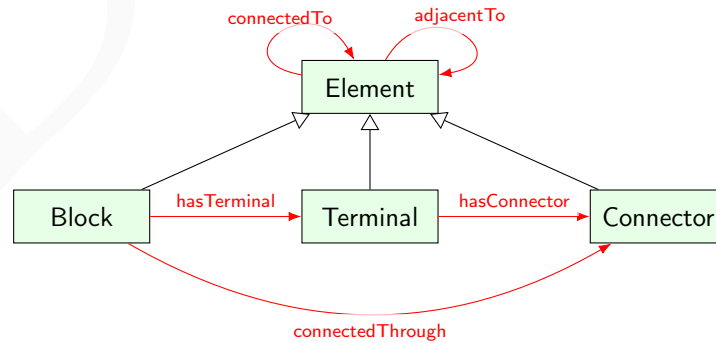


Figure 6.8: Topology relations.

Axioms.

- (31) `connectedTo` is an `intraAspectRelation`.
- (32) `adjacentTo` is a subrelation of `connectedTo`.
- (33) `connectedTo` and `adjacentTo` are symmetric.
- (34) `hasTerminal` is a subrelation of `adjacentTo` that associates a Block with its Terminals.
- (35) `connectedThrough` is a relation from a Block to a Connector such that the relation is equal to the composition of `hasTerminal` and `hasConnector`.
- (36) `hasConnector` is a subrelation of `adjacentTo` that associates a Terminal to its Connectors.
- (37) Blocks and Connectors may be `adjacentTo` to only Terminals.
- (38) Elements may be `connectedTo` any number of Elements.
- (39) A Block may be `adjacentTo` any number of Terminals.
- (40) A Terminal must be `adjacentTo` one Block.
- (41) A Connector must be `adjacentTo` exactly two Terminals.
- (42) A `connectedTo` relationship must be realizable by a sequence of `adjacentTo` relationships.

6.9.3 Partonomy

Partonomy relations relate Elements at different levels of abstraction. An Element can be described by relating it to its *parts*, which are Elements described at a lower level of abstraction, and by it relating to the *whole* it is part of, which is an Element at a higher level of abstraction. This recursive breakdown structure allows Elements to be described at incremental finer—or coarser, level of detail.

The defining partonomy relation is `partOf`. `partOf` is a hierarchical relation between Elements, such that `partOf(a, b)` expresses that *a* is part of *b*. In such a relationship, we call *a* “the part” or “child”, while *b* is called “the whole” or “parent”. Any Element is a `partOf` at most one other Element; the `partOf` hierarchy forms a tree-structured breakdown. `hasPart` is the inverse relation of `partOf`.

`containedIn` is the transitive closure of `partOf`, and `contains` is the transitive closure of `containedIn`.

The parts of an Element must form a legal topological structure as described in [Section 6.9.2](#). Elements can have any kind of Elements as parts, except Terminal whose parts must be Terminals. The parts of an Element are internal to the Element and may not directly connect to external Elements, except to Elements that are `partOf` the Terminal of the parent Element ([Figure 6.9](#)).

The following axioms are defined for partonomy relations ([Figure 6.10](#)):

Axioms.

- (43) `containedIn` is an `intraAspectRelation`.
- (44) `containedIn` is transitive.
- (45) The inverse relation of `containedIn` is `contains`.
- (46) `partOf` is a (non-transitive) subrelation of `containedIn`.
- (47) `partOf` is functional.
- (48) `partOf` is irreflexive.

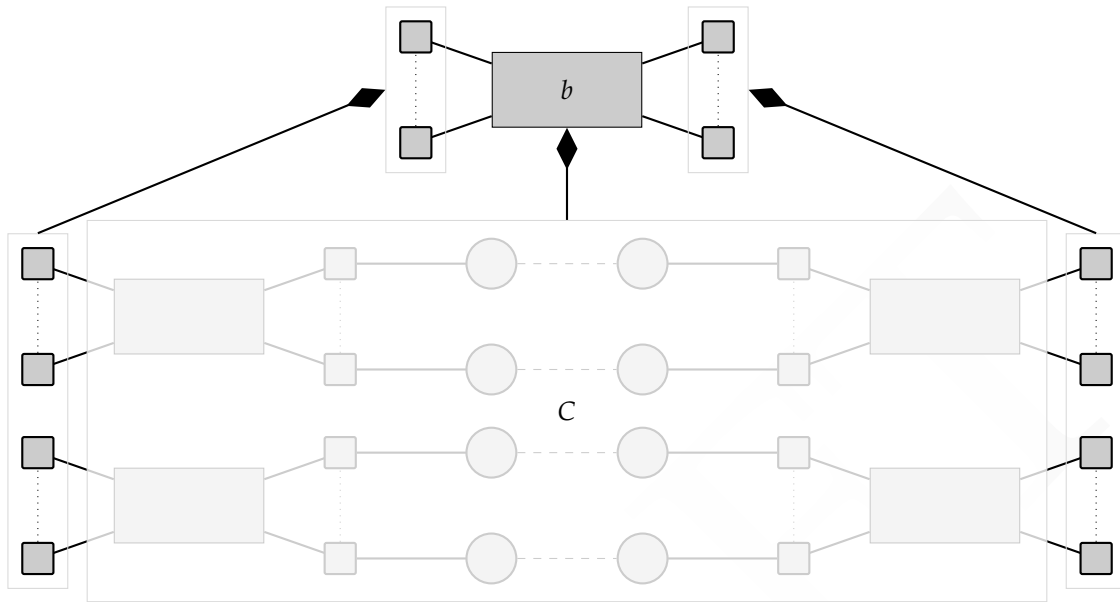


Figure 6.9: Pattern for breakdown hierarchies. The Block b is broken down to a set of Elements (boxed by C) that are all *partOf* b and form a legal topological structure, and is such that *adjacentTo* relationships from a part of b can only go to other parts of b or to the parts of b 's Terminals. All the Terminals that connect into an Element of C (and not between Elements of C) must be *partOf* one of b 's Terminals. Note that the surrounding boxes used in the diagram are simplifications; relations must go between Elements and not sets of Elements.

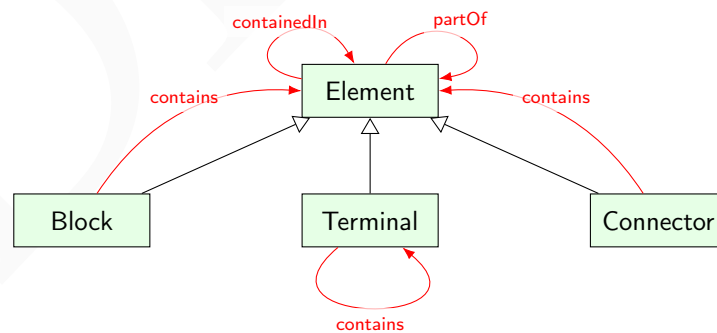


Figure 6.10: Partonomy relations.

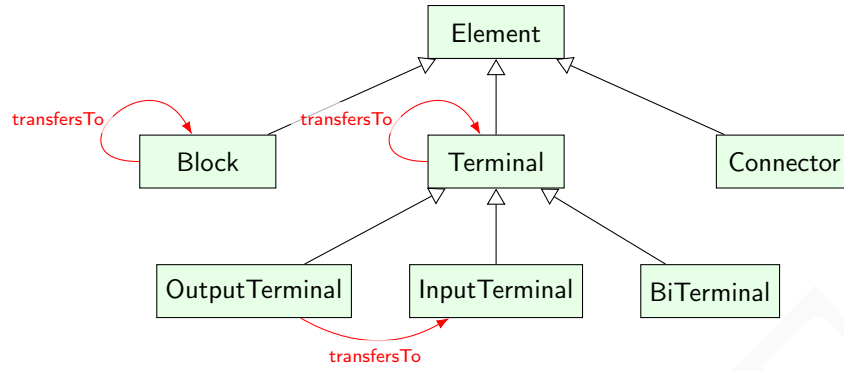


Figure 6.11: Media transfer relations.

- (49) hasPart is the inverse relation of partOf.
- (50) A containedIn relationship must be realized by a chain of partOf relationships.
- (51) A Terminal can only have Terminals as parts.
- (52) If two Elements are adjacentTo each other either, then they are each partOf two Elements that are either equal or they are also adjacentTo each other.

6.9.4 Media transfer

Topologically connected Blocks may interact by transferring media between them. The media transfer must respect the direction and media specified for the Terminals involved in the transfer.

The defining relation for media transfer is transfersTo. transfersTo is a relation between two Terminals or between two Blocks such that $\text{transfersTo}(a, b)$ expresses that (the source Element) a transfers to (the target Element) b . A transfersTo relation between two Blocks must be realized by a transfersTo relationship between Terminals that belong to the Blocks. transfersTo is a non-symmetric subrelation of connectedTo: only Elements that are connectedTo each other can transfersTo each other, with the following restrictions: an OutputTerminal may only transfersTo an InputTerminal, and a BiTerminal may only transfersTo a BiTerminal. An InputTerminal cannot be the source Element in a transfersTo relationship, it may only be the target.

medium is a subrelation of classifier that indicates the medium that a Terminal or Connector handles.

The following axioms are defined for media transfer relations (Figure 6.11):

Axioms.

- (53) medium is a subrelation of classifier that relates a Terminal or a Connector to an external medium classifier.
- (54) transfersTo is a relation between Terminals or between Blocks.
- (55) transfersTo is a subrelation of connectedTo.
- (56) transfersTo is irreflexive.
- (57) OutputTerminals may only transfersTo InputTerminals.
- (58) BiTerminals may only transfersTo BiTerminals.

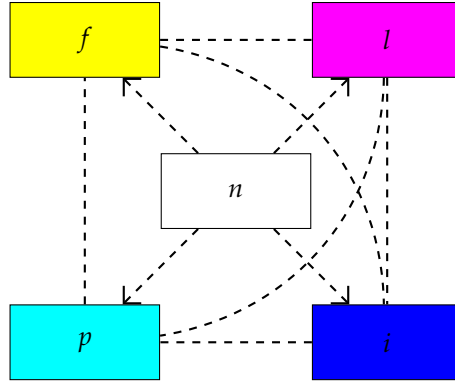


Figure 6.12: Example of cluster of proxy Elements. The AspectElements f , l , p and i are all proxy related, and projections of the NoAspectElement n .

(59) InputTerminals may not transfersTo.

(60) A transfersTo relationship between Blocks must be realized by a transfersTo relationships between Terminals.

6.10 AspectElement relations

Descriptions of different aspects of entities are represented by different AspectElements. AspectElement relations are relations that relate AspectElement which represent different Aspect of the same entity.

An entity may only be represented by at most one AspectElement for each Aspect, and one NoAspectElement.

There are two AspectElement relations. projection is the relation between a NoAspectElement and an AspectElement, such that $\text{projection}(a, b)$ expresses that a is an NoAspectElement representation of an entity and b (called the projection of a) is an AspectElement representations of the same entity. proxy is the relation between two AspectElements, such that $\text{proxy}(a, b)$ expresses that a and b are two different AspectElements that are projections of the same NoAspectElement (Figure 6.12).

The following axioms are defined for AspectElement relations:

Axioms.

(61) projection is an inverse functional interAspectRelation relation from a NoAspectElement to AspectElement.

(62) proxy is a interAspectRelation between two different AspectElements that are projections of the same NoAspectElement.

6.10.1 Requirement–Solution relations

Requirement–Solution relations are used to express that Elements may express requirements that other Element may satisfy and provide solution for.

fulfilledBy is a hierarchical relation between Element, such that $\text{fulfilledBy}(a, b)$ expresses that b is intended as a solution to the requirements posited by a .

The following axioms are defined for Requirement–Solution relations:

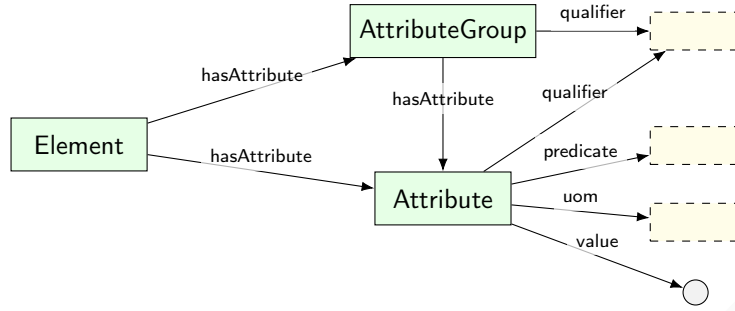


Figure 6.13: Attribute and AttributeGroups.

Axioms.

- (63) fulfilledBy is a subrelation of elementRelation.
- (64) fulfills is the inverse relation of fulfilledBy.

6.11 Attributes

Attributes and AttributeGroups are Descriptors for assigning data values to Elements. The vocabulary for expressing Attributes is shown in Figure 6.13. An Attribute can be specified with a predicate, uom (unit of measure) and a value.

Attributes may be grouped into AttributeGroups, using the relation memberOf or its inverse relation hasMember. memberOf(a, b) expresses that a is member of b .

Both Attributes and AttributeGroups can be qualified by different classifiers (see Section 6.6.5) as a means to describe them.

Axioms.

- (65) hasAttribute is a subrelation of reference between Element and Attribute.
- (66) uom is a subrelation of reference from Attribute.
- (67) value is a datatype relation from Attribute to a value.
- (68) predicate is a subrelation of classifier from an Attribute.
- (69) An Attribute can only be member of an AttributeGroup, and an AttributeGroup can only have Attributes as members.

Chapter 7

Visual Graph Notation

Abstract

This specification defines a visual graph notation for the Information Modelling Framework (IMF). The notation provides systematic guidelines for representing IMF models as node-edge diagrams, establishing a complete visual language that maps IMF's formal semantic constructs to intuitive graphical representations. The notation employs three primary visual encodings: geometric shapes distinguish element types (blocks, connectors, terminals, breakdowns, qualities, and attributes), edge styles represent relationships (topology, breakdown, cross-aspect), and fill colours denote aspects and media classifiers. This specification is designed for broad compatibility with graph-based languages and tools such as RDF, GraphML, and Graphviz, while remaining applicable to general-purpose diagramming and specialized system modelling environments.

Draft Notice *This chapter has not yet been updated to reflect recent developments in [Part I: Fundamentals of IMF](#). See the Draft Notice on [page i](#) for more details.*

Intended readers This chapter is written for modellers, tool developers, and visual designers who need to interpret create, or render graphical representations of IMF models. It assumes familiarity with the IMF core language ([Chapter 4](#)) and definitions ([Chapter 5](#)), but does not require detailed knowledge of the formal structural specification. Readers should be comfortable with graph-based modelling concepts and visual design conventions. The notation is implementation-neutral and can be applied using standard diagramming tools, graph visualisation software, or specialized IMF modelling environments.

The italicized keywords *must*, *must not*, *should*, *should not*, and *may* are used to specify normative features, and are interpreted as specified in RFC 2119.

7.1 Introduction

This specification defines a graph-based visual language for representing IMF models. In this notation, IMF models are represented visually using a graph (node-edge) model.

Elements are represented as nodes, where the different types of Elements, e.g., Block, Terminal and Connector, are represented using different node shapes and sizes. Relationships between Elements are represented as edges between nodes, where different relations are visualized using different arrowheads and line styles.

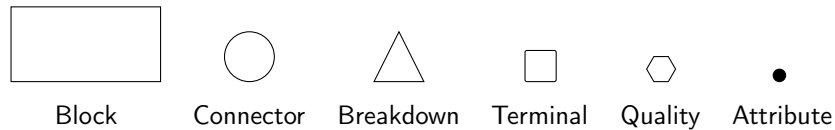


Figure 7.1: All node shapes.

Aspects are represented using different fill colours. Elements can be grouped and the visualization of the group can use visual effects to describe its Elements.

7.2 Nodes are Elements

Elements are visualized using nodes of different shapes as displayed in [Figure 7.1](#). They may also be coloured; this is described in [Section 7.4](#). Nodes *should* be labelled with an identifier or name when this is necessary to distinguish it from other nodes.

[Figure 7.2](#) contains additional node shapes. We use in this document an ellipsis with a dotted line to represent any Element. This is used to simplify the presentation of the different relationships in [Section 7.3](#). A diamond node shape is reserved for future versions of this specification. It is not used in this version.

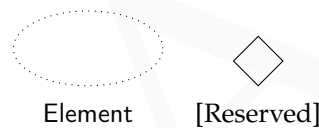


Figure 7.2: Additional node shapes: generic Element and reserved node shapes.

7.2.1 Block

A Block *must* be visualized as a rectangle, and *should* have square corners. The rectangle *should* be scaled so that the horizontal lines are about twice the length of its vertical lines.

7.2.2 Connector

A Connector *must* be visualized by a circle. The diameter of the circle *should* be about half the size of the vertical lines of the Blocks it is topologically adjacent to.

The node *should* be placed halfway between the Elements it relates.

7.2.3 Terminal

A Terminal *must* be visualized by a square, and *should* have rounded corners.

The Terminal node shape *should* be slightly smaller than the Connector it is topologically adjacent to.

A Terminal *should* be placed close to, or may even overlap with, its Block and *should* be placed closer to its Block than to its topologically adjacent Connector.

An InputTerminal *must* be labelled with a minus character. An OutputTerminal *must* be labelled with a plus character.

An InputTerminal *should* be placed left of its Block, and an OutputTerminal *should* be placed right of its Block.

7.2.4 Breakdown

A Breakdown *must* be visualized by a triangle, and *should* have sides of equal length.

The node shape *should* have the same size as the Connector node.

The node *should* be placed halfway between the parent Element and the child Elements it relates, and *should* be placed below the parent Element and above all child Elements.

7.2.5 Quality

Quality *should* be visualized by a hexagon with sides of equal length—alternatively a small circle.

The node shape *should* have a size which is smaller than a Terminal node.

The node *should* be placed close to the Element it is related to.

7.2.6 Attribute

Attribute *should* be visualized by a small dot—alternatively a small circle.

The node shape *should* have a size which is smaller than a Quality node.

The node *should* be placed close to the Element it is related to.

7.3 Edges are relationships

Relationships are visualized by edges (or lines or arrows) between two nodes. The visual encoding of a relationship is determined by four inputs:

1. the line style, which can be:

- solid: —
- dashed: - - - -
- dotted:

2. the arrowhead, which can be :

- head none: —
- head angle: —>
- head open triangle: —▷
- head diamond: —◆

3. the source node, and

4. the target node.

Edges may use different colours for grouping, highlighting or hiding information, but colours on edges have no formal meaning.

Note that the same edge style may be used to encode *different* relations, and that the relationship encodings are in such cases dependent on the source and target nodes.

This specification covers the most common, practical relations in IMF. However, it does not implement the full IMF vocabulary, and some relationships cannot be represented directly. In general, each source node + edge + target node relation is designed to capture the most specific available relationship. As a result, generic or high-level properties are often not supported.

Implementations of this specification *may* use labelling of edges to support more relations.

7.3.1 Topology

Topology relationships *must* be visualized by a solid line with no arrowhead, and *should* be oriented horizontally (see [Figure 7.3](#)).

Media transfer relationships, which also represent a topological relationship, are represented similarly to topology relationships, but with an arrowhead indicating the direction of the transfer. Media transfer *must* be visualized by a solid line with an arrowhead pointing from the source to the target of the transfer, and *should* be oriented horizontally, placing sources left of targets.

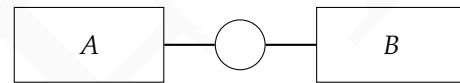


Figure 7.3: Example: Two Blocks connected by connectedTo relationship via a Connector.

Supported topology and media transfer relationships are shown in [Table 7.1](#).

7.3.2 Breakdown

Breakdown relationships *must* be visualized by a solid line and a solid diamond arrowhead pointing from the part and to the whole, and *should* be oriented vertically, placing parts below wholes (see [Figure 7.4](#)).

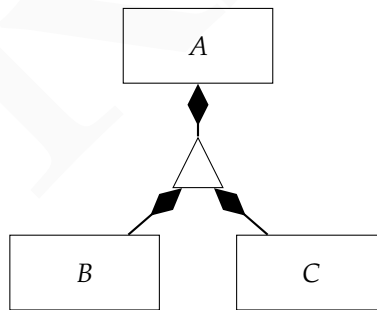


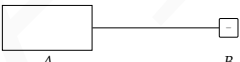
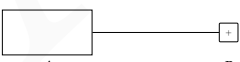
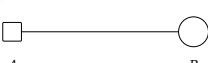
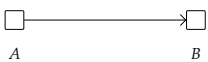
Figure 7.4: Example: Block A broken down into Blocks B and C via a Breakdown node.

Supported breakdown relationships are shown in [Table 7.2](#).

7.3.3 Cross-aspect relations

Cross-aspect relations are represented by dashed lines and different arrow heads.

Table 7.1: Topology relationship encodings

Style	Head	Source	Target	Visualization	Relationship
solid	none	Block	Block		<i>A</i> connectedTo <i>B</i>
solid	none	Connector	Connector		<i>A</i> connectedTo <i>B</i>
solid	none	Terminal	Terminal		<i>A</i> connectedTo <i>B</i>
solid	none	Block	Connector		<i>A</i> connectedVia <i>B</i>
solid	none	Block	Terminal		<i>A</i> hasTerminal <i>B</i>
solid	none	Block	InputTerminal		<i>A</i> hasInputTerminal <i>B</i>
solid	none	Block	OutputTerminal		<i>A</i> hasOutputTerminal <i>B</i>
solid	none	Terminal	Connector		<i>A</i> hasConnector <i>B</i>
solid	angle	Block	Block		<i>A</i> transfersTo <i>B</i>
solid	angle	Terminal	Terminal		<i>A</i> transfersTo <i>B</i>

Proxy relationships *must* be visualized by a dashed line without arrowhead, and *should* be oriented so that proxy elements appear as clusters, i.e., placed close to each other (see [Figure 7.5](#)).

Fulfills relationships *must* be visualized by a dashed line with an open arrowhead pointing from the solution element to the requirement element, and *should* be oriented vertically, placing solution Elements below requirement Elements (see [Figure 7.5](#)). fulfilledBy relationships are represented as the inverse of fulfills.

Projection relationships *must* be visualized by a dashed line with an angle arrowhead, and *should* be oriented so that the source of the projection appears to be the central node, either by placing it in the centre of or above the cluster or proxy elements.

Supported cross-aspect relationships are shown in [Table 7.3](#).

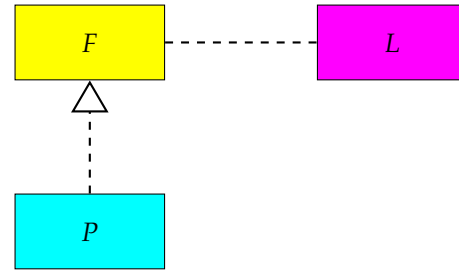


Figure 7.5: Examples of cross-aspect relationships: proxy relationship between two Blocks with different Aspects (yellow function and magenta location), and fulfills relationship with cyan product Block fulfilling a yellow function Block.

7.3.4 Quality and Attribute relations

Quality and Attribute relationships *must* be visualized by a solid line with no arrowhead, and *should* be oriented so that Quality and Attribute nodes are placed close to the node they relate to.

Quality breakdown relationships *must* be visualized by a solid line and a solid diamond arrowhead pointing from the part Quality to the whole Quality, and *should* be oriented vertically, placing part qualities below whole qualities.

[Figure 7.6](#) shows both hasQuality, hasAttribute, and Quality breakdown relationships.

Supported Quality and Attribute relationships are shown in [Table 7.4](#).

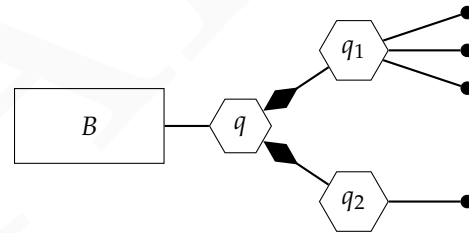


Figure 7.6: Example: Block with hasQuality relationship to a Quality q , which breaks down into Qualities q_1 and q_2 . Quality q_1 has three Attributes, while q_2 has one Attribute.

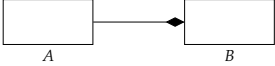
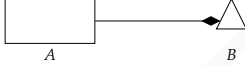
7.3.5 Merging nodes: Equality relationship

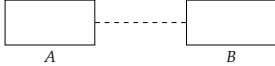
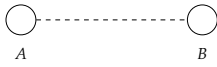
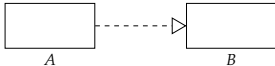
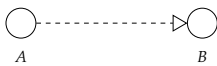
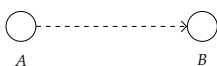
Sometimes it is necessary or convenient to represent one model object with multiple nodes, for instance when different visualizations representing overlapping information is merged.

Equality relationships *can* be used for this purpose as they to express that two objects are equal.

Representing objects with multiple nodes *should* be avoided due to the added complexity this adds to understanding visualizations.

These relationships *should* be visualized by an emphasized solid line or a dotted line, and *should* be labelled with an equals sign.

Style	Head	Source	Target	Visualization	Relationship
solid	diamond	Block	Block		$A ?? B$
solid	diamond	Connector	Connector		$A ?? B$
solid	diamond	Block	Breakdown		$A ?? B$
solid	diamond	Connector	Breakdown		$A ?? B$
solid	diamond	Breakdown	Block		$A ?? B$
solid	diamond	Breakdown	Connector		$A ?? B$
solid	diamond	Breakdown	Terminal		$A ?? B$

Style	Head	Source	Target	Visualization	Relationship
dashed	none	Block	Block		$A \text{ proxy } B$
dashed	none	Connector	Connector		$A \text{ proxy } B$
dashed	open triangle	Block	Block		$A \text{ fulfills } B$
dashed	open triangle	Connector	Connector		$A \text{ fulfills } B$
dashed	angle	Block	Block		$A \text{ projection } B$
dashed	angle	Connector	Connector		$A \text{ projection } B$

Aspect's Category	Colour name	Colour Code
Capability	Yellow	#ffff00 
Space	Magenta	#ff00ff 
Implementation	Cyan	#00ffff 
No Aspect	White	#ffffff 
Unspecified Aspect	Grey	#cccccc 



Figure 7.7: Predefined colours for Aspect's Capability, including a demonstration of their use on Blocks.

7.4 Fill colour and Aspects

Aspects *can* be visualized using a specified fill colour on the Elements' shape.

Colours are given in [Figure 7.7](#).

The colour of additional Aspects *should* be created by mixing the predefined colours so that it reflects the definition of the Aspect. For instance, an Aspect whose primary focus is Activity should be given a colour that is more yellow than magenta or cyan. [Figure 7.9](#) contains a list of suggested colours.

An Element may be left unspecified in the case that their Aspect is clear from surrounding Elements. This is in particular the case for Terminals that share the Aspect of the Block they belong to. [Figure 7.8](#) shows an example.

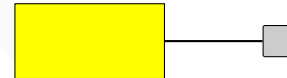


Figure 7.8: Example: A Terminal with unspecified Aspect will "inherit" the Aspect of its Block.

7.5 Fill colour and Medium

In addition to Aspects, fill colours *can* be used to assign a media classifier to Terminals. Suggested colours for media classifiers are given in [Figure 7.10](#).

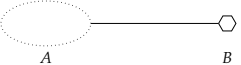
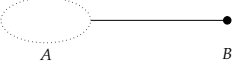
When both Aspect and Medium are relevant, the Aspect colour *should* take precedence for Blocks and Connectors, while the Medium colour *may* be applied to Terminals to distinguish the type of transfer.

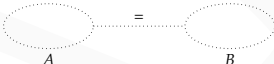
7.6 Grouping

Groups of Elements are visualized by placing the Elements in an enclosing shape, typically a rectangle. [Figure 7.11](#) gives an example of grouping.

7.7 References and metadata

The IMF language supports assigning more information to Elements; this includes Attributes, classifiers and metadata, such as textual definitions.

Style	Head	Source	Target	Visualization	Relationship
solid	none	Element	Quality		$A \text{ hasQuality } B$
solid	none	Quality	Attribute		$A \text{ hasAttribute } B$
solid	none	Element	Attribute		$A \text{ hasAttribute } B$
solid	diamond	Quality	Quality		$A ??? B$

Style	Head	Source	Target	Visualization	Relationship
equals	none	Element	Element		$A ?? B$

Act	Spa	Imp	Colour code
1			#ffff00 
	1		#ff00ff 
		1	#00ffff 
1	1		#ff8080 
1		1	#80ff80 
	1	1	#8080ff 
1	1	1	#aaaaaa 
2	1		#ffaa55 
2	1	1	#bfbf80 
2		1	#aaff55 
1	2		#ff55aa 
1	2	1	#bf80bf 
1	1	2	#80bfbf 
1		2	#55ffaa 
	2	1	#aa55ff 
	1	2	#55aaff 

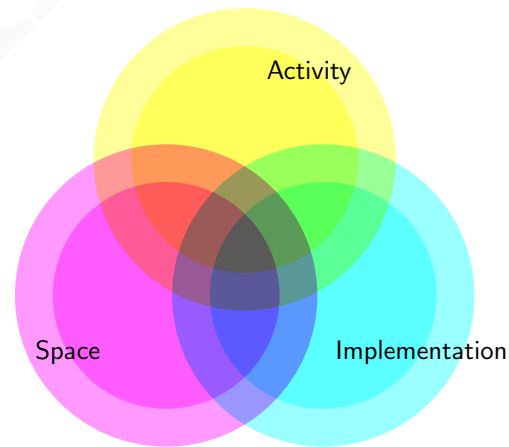


Figure 7.9: Suggested colours for Aspects. The columns Act-ivity, Spa-ce and Imp-lementation indicate how many parts of this colour is added to create the suggested colour.

Medium classifier	Colour name	Colour Code
Fluid	Light Blue	#66ccff 
Water	Blue	#3366ff 
Hydrocarbon	Brown	#993300 
Chemicals	Violet	#9900ff 
Solid material	Pink	#ff66ff 
Mechanical energy	Orange	#ff9900 
Electrical energy	Light Yellow	#ffff99 
Thermal energy	Red	#ff3333 
Mechanical force	Dark Grey	#333333 
Information	Green	#00cc33 
Unspecified Medium	Grey	#cccccc 

Figure 7.10: Predefined colours for Medium classifiers.

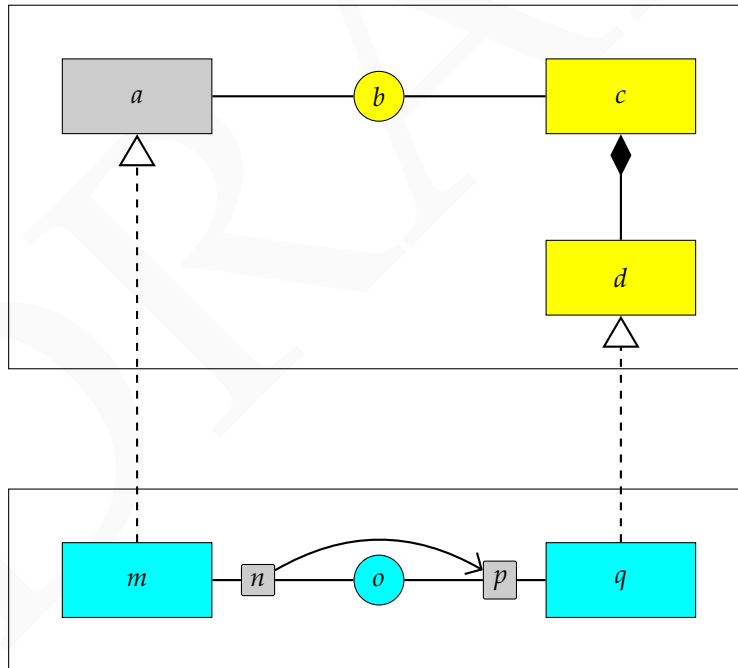


Figure 7.11: Example of grouping. The visualization contains two groups: the Elements labelled a , b , c and d ; and m , n , o , p and q .

In order to keep visualizations of IMF models simple, these additional information objects do not have a graphical representation. Rather, if necessary, they can be represented as lists of attributes that are associates with its Element using appropriate features of the used visualization tool or language.

Part III

IMF as a Framework

Chapter 8

IMF Metamodel Implementation

Abstract

This chapter describes the concrete implementation of the IMF metamodel, bridging the formal structural specification ([Chapter 6](#)) and the user-facing serialisation formats ([Chapter 9](#)). It presents two implementations: the current normative *IMF OWL Ontology and SHACL Shapes*, which define the semantic and validation layer, and the emerging *LinkML implementation*, which serves as a single authoring source that can generate multiple artefacts. The chapter explains why OWL and SHACL are used, details the contents of the ontology and shapes files with examples, and outlines the rationale, benefits, and risks of transitioning to LinkML.

Draft Notice *This chapter has not yet been updated to reflect recent developments in [Part I: Fundamentals of IMF](#). See the Draft Notice on page [i](#) for more details.*

Intended Readers This chapter is written for ontology engineers, semantic web practitioners, tool developers, and advanced data architects who need to understand how the IMF language is implemented in concrete artefacts. It assumes familiarity with the core concepts ([Chapter 4](#)) and the structural specification ([Chapter 6](#)).

8.1 Introduction

While [Chapter 6](#) defines the logic of IMF (structural definitions, intensions, and evaluations), this chapter describes the technical artefacts that make that logic executable in software. In the current release, IMF provides two implementations of the metamodel:

1. **IMF OWL Ontology and SHACL Shapes (normative).** This is the stable, published definition of IMF for semantic reasoning and validation.
2. **LinkML (work in progress).** This is the emerging authoring source that will generate OWL/SHACL and other formats from a single schema definition.

8.2 IMF OWL Ontology and SHACL Shapes

The IMF metamodel is published as W3C-standard Semantic Web artefacts. These files translate the IMF structural specification into machine-readable axioms and constraints.

8.2.1 Why OWL and SHACL

OWL (Web Ontology Language) and SHACL (Shapes Constraint Language) are chosen because they provide complementary capabilities that match IMF requirements. Understanding their distinct roles is essential for practitioners working with IMF artefacts.

8.2.1.1 Complementary World Assumptions

OWL and SHACL operate under fundamentally different logical assumptions:

- **Open World Assumption (OWL):** In OWL, the absence of information does not imply falsity. If a property value is not stated, it is considered unknown rather than false. This allows for incremental modelling and extensibility—new information can be added without invalidating existing assertions.
- **Closed World Assumption (SHACL):** SHACL validates what is explicitly present in the data. If a required property is missing, SHACL reports a validation error. This is essential for data quality enforcement and conformance checking.

IMF requires both perspectives: OWL enables extensible semantic reasoning, while SHACL ensures that published models meet explicit structural requirements.

8.2.1.2 Semantic Reasoning with OWL

OWL captures the intensional structure of IMF and allows reasoners to infer implicit relationships and detect logical inconsistencies. Key reasoning capabilities include:

- **Property characteristics:** The `imf:partOf` property is declared transitive, allowing reasoners to infer the transitive closure of part-whole hierarchies. Similarly, `imf:connectedTo` is symmetric, so asserting that Block A is connected to Block B automatically implies the reverse.
- **Class subsumption:** Reasoners can infer that an instance of `imf:Block` is also an instance of `imf:Element` by traversing the class hierarchy.
- **Consistency checking:** OWL reasoners detect logical contradictions, such as disjoint class violations (for example, an individual cannot be both an `imf:Block` and an `imf:Terminal`, as these are declared disjoint).
- **Property restrictions:** OWL restrictions express constraints such as “a `imf:Terminal` is (always) `imf:adjacentTo` a `imf:Block`” enabling type-level validation through reasoning.

These capabilities support automatic model integration, consistency verification, and discovery of implicit knowledge in IMF models.

8.2.1.3 Structural Validation with SHACL

SHACL provides explicit, closed-world constraints that check whether an IMF model conforms to the required grammar. SHACL validation complements OWL reasoning in several ways:

- **Cardinality enforcement:** SHACL shapes can require that a `imf:Block` has at least one `imf:Terminal` (`sh:minCount 1`), or that an `imf:Attribute` has exactly one `imf:predicate`. OWL alone cannot express such closed-world cardinality requirements.
- **Value type checking:** SHACL validates that property values conform to expected datatypes (for example, `imf:value` must be a literal) or node kinds (IRIs versus blank nodes).
- **Vocabulary conformance:** The IMF terms grammar acts as a whitelist, preventing accidental creation of misspelled or non-standard IMF terms.
- **User-facing error messages:** SHACL validation reports provide actionable feedback for modellers, specifying exactly which constraints were violated and where.

SHACL validation should be applied before publishing or exchanging IMF models to ensure data quality and conformance to the metamodel.

8.2.1.4 Vendor-Neutral Interoperability and Tooling Ecosystem

Both OWL and SHACL are W3C standards supported by a mature ecosystem of open-source and commercial tools, including

- **Ontology editors** for browsing, editing, and visualising OWL ontologies, including class hierarchies and property definitions.
- **Reasoning engines** that provide OWL reasoning capabilities for entailment and consistency checking.
- **SHACL validators** that support validation of RDF graphs against SHACL shapes.
- **Triple stores** that provide scalable storage, SPARQL querying, and reasoning for IMF RDF knowledge graphs.
- **Programming libraries** enable developers to build IMF-aware applications in multiple programming environments.

This broad tool support ensures that IMF implementations are not locked into proprietary platforms and can be integrated into diverse enterprise and research workflows.

8.2.2 Namespaces and IRIs

The normative publication for this version is hosted under the versioned namespace:

- <http://ns.imfid.org/20240531/>

The IMF vocabulary itself is minted in the stable namespace:

- `imf:` <http://ns.imfid.org/imf#>

This separation allows stable term IRIs for models, while versioned artefact IRIs capture specific releases. In practice, ontology and shape files in the release namespace import or reference the stable IMF term IRIs.

8.2.3 IMF Ontology

The IMF ontology file defines the vocabulary and logical structure of the framework. It provides:

- **Class hierarchy:** A clear taxonomy of IMF elements (for example, `imf:Block` as a subclass of `imf:Element`).

- **Property characteristics:** Core relations are given explicit semantics, such as transitivity for `imf:partOf` and symmetry for `imf:connectedTo`.
- **Human-facing annotations:** Terms are labelled and described using standard annotation properties (for example, `skos:prefLabel` and `rdfs:comment`).

The normative ontology file for this release is published at:

- <http://ns.imfid.org/20240531/imf-ontology.owl.ttl>

The ontology is intentionally lightweight, prioritising readability and predictable reasoning behaviour over highly complex description logic constructs. Naming conventions follow `UpperCamelCase` for classes and `lowerCamelCase` for properties.

[Listing 8.1](#), which lists the definition of the class `imf:Block`, and [Listing 8.2](#), which lists the definition of the property `imf:connectedTo`, illustrate the typical modelling patterns used in the IMF ontology, taken from the published OWL ontology. These demonstrate both class definitions with restrictions and property definitions with semantic characteristics.

```
@prefix imf: <http://ns.imfid.org/imf#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix vann: <http://purl.org/vocab/vann/> .
@prefix vs: <http://www.w3.org/2003/06/sw-vocab-status/ns#> .

imf:Block
  vann:termGroup "elements" ;
  a owl:Class ;
  rdfs:subClassOf imf:Element,
    [ a owl:Restriction ;
      owl:allValuesFrom imf:Terminal ;
      owl:onProperty imf:adjacentTo ],
    [ a owl:Restriction ;
      owl:allValuesFrom imf:Block ;
      owl:onProperty imf:transfersTo ] ;
  vs:term_status "testing" ;
  skos:changeNote "0.1.0 :: Added",
    "0.3.0 :: Changed: Update definition and scopeNote" ;
  skos:definition """A [Block] represents an abstraction of any entity
    at any abstraction level. A [Block] also represents a boundary
    that separates what is internal and external to the [Block].
    [Element]s that are part of the [Block] are internal to the
    [Block] and are not directly accessible to [Element]s outside
    (the boundary of) the [Block]...""" ;
  skos:prefLabel "Block" ;
  skos:scopeNote """A [Block] is a basic building block of the IMF
    language. A [Block] can represent anything which is of interest
    and which is deemed convenient to treat as a system or system
    element...""" .
```

Listing 8.1: OWL definition of `imf:Block` class

[Listing 8.1](#) demonstrates the following features:

- **Class hierarchy:** `imf:Block` is declared as a subclass of `imf:Element`, inheriting all properties and characteristics of the parent class.
- **OWL restrictions:** The two `owl:Restrictions` constrain values of specific properties:

- `owl:allValuesFrom imf:Terminal on imf:adjacentTo` means “if a Block has any `adjacentTo` relations, all such related entities must be Terminals.”
- `owl:allValuesFrom imf:Block on imf:transfersTo` similarly restricts media transfer relations to other Blocks.
- **Semantic annotations:** The definition uses standard vocabularies (SKOS, VANN, VS) to provide human-readable labels, definitions, scope notes, and change tracking. These annotations support both documentation generation and ontology governance.
- **Term status tracking:** The `vs:term_status "testing"` annotation indicates the maturity level of this vocabulary term, allowing controlled evolution of the ontology.

```
imf:connectedTo
  vann:termGroup "topology" ;
  a owl:ObjectProperty, owl:SymmetricProperty ;
  rdfs:domain imf:Element ;
  rdfs:range imf:Element ;
  rdfs:subPropertyOf imf:associativeRelation, imf:intraAspectRelation ;
  vs:term_status "testing" ;
  skos:changeNote "0.1.0 :: Added",
    "0.2.0 :: Changed IRI from imf:isConnectedTo to
      imf:connectedTo.",
    ""0.3.0 :: Changed:
      - domain and range from Terminal to Element.
      - removed functional axiom.
      - added symmetric axiom."" ;
  skos:definition ""[connected to] is an [associative relation] and
    an [intra-aspect relation] that relates an [Element] *E1* to an
    [Element] *E2* to specify that *E1* is connected to *E2*.
    [connected to] specifies a topology of [Element]s."" ;
  skos:prefLabel "connected to" .
```

Listing 8.2: OWL definition of `imf:connectedTo` property

Listing 8.2 demonstrates following features:

- **Property characteristics:** The `owl:SymmetricProperty` type means that if `E1 imf:connectedTo E2` is asserted, an OWL reasoner will automatically infer `E2 imf:connectedTo E1`. This reflects the bidirectional nature of topological connections.
- **Domain and range constraints:** Both domain and range are `imf:Element`, indicating that this property relates Elements to Elements. These constraints enable type checking and support reasoning about element connectivity.
- **Property hierarchy:** `imf:connectedTo` is declared as a subproperty of both `imf:associativeRelation` (indicating non-hierarchical structure) and `imf:intraAspectRelation` (indicating that connected elements share aspects). This multiple inheritance enables flexible querying and inference.
- **Change tracking:** The detailed `skos:changeNote` annotations document the evolution of this property across versions, including changes to domain/range and the addition of the symmetric characteristic. This supports ontology versioning and backward compatibility analysis.

Implications for practitioners These examples illustrate several design principles of the IMF ontology:

1. **Lightweight Description Logic:** The ontology prioritises readability and predictable reasoning behaviour over highly complex axioms. OWL restrictions are used judiciously to express essential structural constraints without creating computational bottlenecks.

2. **Consistent naming conventions:** Classes use UpperCamelCase (for example, Block, Terminal), properties use lowerCamelCase (for example, connectedTo, partOf), and individuals use lowerCamelCase (for example, functionAspect).
3. **Rich metadata:** Every significant term includes `skos:prefLabel`, `skos:definition`, and often `skos:scopeNote` and `skos:example`. This makes the ontology self-documenting and supports automated documentation generation.
4. **Vocabulary organisation:** The `vann:termGroup` annotation (for example, “elements”, “topology”, “attributes”) organises terms into coherent groups for documentation and navigation.

Practitioners extending or implementing IMF should follow these patterns to maintain consistency with the normative ontology.

8.2.4 IMF SHACL Shapes

The SHACL suite complements the ontology by defining explicit validation rules. The normative file for vocabulary correctness is:

- <http://ns.imfid.org/20240531/imf-terms-grammar.shacl.ttl>

The terms grammar enforces vocabulary correctness within the IMF namespace. It effectively acts as a whitelist of allowed IRIs and prevents accidental creation of new or misspelled IMF terms.

Example pattern A typical shape targets a class and constrains required relations:

```
@prefix imf: <http://ns.imfid.org/imf#> .
@prefix sh: <http://www.w3.org/ns/shacl#> .

imf:BlockShape
  a sh:NodeShape ;
  sh:targetClass imf:Block ;
  sh:property [
    sh:path imf:hasTerminal ;
    sh:minCount 1 ;
  ] .
```

Listing 8.3: SHACL shape for `imf:Block`

8.3 The LinkML Implementation

LinkML (Linked Data Modelling Language), <https://linkml.io/>, is an open-source project that provides a high-level modelling language based on YAML formatted files. In IMF, LinkML is being introduced as an authoring layer that can generate the normative OWL and SHACL artefacts, as well as additional formats needed by tool builders (see [Section 8.3.2](#) for details). The transition to LinkML is driven by the need for a single source of truth to reduce divergence between OWL and SHACL artefacts, multi-format generation capabilities, and improved developer accessibility through YAML-based schemas. The LinkML implementation is *work in progress*.

8.3.1 Schema Organisation

The LinkML implementation is organised as modular YAML files that mirror the conceptual structure of the IMF language:

- `core.yaml` defines core elements such as `Block`, `Terminal`, and the primary relations.
- `aspects.yaml` defines `Aspect` and the cross-context relations used to model engineering viewpoints.
- `attributes.yaml` defines attribute structures, value handling, and unit metadata.

A preliminary LinkML definition for `imf:Block` is given in [Listing 8.4](#).

```
classes:
  Block:
    is_a: Element
    description: >
      A Block represents an abstraction of any entity at any
      abstraction level...
    slots:
      - hasTerminal
      - adjacentTo
      - transfersTo
    slot_usage:
      hasTerminal:
        range: Terminal
        required: true
        multivalued: true
      adjacentTo:
        range: Terminal
      transfersTo:
        range: Block
```

Listing 8.4: LinkML definition for `imf:Block`

8.3.2 Overview of LinkML

LinkML provides comprehensive capabilities for IMF metamodel development:

- **Multi-format generation:** From a single YAML-based schema definition, LinkML generates RDF/OWL ontology files for reasoning and semantic interoperability, SHACL shapes for validation, JSON-LD context and RDF serialisations for linked data interchange, JSON Schema for web and API tooling, Python data classes for tooling and SDKs, and HTML or Markdown documentation for human-readable reference. LinkML class inheritance maps to `rdfs:subClassOf`, slot constraints translate to OWL restrictions and SHACL property shapes, and cardinality and datatype constraints are enforced across all generated artefacts.
- **Data transformation and validation:** The LinkML runtime converts instance data between JSON, YAML, RDF (Turtle, JSON-LD, RDF/XML), and TSV formats while preserving semantics. The validation framework checks instance data against schemas, reporting violations of cardinality, datatype, and enumeration constraints, providing an alternative to SHACL validation that operates directly on JSON or YAML data before RDF conversion. LinkML can also infer schema elements from example data, supporting incremental schema development.
- **Documentation generation:** The `gen-markdown` command produces browsable reference documentation with dedicated pages for classes, slots, types, and enumerations, with automatic cross-referencing and hyperlinks. For schema visualisation, LinkML can generate UML-style class diagrams, showing inheritance hierarchies, composition relationships, and multiplicities. LinkML can also produce searchable, interactive websites with navigation sidebars, version selection, and responsive design, enabling automatic generation of implementer-facing documentation directly from the LinkML schema.

The adoption of LinkML as the authoring source for IMF metamodel artefacts offers several benefits:

- **Single source of truth:** Maintaining OWL and SHACL in separate files risks divergence over time (for example, a property added to OWL but not to SHACL). LinkML eliminates this by generating both from a single schema.
- **Developer accessibility:** YAML-based schemas are more approachable for domain experts and software engineers than raw RDF/Turtle syntax. This lowers the barrier to contributing to metamodel evolution.
- **Automated consistency:** LinkML's schema validation ensures internal consistency (for example, all referenced ranges must be defined classes) before generation, catching errors early.
- **Reduced maintenance burden:** Changes to the metamodel require editing only the LinkML schema, with all downstream artefacts regenerated automatically. This significantly reduces the effort required for metamodel versioning.

As this is work-in-progress, there are several considerations to address:

- **Generator coverage:** LinkML generators may not yet support all OWL or SHACL constructs used in the current normative artefacts (for example, complex OWL property chains, advanced SHACL-SPARQL constraints). The IMF project maintains a feature compatibility matrix and contributes upstream to LinkML to address gaps.
- **Artefact fidelity:** Automatically generated OWL/SHACL must be semantically equivalent to hand-crafted versions. Continuous integration pipelines compare generated artefacts against reference versions using diff tools and semantic reasoners.
- **Tooling maturity:** LinkML is a relatively recent project (initial release 2021) and its ecosystem is less mature than OWL/SHACL tooling. The normative OWL/SHACL artefacts remain the operational baseline for production use until LinkML tooling reaches equivalent maturity.
- **Community familiarity:** IMF stakeholders may be unfamiliar with LinkML, requiring training and documentation. The IMF project provides migration guides and maintains both LinkML and generated artefacts in the repository for transparency.

Chapter 9

IMF Serialisation Formats

Abstract

This specification describes the different serialisation formats for the Information Modelling Framework (IMF), establishing how IMF models are represented, exchanged, validated, and deployed across different technological contexts. The specification presents four complementary serialisation formats: RDF, OWL, SHACL, and AAS, each providing distinct representations for specific purposes within the digital engineering ecosystem: as semantic knowledge graphs, as formal ontologies for reasoning, as data validation contracts, and as deployed digital twins.

The specification describes systematic transformation patterns between formats, detailing how IMF constructs—elements, relationships, attributes, and constraints—are mapped to each target representation while preserving semantic intent and traceability.

Intended readers This chapter is written for data engineers, integration specialists, and tool developers who need to implement, exchange, or validate IMF models in practice. It assumes familiarity with the IMF language ([Chapter 5](#)) and basic understanding of semantic web technologies (RDF, OWL, SHACL). Knowledge of the formal structural specification ([Chapter 6](#)) is helpful but not required. Readers seeking to integrate IMF with existing data platforms, implement validation pipelines, or deploy IMF-based digital twins will find this chapter essential. Practitioners may also consult this chapter selectively based on their specific serialisation needs.

9.1 RDF: Canonical and Versatile Exchange Format

9.1.1 Purpose and scope

RDF (Resource Description Framework) serves as the canonical serialisation format for IMF models. It functions as the common exchange format from which all other serialisations are derived. The RDF serialisation provides a neutral, open standards-based, graph-based foundation that captures the full semantics of IMF models while remaining independent of any specific reasoning or validation technology.

IMF models are authored and exchanged as RDF graphs that follow the IMF vocabulary. The RDF representation preserves the essential graph structure of models—where elements are connected through multiple relationship types—without imposing artificial hierarchies or constraints.

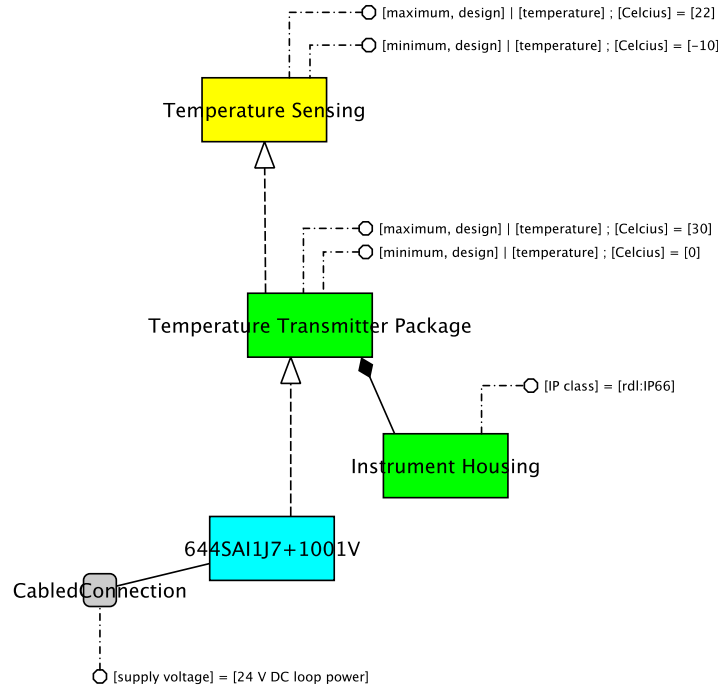


Figure 9.1: Sample IMF model of a temperature transmitter package, visualised using IMF graphical language.

9.1.2 Key characteristics

The IMF RDF representation is fundamentally *instance-centric* (ABox-style). Each concrete model element—whether a block, terminal, attribute, or connector—is represented as an RDF individual identified by an IRI. This approach reflects the nature of system modelling: we model specific system elements and their actual relationships, not abstract schemas.

Element categorisation is expressed through `rdf:type` statements, allowing each element to declare its type within the IMF vocabulary (e.g., `imf:Block`, `imf:Terminal`, `imf:Quality`, `imf:Attribute`). The model structure emerges from triples that connect these individuals:

- Terminals and attributes are linked to their parent blocks through containment predicates
- Relations between elements are expressed via `imf:elementRelation` and its subproperties
- References to external entities or other model elements use subproperties of `imf:reference`

9.1.3 Representation details

RDF's triple structure (subject-predicate-object) naturally expresses the relationships in IMF models. Each triple typically represents one relationship between two elements in an IMF model.

Throughout this chapter, we use the following RDF fragment, and similar fragments taken from the IMF model shown visually in [Figure 9.1](#), as examples to illustrate how IMF models are transformed into OWL, SHACL, and AAS representations:

```

id:n10 a imf:Block ;
    imf:fulfills id:n9 ;
    imf:hasAspect imf:intendedProjectImplementation ;
    imf:hasAttribute id:n4, id:n5 ;
    skos:prefLabel "Temperature Transmitter Package" .

id:n4 a imf:Attribute ;
    imf:hasAttributeQualifier imf:designQualifier, imf:minimumQualifier ;
    imf:predicate rdl:temperature ;
    imf:uom rdl:celcius ;
    imf:value 2 ;
    skos:prefLabel "[minimum, design] | [temperature] ; [Celcius] = [2]" .

```

Listing 9.1: Example RDF fragment of Temperature Transmitter Package

This example shows a block (id:n10) representing a “Temperature Transmitter Package” with two attributes. The attribute id:n4 demonstrates IMF’s attribute modelling: the attribute has qualifiers indicating this is a minimum design value, references external predicates and units, and carries the actual value.

Literal data—such as labels, descriptions and attribute values—is represented using RDF literals attached via predicates like `imf:value`. Where attributes have qualified constraints, these are expressed through additional triples specifying qualifiers, predicates, and units of measure.

The resulting IMF model is a connected graph of individuals, structured and described by the IMF ontology’s vocabulary, where the graph topology directly represents system structure and relationships.

9.1.4 Benefits and use cases

RDF as the baseline format provides several fundamental advantages:

Identifier precision through IRIs: Every element, relationship, and property is identified by an IRI, enabling unambiguous reference and global uniqueness. This supports distributed modelling where different teams can reference the same concepts without collision.

Universal graph representation: The RDF graph model naturally accommodates the multi-relational, hierarchical and non-hierarchical structure of models. Unlike tree-based serialisation formats (XML, JSON), RDF handles arbitrary cross-references and multiple relationship types without awkward encoding schemes.

Extensibility: RDF graphs can be merged, extended, and augmented with additional triples without breaking existing structure. New relationship types or properties can be added incrementally.

Alignment and integration: RDF enables semantic alignment with external vocabularies and ontologies (e.g., industry standards, reference data) through standard mechanisms like `owl:sameAs` and `skos:exactMatch`.

Technology neutrality: As a W3C standard, RDF is supported by a mature ecosystem of tools, databases (triple stores), and query languages (SPARQL). Models can be processed, queried, and integrated using standard technologies.

Primary use cases for the RDF serialisation include:

- Model exchange between organisations and tools
- Integration with semantic web technologies and linked data
- Querying complex model relationships using SPARQL

- Foundation for deriving specialised serialisations (OWL, SHACL, AAS)

9.1.5 Limitations

While RDF provides essential semantic flexibility, it has inherent limitations that motivate the need for additional format representations:

No built-in validation: RDF itself has no standard mechanism to declare that certain properties are required, or that values must fall within specific ranges. An RDF graph is simply a set of true statements—it cannot express “this graph is invalid because property X is missing.”

Limited inference capabilities: RDF alone provides very limited inference capability and lacks the formal semantics needed for sophisticated reasoning about constraints, restrictions, and implied structure.

No standardised packaging: RDF graphs are abstract data structures. For physical exchange—especially in industrial contexts—standardised packaging formats with metadata, versioning, and toolchain integration are needed.

Graph-centric vs. consumption patterns: Many industrial tools and workflows expect hierarchical or tabular, asset-oriented data structures rather than arbitrary graphs. Direct RDF consumption may require additional adapter transformations.

These limitations are not deficiencies of RDF itself, but rather recognition that different tasks require different representational strengths. The following sections describe how OWL, SHACL, and AAS complement RDF by addressing these specific needs. While RDF is the canonical exchange format for IMF models, most engineer users will not need to work directly with RDF syntax. In practice, RDF serves as an interoperability layer that enables querying, validation, and transformation, often accessed through higher-level tools, APIs, or user interfaces.

9.2 OWL: Schema-Level Semantics and Reasoning

9.2.1 Purpose and scope

The OWL (Web Ontology Language) serialisation of IMF models serves a fundamentally different purpose than the RDF instance serialisation. Where the RDF serialisation captures concrete model instances (e.g., specific blocks, terminals, and their connections), the OWL serialisation captures the schema-level semantics: the conceptual structure, constraints, and formal axioms that enable automated reasoning. To illustrate: the RDF representation presents specifications as *individuals* (“Pump P-101 has a flow rate of minimum 150 L/min”), while the OWL representation presents them as *classes* (“Pumps that are like P-101 have a flow rate property of at least 150 L/min”). The individual tells you *what is*, while the class introduces a named concept and what rules must hold in order to be classified as this concept. The OWL serialisation is not intended as a primary exchange format for IMF models, but rather serves as a derived format optimised for quality assessment through formal reasoning and semantic integration with external ontologies.

Transforming IMF models into OWL ontologies enables:

- Formal reasoning over model structure to detect inconsistencies and derive implicit facts
- Alignment of IMF content with external domain ontologies
- Schema-level reuse where model structures become reusable conceptual patterns
- Classification and subsumption checking to verify model correctness

The OWL serialisation is particularly valuable when IMF models need to be integrated into larger semantic frameworks, when formal verification is required, or when models serve as conceptual templates rather than just instance data.

9.2.2 What OWL adds beyond RDF(S)

RDF and RDF Schema (RDFS) provides a flexible graph-based data model with lightweight semantic capabilities through `rdf:type` and `rdfs:subClassOf`. However, RDF(S) alone does not provide the rich, formally-defined axioms needed for automated reasoning about constraints and implied relationships.

OWL builds on RDF(S) by adding description-logic semantics—a mathematically rigorous formalism for representing and reasoning about domain knowledge. Specifically, OWL adds:

Formal reasoning capabilities: OWL reasoners can automatically classify concepts, check for consistency, detect unsatisfiable classes, and infer implicit relationships based on stated axioms. For example, if a model states that a certain block must have at least one output terminal, a reasoner can verify this constraint and flag violations.

Expressive constraint modeling: OWL provides constructs for expressing complex constraints: existential restrictions (“must be connected to *some* X”), universal restrictions (“*all* connections must be of type Y”), cardinality restrictions (“must have *exactly two* inputs”), and value restrictions (“connected to this specific component”).

Schema-level abstraction: While RDF individuals represent concrete instances, OWL classes represent concepts and categories. Converting IMF instances to OWL classes elevates the model to a conceptual schema that can be reused, extended, and composed with other ontologies.

Formal semantics with standardised interpretation: OWL’s semantics are precisely defined, ensuring that different reasoners interpret axioms consistently. This enables reliable, reproducible reasoning across tools and platforms.

9.2.3 Key characteristics

The OWL representation of IMF models is *class-centric* (*TBox-oriented* (terminological box)), focusing on schema-level knowledge rather than instance-level facts. Where the RDF representation consists of individuals and their properties, the OWL representation consists primarily of classes and axioms.

Importantly, OWL is primarily *descriptive* and *inferential* rather than normative: OWL axioms describe what is necessarily true about instances of a class, allowing reasoners to infer additional facts and check logical consistency. Under OWL’s open-world assumption, the absence of information is not treated as an error—it simply means the information is unknown. OWL is therefore not a validation language; it does not prescribe what data *must* be present, but rather describes what logically follows from what is stated. This contrasts with SHACL, which is normative and validation-oriented, following the closed-world assumption (see [Section 9.3](#)).

Key structural characteristics are:

Class-based modeling: Each IMF element that represents a reusable concept becomes an `owl:Class`. These classes represent categories of elements rather than specific instances.

Axiom-rich constraints: Relationships and constraints are expressed through OWL axioms—formal statements about class membership, property restrictions, and logical relationships. These axioms enable reasoners to derive new knowledge and detect inconsistencies.

Hierarchical organisation through subsumption: Generalisation and specialisation relationships are captured through subsumption axioms, creating taxonomies where more specific concepts inherit properties and constraints from more general ones.

Property restrictions as structural requirements: Rather than stating “Block B1 has Terminal T1,” the OWL representation states “Blocks of type X must have at least one Terminal of type Y.” This shifts from describing specific instances to prescribing structural requirements.

9.2.4 Benefits and use cases

The OWL serialisation provides distinct advantages for tasks requiring formal reasoning and conceptual modelling:

Automated consistency checking: OWL reasoners can verify that model structures are consistent with respect to the formal semantics of the IMF ontology and any other connected ontologies. If a model element is ill-defined, the reasoner can detect this automatically.

Inference of implicit knowledge: Reasoners can derive facts that are logically implied but not explicitly stated. For example, if “all Pumps are Actuators” and “all Actuators require power,” then a reasoner can infer that all Pumps require power.

Semantic interoperability through alignment: OWL ontologies can be aligned with external vocabularies using mapping axioms (`owl:equivalentClass`, `owl:equivalentProperty`). This enables IMF models to interoperate with domain-specific standards and reference ontologies.

Conceptual reuse and extension: By capturing model patterns as OWL classes, these patterns become reusable templates. New models can import and extend existing OWL ontologies, inheriting their structure and constraints.

Classification and taxonomy management: OWL reasoners can automatically place concepts within taxonomies based on their properties, maintaining correct conceptual hierarchies even as models evolve.

Primary use cases of the OWL serialisation include:

- Formal verification of model correctness against semantic constraints
- Integration with semantic web knowledge graphs and linked open data
- Creating reusable conceptual libraries for domain-specific modelling
- Aligning proprietary models with industry-standard ontologies
- Supporting model evolution through incremental ontology refinement

9.2.5 Translation patterns overview

The transformation from the RDF serialisation to the OWL serialisation follows a systematic set of patterns. Each pattern addresses a specific IMF construct and produces corresponding OWL axioms. The complete RDF to OWL translation follows these patterns:

Element to Class Each IMF element becomes an `owl:Class`

Type Declaration Elements are declared as subclasses of their IMF type classes (e.g., `imf:Block`, `imf:Terminal`, `imf:Connector`)

Specialisation `imf:specializes` becomes `rdfs:subClassOf`

Element Relations Relationships like `imf:partOf`, `imf:hasTerminal` become existential or cardinality restrictions

Fixed References Reference properties with fixed targets become `owl:hasValue` restrictions

Attribute Constraints Min/max value constraints on attributes become datatype restrictions with facets

The following subsections provide detailed explanations and examples for each pattern, illustrating how each IMF construct maps to OWL and why that mapping preserves semantic intent.

9.2.6 Mapping details: from RDF instances to OWL classes

9.2.6.1 Elements to classes and subclass hierarchies

Semantic rationale In the RDF representation, each element (a specific block, terminal, or connector) is an individual—a concrete thing in the model. In the OWL representation, we want to capture not just that specific element, but the concept it exemplifies: what type of element it is, what properties all elements of that type share, and what constraints they must satisfy.

Mapping pattern

Each IMF element individual becomes an `owl:Class` and a subclass of its IMF element metaclass (e.g., `imf:Block`, `imf:Terminal`).

Example

```
# RDF serialisation:
id:n10 a imf:Block ;
      skos:prefLabel "Temperature Transmitter Package" .

# OWL serialisation:
id:n10 rdf:type owl:Class ;
      rdfs:subClassOf imf:Block ;
      rdfs:label "Temperature Transmitter Package" .
```

Why this works The OWL class `id:n10` represents “things that are like this Temperature Transmitter Package block”—inheriting all restrictions defined for blocks. This enables reasoning: if we later define that all blocks must have certain properties, the reasoner can verify that instances of this class satisfy those requirements.

9.2.6.2 Specialisation to subclass axioms

Semantic rationale Specialisation in IMF expresses that one element is a more specific variant of another, inheriting its structure while adding refinements. In OWL, this directly corresponds to the subsumption relationship: more specific classes inherit axioms from more general classes.

Mapping IMF specialisation relationships (expressed via properties like `imf:specialises`) become `rdfs:subClassOf` axioms:

```
# RDF:
:HighPressurePump imf:specialises :Pump .

# OWL:
:HighPressurePump rdfs:subClassOf :Pump .
```

Why this works OWL's subsumption semantics ensure that any constraint stated about `:Pump` automatically applies to `:HighPressurePump`. If pumps must have power connections, high-pressure pumps inherit this requirement. The reasoner enforces this inheritance automatically.

9.2.6.3 Relations to restrictions and axioms

Semantic rationale In the RDF instance graph, relations connect specific elements: "Block B1 is connected to Block B2 via Connector C1." In the OWL schema, we want to express structural requirements: "Blocks of type X must be connected to at least one Block of type Y."

Mapping Element relations become OWL restrictions—axioms that constrain what relationships instances of a class must have. The most common pattern uses existential restrictions:

```
# RDF serialisation:
id:n10 imf:hasAttribute id:n4 .

# OWL serialisation:
id:n10 rdfs:subClassOf [
  rdf:type owl:Restriction ;
  owl:onProperty imf:hasAttribute ;
  owl:someValuesFrom id:n4
] .
```

This states: "Instances of this block class have at least one attribute of type `id:n4`."

Why this works OWL restrictions express structural necessity. An OWL reasoner will infer that an instance stated to be a (block of type) `id:n10` actually satisfies this requirement of having an attribute of type `id:n4`, even if such an attribute is not explicitly given. Hence, the instance "inherits" the structural schema statements from the ontology.

9.2.6.4 References to value restrictions

Semantic rationale References in IMF point to specific external entities or fixed concepts (e.g., a specific unit of measure, a standard code). In OWL, when these references are fixed—meaning all instances must refer to the same target—we use value restrictions to enforce this.

Mapping Fixed references become `owl:hasValue` restrictions:

```
# RDF serialisation:
id:n4
  imf:predicate rdl:temperature ;
  imf:uom rdl:celcius .

# OWL serialisation:
id:n4
  rdfs:subClassOf [
    rdf:type owl:Restriction ;
    owl:onProperty imf:predicate ;
    owl:hasValue rdl:temperature ] ;
  rdfs:subClassOf [
    rdf:type owl:Restriction ;
    owl:onProperty imf:uom ;
    owl:hasValue rdl:celcius ] .
```

This states: "Instances of this attribute class reference the predicate `rdl:temperature` and the unit of measure `rdl:celcius`."

Why this works Value restrictions enable reasoners to verify that instances use the correct references. If an attribute instance references a different predicate or unit, it violates the class definition and can be flagged as inconsistent.

9.2.6.5 Attributes to datatype restrictions

Semantic rationale Attributes in IMF carry scalar values with associated datatypes and constraints (min/max ranges, enumerations). In OWL, we want to express these as schema-level constraints: “Attribute X must have a value between Y and Z.”

Mapping Attribute constraints become OWL datatype restrictions.

Example

```
# RDF serialisation:
id:n4 a imf:Attribute ;
    imf:hasAttributeQualifier imf:minimumQualifier ;
    imf:value 2 .

# OWL serialisation:
id:n4 rdfs:subClassOf [
    rdf:type owl:Restriction;
    owl:onProperty imf:value
    owl:allValuesFrom [
        rdf:type rdfs:Datatype;
        owl:onDatatype xsd:integer;
        owl:withRestrictions ( [ xsd:minInclusive 2 ] )
    ] ] .
```

This prescribes: “Instances of this attribute have a value that is not smaller than 2.”

Why this works Reasoners can verify that attribute values match required values. This catches data-quality errors automatically: if someone assigns a conflicting value, the reasoner detects the violation.

9.3 SHACL: Validation and Quality Assurance

9.3.1 Purpose and scope

The SHACL (Shapes Constraint Language) serialisation of IMF models addresses a fundamentally different need than either the RDF or OWL serialisations: validation of concrete data against explicit conformance rules. Where the RDF representation captures what is, and the OWL representation enables reasoning about what follows logically—what *is* true, the SHACL representation specifies what is required structurally—what *must* be explicitly stated, and enables automated checking of those requirements.

In contrast to OWL’s descriptive and inferential nature, SHACL is primarily *normative* and *constraint-based*: it prescribes what data must look like and validates whether actual data conforms to those prescriptions. SHACL operates under a closed-world assumption, treating missing information as a potential violation rather than unknown truth.

In the SHACL standard, a validation rule is called a *shape*. SHACL shapes can be applied to IMF RDF graphs to verify:

- Required properties are present

- Cardinalities are satisfied (min/max counts)
- Value ranges and datatypes are correct
- References point to valid targets
- Structural patterns are followed

SHACL is essential for quality assurance workflows, model governance, and data exchange contracts. It provides actionable validation reports that can pinpoint exactly what is wrong when a model fails to conform, enabling efficient debugging and correction.

9.3.2 What SHACL adds beyond RDF

RDF alone is permissive: any triple can be added to any graph—“*Anybody can say Anything about Any topic*”, and all graphs are equally valid RDF. RDF can express facts and types, but it provides no native mechanism to declare “this graph is incorrect because it lacks property X” or “this value violates constraint Y.”

SHACL introduces a standard, declarative language for constraint checking. Specifically, SHACL adds:

Explicit validation rules: SHACL shapes define precise requirements: which properties must be present, how many values they must have, what datatypes are allowed, and what patterns must be followed. These rules are first-class artifacts that can be version-controlled, published, and evolved independently of data.

Separation of data and constraints: The instance graph (RDF data) remains pure data, while the shapes graph encodes conformance requirements. This separation enables different validation profiles—strict vs. lenient, development vs. production—to be applied to the same data.

Actionable error reporting: SHACL validation engines produce detailed reports that identify exactly which nodes violate which constraints, with specific property paths and failure messages. This supports systematic data quality improvement.

Closed-world validation: While OWL reasoners operate under *open-world assumption* (absence of a fact does not entail that the fact is false), SHACL operates under *closed-world assumption*: if a required property is missing from the graph, it is assumed to not be true—and can hence raise an error. This matches practical validation needs.

9.3.3 Key characteristics

The key characteristics of the SHACL representation of IMF models are:

Shapes as validation templates: SHACL defines two main constructs: `sh:NodeShape` (constraints on nodes/resources) and `sh:PropertyShape` (constraints on specific properties). Each shape describes requirements that target nodes must satisfy.

Path-based constraint specification: SHACL uses property paths (`sh:path`) to identify which properties are being constrained, supporting simple properties, inverse properties, sequences, and alternatives.

Rich constraint vocabulary: SHACL provides a comprehensive set of constraint types: cardinality (`sh:minCount`, `sh:maxCount`), datatypes (`sh:datatype`), value ranges (`sh:minInclusive`, `sh:maxInclusive`), node kinds (`sh:IRI`, `sh:Literal`), patterns (`sh:pattern`), closed shapes (`sh:closed`), and many others.

Target selectors: Shapes specify which nodes they apply to using target declarations (`sh:targetClass`, `sh:targetNode`, `sh:targetSubjectsOf`), enabling precise, selective validation.

9.3.4 Benefits and use cases

SHACL provides distinct advantages for data quality and governance:

Automated conformance testing: SHACL engines can validate entire model repositories, flagging non-conformant models automatically. This enables continuous integration workflows where models must pass validation before publication.

Explicit quality contracts: SHACL shapes serve as formal contracts: “Models in this repository must satisfy these requirements.” This makes implicit assumptions explicit and enforceable.

Detailed diagnostics: When validation fails, SHACL reports identify the exact nodes, properties, and constraint violations involved. This dramatically reduces debugging time compared to manual inspection or informal checking.

Flexible validation profiles: Different shapes graphs can enforce different validation profiles. Development environments might use lenient shapes, while production environments enforce strict shapes. The same instance data can be checked against multiple profiles.

Interoperability validation: For data exchange, SHACL shapes can encode partner-specific requirements, ensuring that exported models satisfy recipient expectations before transmission.

Regression prevention: Once a model passes validation, the shapes graph serves as a regression test. Future changes that violate constraints are caught immediately.

Primary use cases of the SHACL serialisation include:

- Quality gates in model development and publication workflows
- Contract validation for data exchange between organisations
- Automated testing in continuous integration pipelines
- Debugging and troubleshooting non-conformant models
- Governance and compliance checking against organisational standards

9.3.5 Translation patterns overview

The transformation from the RDF serialisation to the SHACL serialisation follows a systematic set of patterns. The RDF to SHACL translation uses the following patterns:

Element to NodeShape: Each element type becomes a `sh:NodeShape` with `sh:targetClass`

Element Relations to PropertyShapes: Relations become property shapes with `sh:node` constraints

Fixed Values: `sh:hasValue` for references that must point to specific targets

Range Constraints: `sh:minInclusive/sh:maxInclusive` for numeric ranges

The following subsections provide detailed explanations and examples for each pattern.

9.3.6 Mapping details: from RDF instances to SHACL shapes

Transforming from the RDF serialisation to the SHACL serialisation involves extracting the implicit structural requirements from instance data and making them explicit as validation rules. This process identifies patterns in the instance graph and generalises them into constraints that future instances must satisfy.

9.3.6.1 Elements to node shapes

Semantic rationale Each type of IMF element (blocks, terminals, connectors, attributes) has structural expectations: certain properties must be present, certain relationships must exist. A node shape captures these expectations as a reusable validation template.

Mapping Each IMF element individual with its type declaration becomes a `sh:NodeShape` targeting the element via `sh:targetNode`. The shape's IRI is derived from the element IRI with a "-Shape" suffix, and property constraints are derived from the element's structure.

Example

```
# RDF serialisation:
id:n10 a imf:Block ;
      skos:prefLabel "Temperature Transmitter Package" .

# SHACL serialisation:
id:n10-Shape a sh:NodeShape ;
  sh:targetNode id:n10 ;
  sh:property [ ... ] . # property constraints ...
```

This shape targets the specific block `id:n10`. Any instance data for this block that doesn't satisfy the shape's properties is flagged as non-conformant.

Why this works Node shapes provide a centralised place to declare all requirements for a given element type. As requirements evolve, shapes can be updated, and existing instances automatically re-validated against the new rules.

9.3.6.2 Fixed-value references

Semantic rationale Some references must point to specific, fixed targets (e.g., a sensor must reference a particular unit of measure). SHACL can enforce that the referenced value is exactly what's required.

Mapping Reference properties (subproperties of `imf:reference`) or fixed `imf:value` properties without min/max qualifiers become property shapes with `sh:hasValue` constraints specifying the exact required value. For each fixed reference, the property shape enforces the exact value must be present, with `sh:minCount` and `sh:maxCount` set to 1.

Example

```
# RDF serialisation:
id:n10 a imf:Block ;
      imf:hasAttribute id:n4 .
id:n4 a imf:Attribute ;
      skos:prefLabel "[minimum, design] | [temperature] ; [Celcius] = [2]" .
      imf:uom rdl:celcius ;

# SHACL serialisation:
id:n10-Shape rdf:type sh:NodeShape ;
  sh:targetNode id:n10 ;
  sh:property [
    sh:path imf:hasAttribute ;
    sh:node <http://example.com/id/n4-Shape> ] .
id:n4-Shape rdf:type sh:NodeShape;
```

```

sh:name "[minimum, design] | [temperature] ; [Celcius] = [2]";
sh:property [
  sh:path imf:uom ;
  sh:hasValue <http://example.com/rdl/celcius> ] .

```

This states that: “Instances of block type id:n10 (Temperature Transmitter Package) must have an attribute of shape id:n4-Shape, which reference exactly the unit of measure celcius”.

Why this works sh:hasValue provides direct value matching. If a the attribute instance references psi instead of celcius, validation fails with a message indicating the mismatch.

9.3.6.3 Attribute constraints

Semantic rationale Attributes have datatype requirements (integers, doubles, strings) and often range constraints (min/max values). These need to be validated to prevent data quality errors.

Mapping Attribute elements with imf:value properties and optional imf:hasAttributeQualifier for min/max constraints become property shapes with sh:minInclusive, and sh:maxInclusive constraints. For each attribute with min/max qualifiers, the range bounds are extracted and corresponding SHACL range constraints are created.

Example

```

# RDF serialisation:
id:n4 a imf:Attribute ;
  imf:hasAttributeQualifier imf:minimumQualifier ;
  imf:value 2 .

# SHACL serialisation:
id:n4-Shape rdf:type sh:NodeShape;
  sh:targetClass id:n4 ;
  sh:property [
    sh:path imf:value ;
    sh:minInclusive 2 ] .

```

This enforces: “The attribute must have a value not less than 2.”

Why this works SHACL’s constraints provide precise value checking. Incorrect values are immediately flagged, preventing invalid data from propagating through workflows.

9.4 AAS: Industrial Exchange and Digital Twin Integration

9.4.1 Purpose and scope

The Asset Administration Shell (AAS) serialisation of IMF models serves the needs of industrial data exchange and digital twin ecosystems. Where the RDF, OWL, and SHACL serialisations focus on semantic richness, reasoning, and validation, the AAS serialisation focuses on standardised packaging, hierarchical organisation, and practical interoperability within industrial toolchains. The AAS serialisation should be understood as a projection of selected IMF information into the AAS framework for interoperability purposes. It does not replace the IMF model or its RDF serialisation, but rather enables integration with AAS-based tools and ecosystems by packaging IMF content according to AAS structural conventions.

The AAS serialisation packages IMF models into AASX files—standardised containers that include asset metadata, submodels, and references to external resources. This format is designed for exchange between industrial partners, integration with manufacturing execution systems, and deployment in digital twin infrastructures.

The primary goals of the AAS serialisation are:

- Complete, self-contained data exchange following the AAS metamodel
- Hierarchical organisation that aligns with asset-centric thinking in industry
- Integration with AAS registries, repositories, and runtime environments
- Reference consistency and semantic traceability back to IMF IRIs

9.4.2 What AAS adds beyond RDF

The RDF serialisation provides a general-purpose, flexible graph representation ideal for semantic integration and knowledge representation. However, industrial ecosystems have specific needs that raw RDF graphs don't directly address:

Standardised packaging and metadata: AAS defines a container format (AASX) that bundles asset information, submodels, file attachments (CAD drawings, manuals), and metadata (versioning, lifecycle state, provenance) into a single, self-contained package. This packaging is essential for practical exchange where recipients need not just data, but context.

Hierarchical, asset-centric organisation: Industrial systems typically organise information around assets (equipment, machines, products) and their submodels (technical data, documentation, operational data). AAS provides a standardised hierarchical structure—AAS → Submodels → SubmodelElements—that aligns with this mental model, making data easier to consume by industrial tools that expect such organisation.

Toolchain integration: AAS is supported by a growing ecosystem of industrial tools: AAS editors (AASX Package Explorer), registries (for asset discovery), submodel repositories (for data sharing), and digital twin frameworks. The AAS serialisation may enable integration with these tools without requiring custom adapters, depending on tool compliance with the AAS standard.

Lifecycle and interface standardisation: AAS defines standard interfaces (AAS API) for runtime access to digital twins, supporting industrial use cases like condition monitoring, predictive maintenance, and production optimisation. The format supports lifecycle management (versioning, approval workflows) native to industrial contexts.

9.4.3 Key characteristics

The AAS representation of IMF models is *asset-centric and hierarchical*, organising information around the concept of administered assets and their structured submodels. Key characteristics are:

Hierarchical structure: Unlike RDF's flat graph, AAS imposes a tree-like hierarchy: each AAS represents one asset and contains multiple Submodels; each Submodel contains SubmodelElements (properties, collections, references, etc.).

Semantic traceability: While AAS enforces a hierarchical structure, it preserves semantic links to original IMF concepts through `semanticId` attributes. Every AAS element can reference an IMF IRI, maintaining traceability even after projection from graph to tree.

Standardised identifiers: AAS uses standardised identification schemes (IRIs, ID-Short patterns) ensuring that assets and submodels can be uniquely referenced across organisational boundaries.

Rich metadata support: AAS elements can carry extensive metadata: descriptions, administrative information (version, revision, creator), data specifications, and lifecycle state.

9.4.4 Benefits and use cases

The AAS serialisation provides distinct advantages for industrial deployment and exchange:

Industry-standard packaging: AASX files are intended to be recognised across industrial tool ecosystems. Organisations can exchange IMF models as AAS packages to allow recipients to open, parse, and consume them using compliant tools.

Consumption-friendly structure: While graph representations are powerful for querying and semantic reasoning, many industrial applications simpler formats. AAS provides a simpler tree-structure natively, possibly reducing integration effort.

Digital twin integration: AAS is the de facto standard for Industry 4.0 digital twins. By representing IMF models in AAS, they become deployable in digital twin platforms that support the AAS standard, enabling runtime monitoring, control, and analytics.

Registry and discovery: AAS registries can enable asset discovery across distributed systems. IMF models published as AAS may be registered, searched, and retrieved using standard registry protocols where such infrastructure is available.

Lifecycle management: AAS supports versioning, approval workflows, and lifecycle state management. IMF models may be managed through industrial change-control processes using AAS mechanisms.

Cross-organisational exchange: AAS is designed for B2B and supply-chain scenarios. Standardised packaging, identification, and interfaces aim to ensure that models exchanged between organisations remain interpretable and usable by compliant systems.

Primary use cases for the AAS serialisation include:

- Exchanging information models with industrial partners
- Deploying models in digital twin platforms and monitoring systems
- Publishing models to AAS registries for discovery and reuse
- Integrating models with manufacturing execution and enterprise systems
- Supporting product lifecycle management (PLM) workflows

9.4.5 Translation patterns overview

The transformation from the RDF serialisation to the AAS serialisation follows a systematic set of patterns. The complete RDF to AAS translation follows these patterns:

Block to AAS: Each `imf:Block` becomes an AAS with `global_asset_id` = Block IRI

Technical Data Submodel: Contains:

- Attributes → Properties
- Terminals → SubmodelElementCollections
- Dependencies → Nested collections and RelationshipElements

Capabilities Submodel: Created when block has `imf:fulfills` relationships

Semantic Traceability: All AAS elements include `semantic_id` pointing to original IMF IRIs

Datatype Mapping: XSD datatypes mapped to AAS datatypes

Units: Stored as Qualifiers with `value_id` references

Classifiers: Mapped to `supplemental_semantic_id`

The following subsections provide detailed explanations and examples for each pattern.

9.4.6 Mapping details: from RDF graphs to AAS hierarchies

Transforming from the RDF serialisation to the AAS serialisation requires projecting a flexible graph structure onto a prescribed hierarchical structure. This projection must preserve semantic content while adapting to AAS's organisational constraints.

9.4.6.1 IMF Blocks to AAS shells and submodels

Semantic rationale In AAS, each significant asset is represented by an AAS shell with associated submodels for different aspects of information (technical data, capabilities, documentation).

Mapping Each `imf:Block` individual with its label, attributes, terminals, and relationships becomes an AAS shell with `globalAssetId` set to the block IRI. The block's label becomes the `idShort`. A `TechnicalData` submodel is created containing the block's attributes, terminals, and dependency relationships. If the block has `imf:fulfills` relationships to function-aspect elements, a `Capabilities` submodel is created listing those capabilities.

Example

```
# RDF serialisation:
id:n10 a imf:Block ;
  imf:fulfills id:n9 ;
  imf:hasAttribute id:n4, id:n5 ;
  skos:prefLabel "Temperature Transmitter Package" .

# AAS serialisation (conceptual structure):
AAS:
  id: "http://example.org/id/n10/aas"
  idShort: "TemperatureTransmitterPackage"
  globalAssetId: "http://example.org/id/n10"
  submodels:
    - TechnicalData (containing attributes id:n4, id:n5)
    - Capabilities (if id:n9 has functionAspect)
```

9.4.6.2 IMF Attributes to AAS Properties

Mapping Each `imf:Attribute` individual with `imf:value`, optional `imf:uom` (unit of measure), optional `imf:classifier`, and a label becomes an AAS Property element within the `TechnicalData` submodel. The attribute's label is used to create the `idShort`, and the `imf:value` literal is mapped to the Property's value with appropriate XSD-to-AAS datatype conversion. The `semanticId` references the original attribute IRI. Classifiers are added as `supplementalSemanticId` entries, and units of measure become Qualifiers with type "Unit".

Example

```
# RDF serialisation:
id:n4 a imf:Attribute ;
    imf:hasAttributeQualifier imf:designQualifier, imf:minimumQualifier ;
    imf:predicate rdl:temperature ;
    imf:uom rdl:celcius ;
    imf:value 2 ;
    skos:prefLabel "[minimum, design] | [temperature] ; [Celcius] = [2]" .

# AAS serialisation (conceptual structure):
Property:
  idShort: "minimumDesignTemperatureCelcius2"
  value: 2
  valueType: xs:integer
  semanticId: "http://example.org/id/n4"
  qualifiers:
    - type: "Unit"
      value: "celcius"
      valueId: "http://example.org/rdl/celcius"
    - type: "Predicate"
      valueId: "http://example.org/rdl/temperature"
```

9.4.6.3 IMF Terminals to AAS SubmodelElementCollections

Mapping `imf:Terminal` individuals linked to a block via `imf:hasTerminal` become a “Terminals” SubmodelElementCollection within a specific SubmodelElementCollection “IMF” in the TechnicalData submodel. Each terminal becomes a nested SubmodelElementCollection named after the terminal’s label, containing Property elements for each of the terminal’s attributes.

Example

```
# RDF serialisation:
id:n1 a imf:Block ;
    skos:prefLabel "644SAI1J7+1001V" ;
    imf:hasTerminal id:n0 .
id:n0 a imf:Terminal ;
    skos:prefLabel "CabledConnection" .

# AAS serialisation (conceptual structure):
IMF:
  Terminals:
    SubmodelElementCollection:
      idShort: "CabledConnection"
      semanticId: "http://example.org/id/n0"
```

Terminals are organised as SubmodelElementCollections within the TechnicalData submodel, each containing the terminal’s attributes.

9.4.6.4 IMF Dependencies to AAS RelationshipElements

Mapping Dependency relationships such as `imf:fulfils`, `imf:partOf`, `imf:containedIn`, `imf:specialisationOf`, `imf:proxy`, and their inverse properties become a “Dependencies” SubmodelElementCollection in the SubmodelElementCollection “IMF” containing nested SubmodelElementCollections for related elements and RelationshipElements capturing the relationships. For each dependency relationship (both forward and inverse), a SubmodelElementCollection represents the related element and a RelationshipElement links the source and target with its `semanticId` referencing the IMF predicate IRI.

Example

```
# RDF serialisation:
id:n1 a imf:Block ;
  skos:prefLabel "644SAI1J7+1001V" ;
  imf:hasTerminal id:n0 .
id:n0 a imf:Terminal ;
  skos:prefLabel "CabledConnection" .

# AAS serialisation (conceptual structure):
IMF:
  Dependencies:
    SubmodelElementCollection:
      idShort: "CabledConnection"
      semanticId: "http://example.org/id/n0"
```

9.4.6.5 Capabilities Submodel from fulfills relationships

Mapping `imf:fulfills` relationships (direct or transitive) to elements with `imf:hasAspect imf:functionAspect` become a Capabilities submodel containing AAS Capability elements. The graph is queried for all function-aspect elements that the block fulfills (following the transitive closure of `imf:fulfills`), and for each such element, an AAS Capability is created with `idShort` from the element's label and `semanticId` pointing to the capability IRI.

Example

```
# RDF serialisation:
id:n10 a imf:Block ;
  imf:fulfills id:n9 ;
id:n9 a imf:Block ;
  skos:prefLabel "Temperature Sensing" .

# AAS serialisation (conceptual structure):
Capabilities Submodel:
  Capability:
    idShort: "TemperatureSensing"
    semanticId: "http://example.org/id/n4"
```

The Capabilities submodel queries the graph for all function aspect elements that this block fulfills (directly or transitively), creating AAS Capability elements for each.

Chapter 10

IMF Workbench

Abstract

This chapter describes the IMF Workbench, a modular toolchain for processing Information Modelling Framework (IMF) models. The workbench automates key ontology engineering tasks including transformation of IMF models to RDF, generation of OWL type ontologies and SHACL shape definitions, validation against IMF grammars, OWL reasoning for consistency checking and classification, and visualization of models. The toolchain integrates various specialized software packages and purpose-built transformation modules maintained as separate Git repositories. In addition, the workbench supports generating Asset Administration Shell (AAS) representations and Industrial Data Ontology (IDO)-compatible OWL ontologies from IMF models. The chapter covers installation, the available transformation and validation capabilities, and the mappings used for external format export.

Intended readers This chapter is written for practitioners who need to operationalise IMF models, including tool developers, data engineers, and advanced users responsible for model validation, transformation, and integration. It assumes familiarity with command-line tools, basic knowledge of RDF and semantic web technologies, and understanding of the IMF serialisation formats ([Chapter 9](#)). Readers focused solely on conceptual modelling may treat this chapter as an optional reference. Those working through the use case ([Chapter 11](#)) will find this chapter useful for understanding how the example models were generated and validated.

10.1 Introduction

The IMF Workbench is a modular toolchain that automates the transformation, validation, and visualization of IMF models. It bridges the gap between model authoring and the semantic web technologies that underpin the framework of IMF, enabling practitioners to generate machine-readable ontologies, validate models against IMF grammars, and produce documentation without manual intervention.

The workbench supports IMF models expressed in both GraphML and RDF formats. From these inputs, it can generate RDF graphs, OWL ontologies, SHACL shape definitions, HTML documentation, validation reports, graph visualizations, and generate Asset Administration Shell (AAS) representations.

The IMF workbench is currently available for execution only via a command line interface. However, most of the loosely coupled components that make out the workbench are implemented using open standards-based declarative specifications that can be used independently of the workbench.

The IMF workbench is published in an open Git repository: <https://gitlab.com/imf-lab/tool/imf-workbench>.

The toolchain orchestrates several specialized software packages:

- **Apache Jena**¹ – Provides RDF processing capabilities through RIOT (for serialization and syntax validation) and SHACL (for constraint validation)
- **Saxon HE**² – Executes XSLT transformations for converting GraphML to RDF
- **HermiT**³ – Performs OWL reasoning, including consistency checking and class hierarchy classification

In addition, the workbench incorporates several submodule components that are maintained in separate Git repositories:

- **imf-graphml** – Handles GraphML to RDF transformation using XSLT stylesheets
- **imf-ottr-transforms** – Generates OWL and SHACL type definitions using OTTR⁴ (Reasonable Ontology Templates)
- **imf-rdfvizler** – Produces visual graph representations of RDF models
- **imf-aas** – Translates IMF RDF models to Asset Administration Shell format
- **imf-ido** – Translates IMF RDF models to Industrial Data Ontology (IDO) compatible OWL ontologies

This modular architecture allows each component to evolve independently while maintaining integration through the main workbench build system.

10.2 Installation

The IMF Workbench requires GNU Make for build orchestration, Git for repository and submodule management, and Java (JRE/JDK 8 or higher) for certain software packages. Some submodules may have additional requirements such as Python; consult the individual submodule documentation for details.

Installation proceeds in two steps. First, clone the repository with all submodules initialized. Second, run the installation target, which downloads Apache Jena, Saxon HE, and the HermiT reasoner into the local `software/` directory. After installation, the toolchain is ready to process model data.

The repository structure places input models and generated outputs in the `data/` directory, with each project occupying its own subdirectory. The `software/` directory contains the downloaded tools, while the submodule directories house the transformation and visualization components.

10.3 Data Transformation

The IMF Workbench provides several transformation capabilities that convert IMF models into semantic web formats suitable for reasoning, validation, and documentation.

10.3.1 GraphML to RDF Conversion

For IMF models authored in GraphML format, the workbench provides transformation to RDF. This transformation extracts IMF constructs—blocks, terminals, attributes, and relations—from the GraphML

¹<https://jena.apache.org/>

²<https://www.saxonica.com/>

³<http://www.hermit-reasoner.com/>

⁴<https://ottr.xyz/>

representation and maps them to corresponding RDF structures. The transformation is implemented as an XSLT stylesheet executed by Saxon, with the resulting RDF/XML subsequently serialized to Turtle format using Jena RIOT for improved readability.

The GraphML transformation preserves the semantic content of the IMF model while expressing it in a format amenable to further processing by semantic web tools. The output RDF graph represents the model instance and serves as input to subsequent validation and type generation processes.

10.3.2 OWL Serialisation Generation

The workbench can produce an OWL ontology representation from IMF models, following the specification in [Section 9.2](#).

The IMF-to-OWL transformation is implemented using OTTR (Reasonable Ontology Templates), a template-based approach for generating RDF/OWL knowledge graphs and ontologies. The transformation implementation consists of:

OTTR Templates: Reusable templates that encode OWL axiom patterns, such as how to create subclass axioms, existential restrictions, and datatype restrictions. Each template takes parameters (IRIs, literals) and produces OWL statements serialized as RDF.

OTTR Mappings: SPARQL-based queries that extract data from the IMF RDF graph and bind it to template parameters. For example, a mapping query selects all blocks and their attributes, then invokes the appropriate templates to generate OWL class definitions.

Template Expansion: The OTTR engine (Lutra) processes the mappings against the input graph, expanding templates to produce the complete OWL ontology.

This approach separates the transformation logic (templates) from the data extraction (mappings), enabling maintainable, reusable transformation pipelines. The resulting OWL ontology can be imported into ontology editors (Protégé), reasoned over using standard reasoners (HermiT, Pellet), and published as reusable conceptual schemas.

10.3.3 SHACL Serialisation Generation

The workbench can produce SHACL shape definitions from IMF models, following the specification in [Section 9.3](#). SHACL shapes serve a dual purpose: they enable validation of model instances against structural and cardinality constraints, and they provide a basis for generating human-readable documentation.

The IMF-to-SHACL transformation is implemented using OTTR (Reasonable Ontology Templates), as the OWL transformation. The implementation consists of:

OTTR Templates: Reusable templates that encode SHACL shape patterns, including node shapes, property shapes with various constraint types (cardinality, datatype, value ranges), and closed shapes.

OTTR Mappings: SPARQL-based queries that extract structural patterns from the IMF RDF graph and bind them to template parameters. For example, mappings query for all element relations to generate property shapes with appropriate cardinality constraints.

Template Expansion: The OTTR engine (Lutra) processes the mappings against the input graph, expanding templates to produce the complete SHACL shapes graph.

The generated SHACL shapes can be stored separately from instance data and applied using SHACL validation engines (pySHACL, Apache Jena SHACL, TopQuadrant SHACL). Validation can be integrated into CI/CD pipelines, model repositories, or data exchange gateways to ensure continuous conformance.

10.3.4 HTML Documentation Generation

The workbench can generate human-readable HTML documentation from SHACL shape definitions. This documentation presents the type structure in an accessible format, including diagrams that visualize the relationships between types.

The documentation generation leverages the SHACL Play⁵ service, which renders SHACL shapes as structured HTML pages. The resulting documentation provides a navigable reference for the types defined in a model, useful for communication among project stakeholders and as a reference during model development.

10.4 Validation and Reasoning

Validation and reasoning capabilities form a core function of the IMF Workbench, providing automated verification that models conform to IMF specifications and are logically consistent.

10.4.1 SHACL Validation

The workbench performs SHACL validation against multiple shape graphs to ensure comprehensive conformance checking. Validation operates at two levels: validation of the generated SHACL type models themselves, and validation of RDF model instances against those type models.

10.4.1.1 Type Model Validation

Generated SHACL type models are validated against several reference shape graphs:

- **Standard Vocabulary Elements** – Ensures that type definitions use vocabulary terms correctly and consistently according to established conventions
- **SHACL-SHACL** – The W3C meta-shapes that verify the generated SHACL is syntactically valid and well-formed according to the SHACL specification
- **IMF Types Grammar** – Shapes specific to the IMF that verify type definitions conform to IMF structural requirements and naming conventions

This multi-layer validation catches errors ranging from basic syntax issues to IMF-specific semantic violations, providing detailed diagnostic information when problems are detected.

10.4.1.2 Model Instance Validation

RDF model instances undergo validation against both general IMF grammars and project-specific type constraints:

- **Standard Vocabulary Elements** – Verifies correct use of vocabulary terms in the model instance
- **IMF Terms Grammar** – Validates that IMF terms (blocks, terminals, attributes) are correctly formed and properly related
- **IMF Model Grammar** – Checks overall model structure, including aspect organisation and inter-aspect relations
- **Generated Type Shapes** – Validates the instance against the specific types defined for the project, ensuring domain-specific constraints are satisfied

⁵<https://shacl-play.sparna.fr/>

Validation results are written to log files that document each validation pass, listing any constraint violations with details about the offending nodes and the specific shapes they fail to satisfy.

10.4.2 OWL Reasoning

For deeper semantic analysis, the workbench employs the HermiT OWL reasoner to perform logical reasoning over the combined RDF model and OWL type ontology.

10.4.2.1 Consistency Checking

Consistency checking determines whether the combined model and ontology is logically satisfiable—that is, whether there exists any interpretation under which all axioms can be simultaneously true. An inconsistent ontology indicates a logical contradiction, often arising from conflicting type assignments or impossible constraint combinations.

When HermiT detects inconsistency, the diagnostic output helps identify the source of the contradiction, though root cause analysis may require examination of the interacting axioms.

10.4.2.2 Classification

Classification computes the inferred class hierarchy by determining all subsumption relationships between classes. This process reveals implicit type relationships that may not be explicitly stated in the model but follow from the OWL axioms.

The classification results augment the explicit type structure with inferred relationships, providing a complete picture of the type hierarchy. This is particularly useful for understanding how types relate to each other and for identifying potential modeling issues such as unintended equivalences or subsumptions.

10.4.3 Syntax Validation

Throughout the transformation pipeline, Jena RIOT performs RDF syntax validation, ensuring that all generated RDF is well-formed and parseable. This catches low-level serialization errors that might otherwise propagate through the pipeline and cause confusing failures in downstream tools.

10.5 Visualization

The IMF Workbench can generate visual representations of RDF models using the `imf-rdfvizler` component. These visualizations render the model graph as SVG (Scalable Vector Graphics) images, depicting blocks, terminals, and their interconnections.

Visual graph representations provide an alternative view of model structure that complements textual RDF serializations. They are particularly useful for understanding the topology of a model, identifying patterns and anomalies, and communicating model structure to stakeholders who may not be familiar with RDF syntax.

The visualization component applies layout algorithms to arrange graph elements for readability and uses visual conventions (shapes, colors, edge styles) to distinguish different types of model elements. The SVG output format allows the visualizations to be embedded in documentation, viewed in web browsers, or further processed with vector graphics tools.

10.6 External Format Export

The IMF Workbench supports export of IMF models to external formats, enabling integration with other systems and standards.

10.6.1 Asset Administration Shell Export

The `imf-aas` module translates IMF RDF models to the Asset Administration Shell (AAS) format. AAS is an industry standard for describing assets in the context of Industry 4.0, providing a standardised way to represent asset information for interoperability across industrial systems. For detailed information on the IMF to AAS mapping, see [Section 9.4](#).

The IMF-to-AAS transformation is implemented in Python using `RDFLib` for graph processing and the `BaSyx` AAS SDK for generating AAS structures. The implementation consists of:

RDFLib: Used to load and query the IMF RDF graph. SPARQL queries extract blocks, attributes, terminals, and relationships from the graph.

BaSyx Python SDK: Used to programmatically create AAS shells, submodels, and submodel elements. The SDK handles the AAS metamodel compliance and provides serialization to AASX format.

pySHACL: Optionally used for pre-validation of the IMF graph against SHACL shapes before transformation, ensuring data quality.

The transformation pipeline follows these steps:

1. Load the IMF RDF graph from a Turtle file using `RDFLib`
2. Optionally validate the graph against IMF SHACL shapes using `pySHACL`
3. For each `imf:Block` in the graph, create an AAS shell with Technical Data and Capabilities submodels using `BaSyx`
4. Serialize all AAS shells and submodels into an AASX package with JSON serialization

10.6.2 Industrial Data Ontology Export

The `imf-ido` module translates IMF RDF models to OWL ontologies compatible with the Industrial Data Ontology⁶ (IDO). IDO is a reference ontology for industrial data based on ISO 15926, providing a formal semantic foundation for representing industrial information.

The translation is implemented through a series of SPARQL CONSTRUCT queries that map IMF vocabulary and model elements to their IDO equivalents. The mapping introduces a bridging vocabulary (`imf-ido:`) that defines classes and properties relating IMF concepts to IDO concepts.

⁶<http://rds.posccaesar.org/ontology/lis14/rdl/>

Part IV

IMF in Practice

Chapter 11

Use Case: Gas Cooling System

Abstract

This chapter presents a worked industrial use case demonstrating the application of IMF to a gas cooling system. Building on the conceptual and formal foundations developed in earlier chapters, it shows how IMF is used to construct, relate, and analyse engineering *appearances* across the *Capability*, *Implementation*, and *Space* information domains.

The use case illustrates how the explicit use of *aspects* allows distinct engineering perspectives and lifecycle stages to be modelled without loss of discipline-specific meaning. Core IMF constructs are applied in practice, including *Breakdown* elements for managing overlapping system decompositions and *Quality* elements for the structured representation of properties and requirements.

Finally, the chapter demonstrates how an intensional IMF model can be queried and interpreted to support cross-domain engineering analysis. Derived representations using semantic technologies such as RDF, OWL, SHACL, and SPARQL are used to answer a realistic operational question—tracing a functional failure to its physical location, thereby illustrating the practical value of IMF as a semantic core within a digital-engineering ecosystem.

Intended readers This chapter is written to accommodate two types of readers.

- Readers who are familiar with previous chapters will recognise the concepts of *appearances*, *aspects*, and *breakdowns* as introduced in earlier chapters. For them, this chapter serves as a concrete instantiation of those ideas in a realistic engineering setting.
- Readers who have jumped directly to this chapter, for example, practitioners seeking a quick introduction or a concrete example, can also read it independently and refer to the concepts elaborated in other chapters when necessary.

The use case is structured so that the essential ideas of IMF are reintroduced informally, through their application, before being used in queries and analyses. Where IMF terminology is used, its meaning can generally be inferred from context, even without detailed knowledge of the underlying concepts and formalism. This chapter should therefore be read as a *guided entry point* into IMF modelling practice. It shows what an IMF model looks like, what kinds of engineering questions it supports, and how it relates to familiar artefacts such as P&IDs, equipment lists, and spatial layouts.

11.1 Purpose and Scope of the Use Case

Previous chapters introduced the conceptual foundations, language constructs, and formal interpretation of the Information Modelling Framework (IMF). This chapter shifts from theory to practice by applying IMF to a realistic industrial scenario: a gas cooling system drawn from typical process-engineering contexts.

The purpose of this chapter is not to introduce new IMF constructs, nor to prescribe a universal modelling template. Instead, it serves as a *worked example* that demonstrates how the concepts developed earlier in the manual are instantiated, combined, and used to support concrete engineering questions. The chapter is intended to show how IMF is applied in practice, what kinds of modelling decisions arise, and how those decisions affect analysis, querying, and integration.

The use case has three primary objectives:

1. **Demonstrate IMF modelling practice.** The chapter shows how a single engineered system is modelled through multiple *appearances*, each scoped by an explicit *aspect*. Functional capability, physical implementation, and spatial extension are represented in parallel, modelled with different breakdown and topology using a unified language. Particular attention is paid to the use of Breakdown, connectivity relations, and cross-aspect correspondences.
2. **Illustrate the operational value of IMF.** Using a realistic engineering question, *identifying the source and impact of a gas leak*, the chapter demonstrates how IMF models support structured, cross-domain queries. The example mirrors how engineers reason in practice: moving between functional behaviour, physical assets, and location-dependent constraints.
3. **Position IMF within a digitalised engineering ecosystem.** The chapter shows how an IMF model can be serialised into different technical representations, including RDF graphs, OWL interpretations, and industrial exchange formats AAS. These representations are derived views of a single intensional IMF model, each serving specific purposes such as querying, interoperability, verification, or reasoning, rather than alternative or competing models.

11.1.1 What is IMF in this Use Case?

In this use case, IMF functions as a *logical digital twin*. While geometric digital twins (e.g. 3D CAD models) capture how a system looks, an IMF model captures how a system *works* and how its engineering information is *structured*. The model represents the system as a connected graph of system elements, qualities, and relations, rather than as a collection of documents or drawings.

Specifically, the IMF model demonstrated in this chapter formalises four classes of engineering questions that are typically distributed across heterogeneous artefacts:

- **Breakdown (composition):** What systems and subsystems exist, and how are they decomposed under a given context?
- **Topology (interaction):** How do system elements connect and interact, for example through the flow of media or signals?
- **Qualities (properties and requirements):** What attributes, constraints, or performance characteristics apply to system elements?
- **Aspects (context):** Under which modality, information domain, and engineering interest is a particular statement intended to hold?

In short, IMF is a *cross-discipline language for engineers* to create precise information models of industrial systems, not as 3D shapes, but as a collection of connected system elements, attributes, and relationships. By

making these structures explicit and context-dependent, IMF enables precise reasoning across disciplinary boundaries and lifecycle stages. Technically, the models are realised using open semantic standards such as RDF, OWL, and SHACL, ensuring that IMF models are interoperable, vendor-neutral, and amenable to automated validation and querying.

The following sections apply these principles step by step to the gas cooling system, showing how functional, implementation, and spatial appearances are integrated into a coherent IMF model and how that model supports cross-domain engineering analysis.

11.1.2 Why use IMF for this System?

The gas cooling system is intentionally chosen as a use case because it exhibits several characteristics that are typical of industrial systems and problematic for conventional, document-centric engineering practice. Information about the system is distributed across multiple disciplines, tools, and lifecycle stages, and engineering questions frequently require navigating between functional intent, physical realisation, and spatial or organisational context.

IMF addresses these challenges by providing a single, coherent modelling framework in which such heterogeneous information can be represented, related, and analysed without forcing it into a single canonical system view. The value of this approach can be understood from different stakeholder perspectives.

Engineering Perspective

From an engineering point of view, this use case illustrates how IMF supports everyday reasoning tasks that are otherwise time-consuming and error-prone.

- **Managing fragmentation.** Engineering information relevant to a single question is often scattered across P&IDs, datasheets, 3D models, maintenance registers, and informal documentation. IMF allows these fragments to be represented as structured, connected appearances of the same system, reducing reliance on manual cross-referencing and personal expertise.
- **Separating concerns.** In traditional diagrams, functional requirements and physical assets are frequently conflated. By making *aspects* explicit, IMF distinguishes capability from physical implementation, intent and actual realisation, etc.
- **Cross-domain traceability.** The use case demonstrates how a physical problem, such as a gas leak at a pump, can be traced back through the model to its functional role and system-level impact. This traceability is achieved through explicit relations between appearances, rather than through ad-hoc naming conventions or manual interpretation.
- **Automated impact analysis.** Because the IMF model forms a connected graph, questions such as “Which functions are affected if this component is unavailable?” or “Which spatial locations are involved in this process path?” can be answered through automated queries and reasoning rather than manual document review.

Data Management and Digitalisation Perspective

From the perspective of data management, integration, and digital transformation, IMF provides a semantic backbone for governing engineering information as a long-lived corporate asset.

- **Semantic normalisation.** IMF provides a common structural language that aligns information from different disciplines and tools without requiring them to adopt identical schemas or naming conventions. Meaning is aligned at the level of structure and context rather than at the level of file formats.

11.2 Use Case Overview

To demonstrate how IMF manages information across disciplines, we use the gas cooling system shown in [Figure 11.1](#). The system is designed to cool a hot gas stream using seawater through an intermediate cooling medium, preventing direct contact between the two fluids. The process operates in two primary stages of heat exchange:

- **Gas Cooling.** Hot gas is cooled by a circulating cooling medium within a primary heat exchanger.
- **Medium Regeneration.** The warmed cooling medium is pumped to a secondary heat exchanger, where it is cooled by seawater. The chilled medium is collected in a storage tank before being recirculated to the primary exchanger.

From a process-engineering perspective, this is an intuitive small system. It represents a pure process concept: a schematic that highlights the essential transformation of heat from gas to seawater via an intermediate medium, without concern for implementation detail. At this level, the system can be understood primarily in terms of function and flow.

However, even for such a small system, complexity grows rapidly once the system must be realised as an engineered plant. As shown in [Figure 11.2](#), the same process concept expands into a dense, multi-disciplinary design containing a large number of uniquely identified components, instruments, and connections. Each alphanumeric tag corresponds to a specific physical or logical entity, valves, sensors, pumps, control loops, introduced to satisfy the requirements of piping, instrumentation, electrical, control, and safety engineering.

This transition marks the critical step from *functional design* to *executable engineering*. Information that was implicit or abstract in the process concept becomes explicit, detailed, and tightly interdependent. At the same time, responsibility for the system is distributed across disciplines, tools, and lifecycle stages. The resulting information landscape is necessarily fragmented: no single document or model captures the system in its entirety, yet engineering questions routinely cut across these fragments.

This characteristic makes the gas cooling system a suitable use case for IMF. The system exhibits multiple, overlapping views of the same underlying reality: functional behaviour, physical realisation, and spatial arrangement. These views are all valid, all necessary, and all structurally different. IMF is applied in this chapter to model these different appearances explicitly, relate them systematically, and enable reasoning across them without collapsing them into a single unified representation.

With IMF, the aim is not to simplify the system, but to manage its complexity in a principled way. All disciplines use the same minimal and precise modelling language, while contextual meaning is captured through aspects rather than implicit assumptions. Because IMF is based on knowledge-graph and semantic technologies, the resulting models support standards-based exchange, querying, and automated reasoning, providing a shared semantic core for cross-disciplinary engineering work.

11.3 Integrating Multiple Aspects in One IMF Model

IMF models represent engineered systems through multiple *aspects*, each capturing a distinct and explicitly scoped engineering perspective. In this use case, three central aspects are employed: the *function aspect*, the *product aspect*, and the *location aspect*. Together, these aspects allow the system to be described in terms of what it is intended to do, how it is physically realised, and where it is situated in space.

Before detailing each aspect, [Table 11.1](#) summarises their respective purposes and modelling roles. This table is intended as a guide for readers who may be new to IMF or who have not read the earlier chapters in detail.

Views on identifiers in a simple cooling system plant

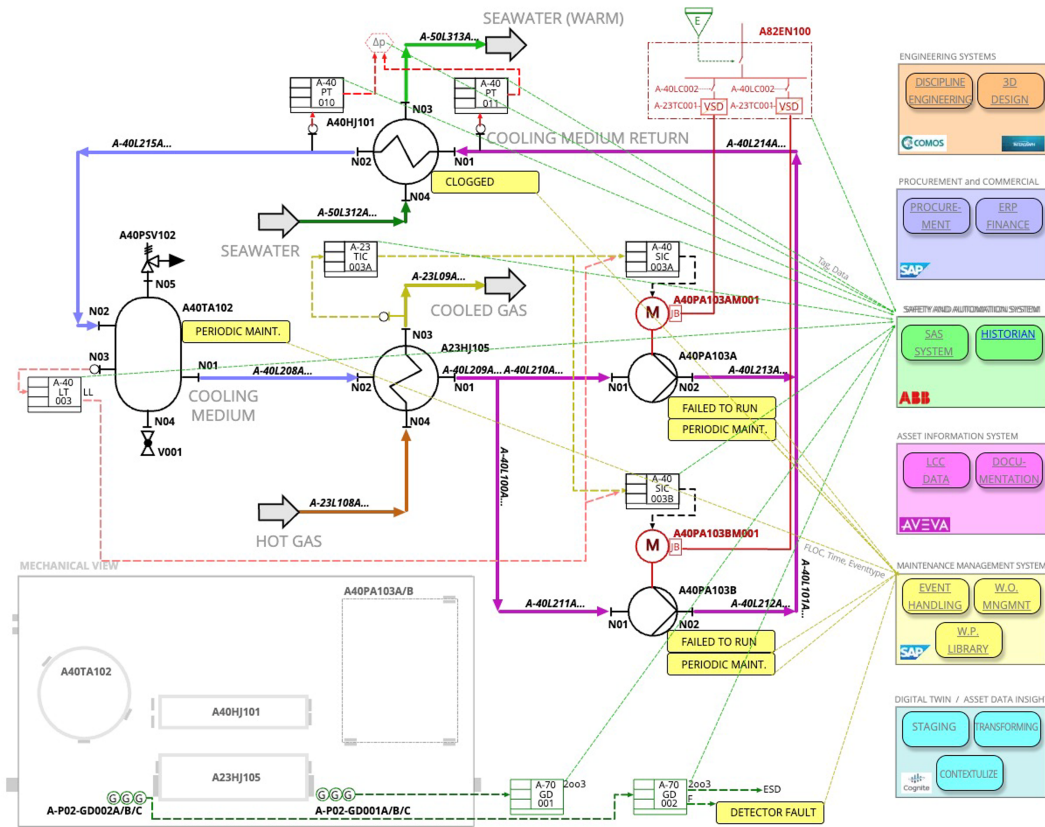


Figure 11.2: Multi-disciplinary Detailed Design of the Cooling System

The separation into aspects is an IMF-mandated principle: each appearance must be scoped to a single, explicit context. However, the *number of aspects*, their *granularity*, and their *naming* are project-specific modelling decisions. The following subsections explain how these principles are applied in this use case.

11.3.1 Function Aspect

The function aspect models the *capability* of the system based on its purpose, requirements, and flow of energy or information. It abstracts away from specific physical components and locations in order to focus on design intent and logical behaviour.

Design rationale: Separating function from implementation ensures that functional reasoning (e.g., impact analysis, requirement coverage) is not constrained or distorted by early design decisions or as-built details.

11.3.1.1 System Overview and Functional Breakdown

Figure 11.3 illustrates the gas cooling system from a functional perspective. The primary function of the system is Cooling. Through functional analysis (Figure 11.3b), this capability is decomposed into the following subsystems:

Table 11.1: Overview of aspects used in the gas cooling system model

Aspect	Modality	Category	Engineering documents	Key modelling choices
Function	intended	capability	Process flow diagrams, functional decompositions, requirement structures	Capture system capability and intent, Abstract away from physical components; decompose by purpose and behaviour
Product	intended	implementation	P&IDs, equipment lists, bills of materials	Describe the physical implementation of the system, model concrete components, assemblies, and physical interfaces
Location	intended	space	Layout drawings, area classifications, safety zones	Represent spatial placement and logical areas, allow overlapping and non-exclusive spatial groupings

- **Heat Exchange:** Two independent heat exchange subsystems.
- **Liquid Storage:** A single fluid storage subsystem with a pressure relief capability.
 - **Pressure Relief:** A single pressure relief subsystem.
- **Dual Pumping:** A pumping subsystem comprised of two parallel pump assemblies.

This decomposition reflects *functional necessity*, not physical layout or equipment selection. At this stage, questions such as redundancy, control logic, and safety intent can be analysed without reference to specific components.

11.3.1.2 Building the IMF Function Model

Based on the functional breakdown, an IMF function model is constructed, as shown in the [Figure 11.3c](#). The modelling process follows these steps:

1. **Create function blocks.** Six IMF function blocks (in yellow) are created to correspond to the identified subsystems: the overall Cooling System, two Heat Exchange systems, the Pressure Relief system, the Liquid Storage system, and the Dual Pumping system.
2. **Define terminals and media.** Terminals are assigned to each function block based on the type of medium they handle (e.g., gas, coolant, electrical signal). This defines the interfaces through which the functions interact.
3. **Establish functional relations.** The blocks are linked using precise IMF relations to reflect the system architecture:
 - `partOf` assembles 5 sub-function blocks into the overall Cooling System root node.
 - `connectedTo` defines logical interaction and flow between function blocks.

11.3.1.3 Extended Model with Auxiliary Systems

Further analysis of the complex system diagram reveals the presence of critical auxiliary systems beyond the core cooling loop. The IMF model is extended to include these, providing a broader functional representation:

- **Instrumented Safety System:** A function block is added for this system, which itself contains sub-functions for Gas Leak Detection and Ignition Source Tripping.

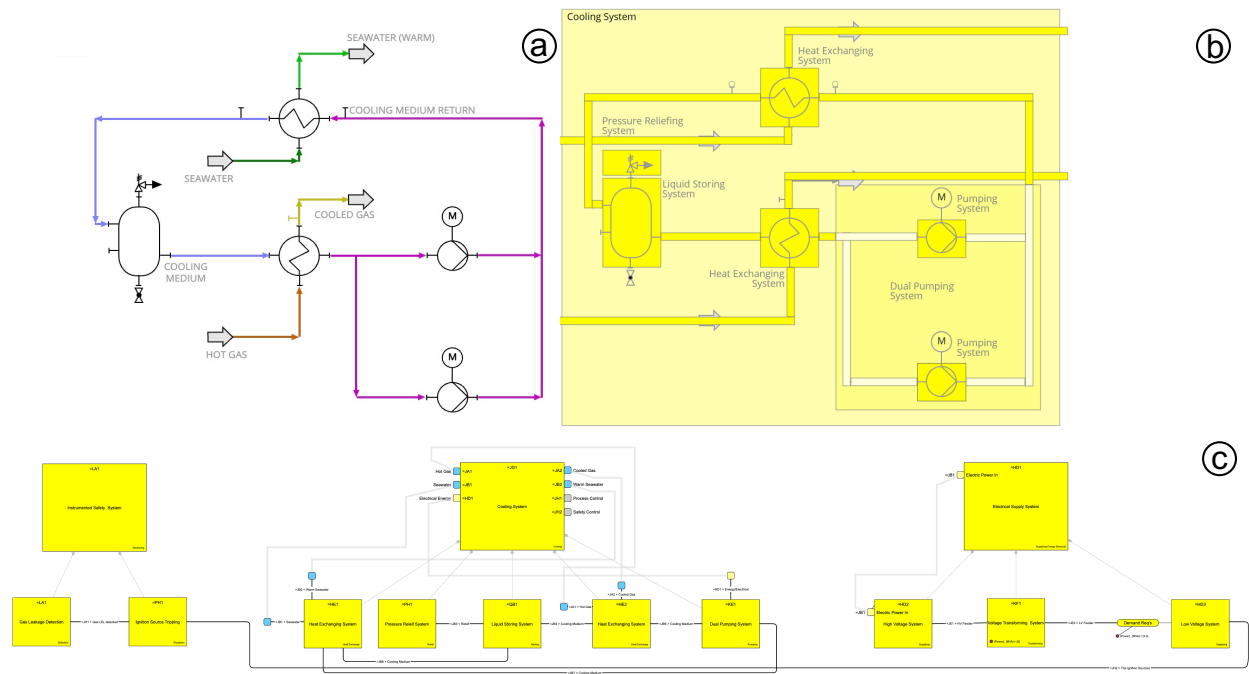


Figure 11.3: Functional view and IMF function model of the gas cooling system.

- **Electrical Supply System:** A function block is added to represent the power infrastructure, decomposed into High Voltage Supply, Low Voltage Supply, and Voltage Transformation sub-functions.

These additional blocks are integrated into the overall model using the same relational logic (partOf, connectedTo), creating a complete, hierarchical map of functions within the gas cooling system. This model serves as a source for understanding functional interactions and dependencies across disciplines.

11.3.2 Product Aspect

The product aspect models the *implementation* of the system as a breakdown of physical components and their connections. It provides the detailed physical description for procurement, construction, and maintenance.

Design rationale: Unlike the function aspect, the product aspect is more concrete. Every block corresponds to a tangible artefact that can be installed, replaced, or inspected.

11.3.2.1 System Overview and Physical Product Breakdown

Figure 11.4 is the example builds upon the functional model of the gas cooling system to define its physical implementation through an IMF product model. The main product is the Cooling System. Through product breakdown analysis Figure 11.4b, its primary physical subsystems and components are identified.

- **Two Heat Exchange Equipment:** This includes the Heat Exchanger Liquid and Heat Exchanger Gas units.
- **Storage Tank Package (Vessel):** An assembly consisting of a Medium Tank, a Pressure Relief Valve, and a Drain Valve.

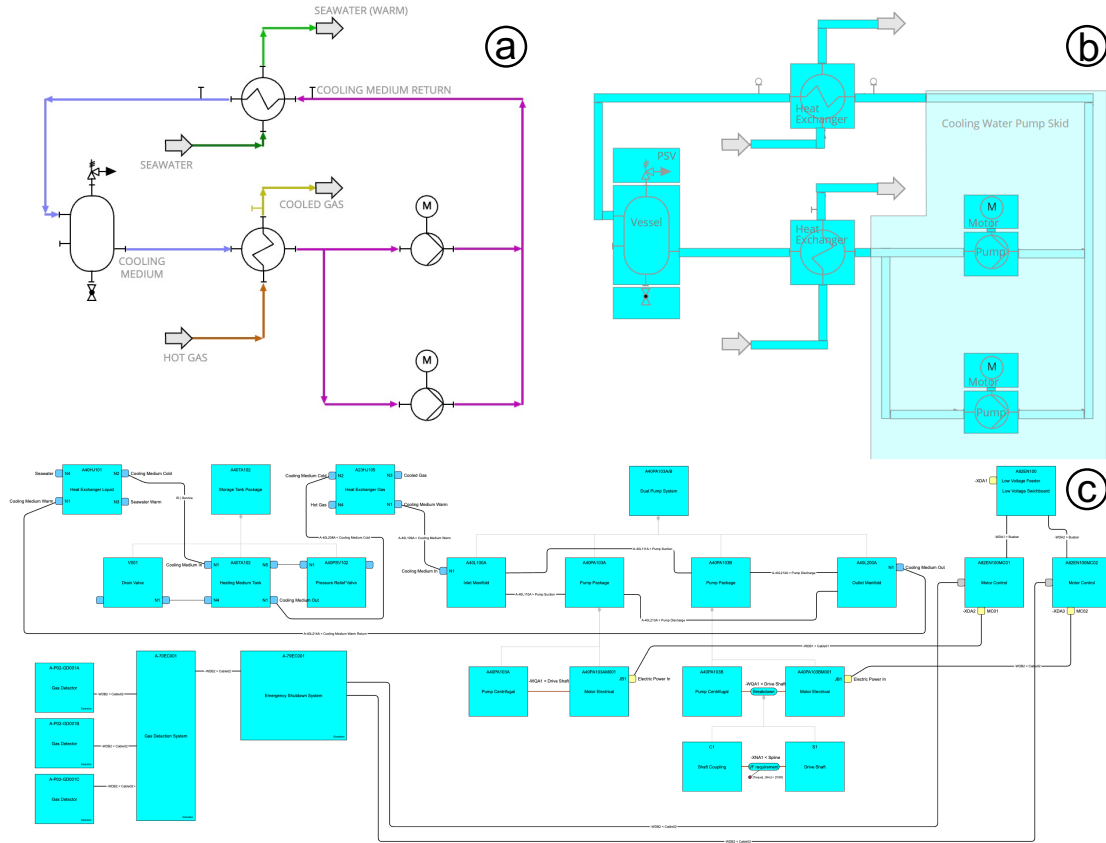


Figure 11.4: Product view and IMF product model of the gas cooling system

- **Cooling Water Pump Skid:** A redundant pumping assembly containing an Inlet Manifold, an Outlet Manifold, and two identical Pump Packages. Each Pump Package is further composed of a Centrifugal Pump and an Electrical Motor, which are physically linked via a Drive Shaft and Shaft Coupling.

11.3.2.2 Extended Model with Safety and Control Systems

Integrating with the previous functional analysis and a review of the complex system diagram reveals critical safety and control systems required for operation:

- **Emergency Shutdown System:** A safety-critical product system that interfaces with the main process. Its core component is a Gas Detection System, which itself comprises multiple Gas Detector sensor units.
- **Motor Controller:** The ESD system connects to Motor Control Devices to execute shutdown commands, physically linking the safety instrumentation to the primary equipment.

11.3.2.3 Constructing the IMF Product Model

Based on this physical breakdown, the IMF product model (shown in the bottom diagram) is constructed. The modelling procedure is as follows:

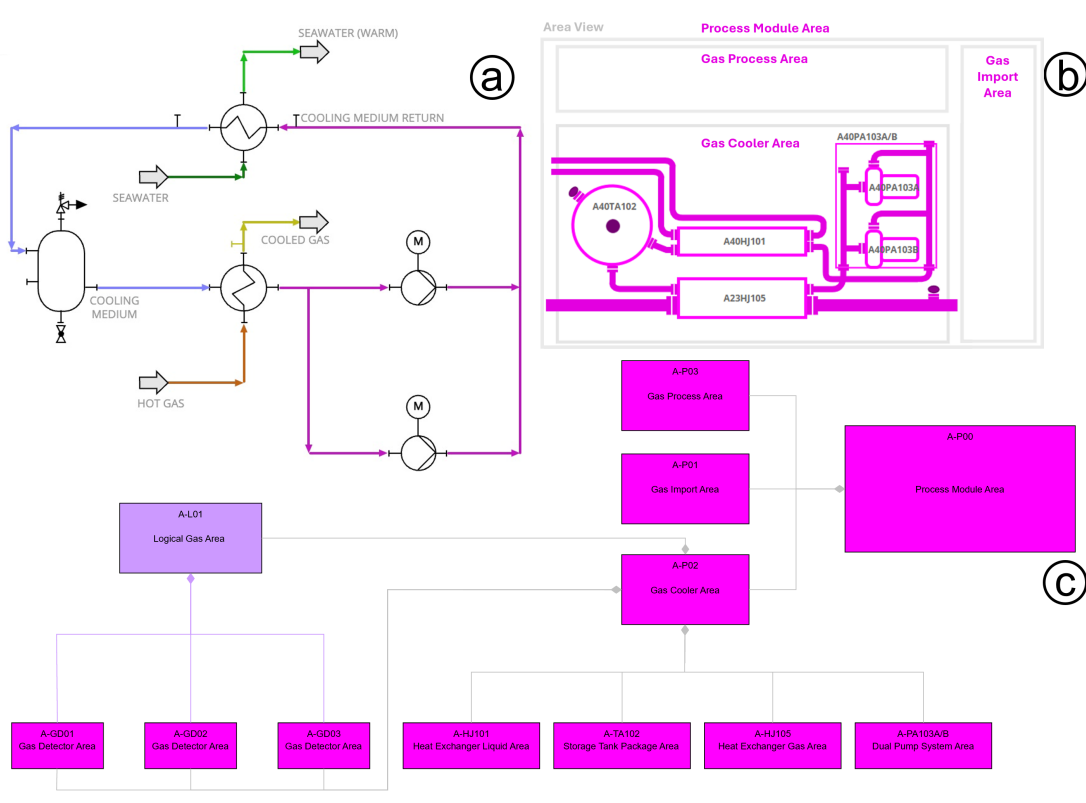


Figure 11.5: Spatial view and IMF location model of the gas cooling system

1. **Create Product Blocks:** Cyan-coloured product blocks are built to represent every identified physical device and assembly, from the main Cooling System block down to individual components like Gas Detector and Shaft Coupling.
2. **Define Terminals:** Terminals are assigned to blocks based on the interface media they handle (e.g., process fluid, electrical power, signal). This explicitly defines the physical connection points.
3. **Establish Structural and Interface Relations:** Blocks are linked using IMF relations to accurately reflect the system's Bill of Materials (BOM) and physical connectivity:
 - The *partOf* relation assembles components into their parent assemblies (e.g., Motor *partOf* Pump Package, Pump Package *partOf* Dual Pump System).
 - The *connectedTo* relation defines the physical interfaces between components and subsystems (e.g., Heat Exchanger Liquid *connectedTo* Dual Pump System, Gas Detection System *connectedTo* Motor Control).

This hierarchical product model provides a complete, unambiguous digital representation of the physical system, integrating core process equipment with essential safety and control devices.

11.3.3 Location Aspect

The location aspect models the *spatial extension* of the system based on physical or logical grouping in space. This view is essential for spatial planning, access control, and safety/emergency response.

The figure below depicts the spatial and area modelling of the example gas cooling system, showing its area view [Figure 11.5b](#) and corresponding location model [Figure 11.5c](#).

11.3.3.1 Area Hierarchy Analysis

From the area view, the Gas Cooler Area contains the following equipment zones:

- Heat Exchanger Liquid & Gas area (A40HJ101 and A23HJ105)
- Storage Tank Package area (A40TA102)
- Dual Pump System area (A40PA103A/B)

Examining the broader system context, figure 11.5 b shows that this Gas Cooler Area is itself part of a larger Process Module Area, which also includes two undefined areas: Gas Process Area and Gas Import Area. While these supporting areas are not fully depicted in the provided Process Flow Diagram (PFD), their existence is necessary to ensure the cooling system functions as intended.

11.3.3.2 Spatial Overlap and Logical Areas

Following the product model, the Gas Detection System (comprising three gas detectors) is critical for safety. Spatially, these detectors are located within the Gas Cooler Area.

Importantly, the gas detectors exhibit spatial overlap across different area classifications:

- **Physical Placement:** Each detector is physically located within the Gas Cooler Area
- **Detecting Coverage:** Each detector is also part of its individual Sensor Detecting Spatial Area. The union of all individual detecting areas forms a Logical Gas Area, which is also contained within the Gas Cooler Area

11.3.3.3 Constructing the IMF Spatial Location Model

Based on the spatial analysis, the IMF location model (shown in the bottom diagram) is constructed by applying hierarchical containment and spatial overlap relationships:

1. Define Hierarchical Areas: Location blocks are created for each identified area:

- Top-level: Process Module Area
- Second-level: Gas Cooler Area, Gas Process Area, Gas Import Area
- Equipment zones: Heat Exchanger Liquid Area, Heat Exchanger Gas Area, Storage Tank Area, Pump System Area, Gas Detector Area

2. Establish Containment Relations: The *partOf* relation defines the hierarchical structure:

- Equipment zones are *partOf* the Gas Cooler Area
- Gas Cooler Area is *partOf* the Process Module Area
- **Model Spatial Overlap:**
 - Each Gas Detector is *partOf* the Gas Cooler Area

- Each Gas Detector also has a Sensor Detecting Spatial Area, the union of all the Sensor Detecting Spatial Area is Logical Gas Area
- The collective Logical Gas Area is *partOf* the Gas Cooler Area

This modelling captures both the physical installation locations and the logical detection zones within the same spatial framework.

11.3.4 Complete IMF Model

After creating the function, product, and location aspect models independently, they must be integrated into a unified system view.

In IMF, in order to flip between aspects, IMF supports inter-aspect relations. One basic inter-aspect relation is the *proxy* relation. The *proxy* relation connects elements that represent different views of the same entity. In this system model, for example, it can be used to establish the following cross-aspect correspondences:

- Location block:Heat Exchanger Liquid Area ↔ Product block:Heat Exchanger Liquid
- Product block:Heat Exchanger Liquid ↔ Function block:Heat Exchanger System
- Location block:Pump System Area ↔ Product block:Dual Pump System
- Product block:Dual Pump System ↔ Function block:Dual Pumping System

Applying the *proxy* relation across all corresponding elements creates a fully integrated IMF model of the gas cooling system as shown below. This unified model maintains consistency across functional, physical, and spatial aspects, enabling navigation and analysis of the system from multiple engineering perspectives.

11.4 Querying the IMF Model: Finding the Source of a Gas Leak

A key strength of IMF models lies in their ability to support queries that cut across multiple engineering aspects, enabling complex questions to be answered in a precise and reproducible way.

This section illustrates that capability using a realistic operational scenario. During normal operation, a gas detector raises an alarm, indicating a potential gas leak. An operator or engineer must quickly determine the most likely source of the leak in order to take targeted isolation or mitigation measures. Rather than manually correlating alarm lists, area drawings, and P&IDs, this question can be answered directly by querying the IMF model.

In this example, the IMF model is represented as a knowledge graph using RDF, and the query is expressed in the SPARQL query language. SPARQL is shown here for transparency and precision.

Engineers are **NOT required** to write or understand SPARQL to use this functionality. The IMF toolkit is intended to provide built-in query templates and intuitive user interfaces (currently under development) that expose the same reasoning capabilities without requiring knowledge of the underlying query language.

The engineering question addressed is the following: *Given a gas detector that has raised an alarm, which physical components in the same area are potential sources of the gas leak?*

Given: A gas detector with tag A-P02-GD001B in an active alarm state.

Task: Identify all physical components (system blocks in the **Product Aspect**) that meet two criteria:

1. They are specified to handle **gas media**.
2. They are located within the same physical area (**Location Aspect**) as the alarming detector.

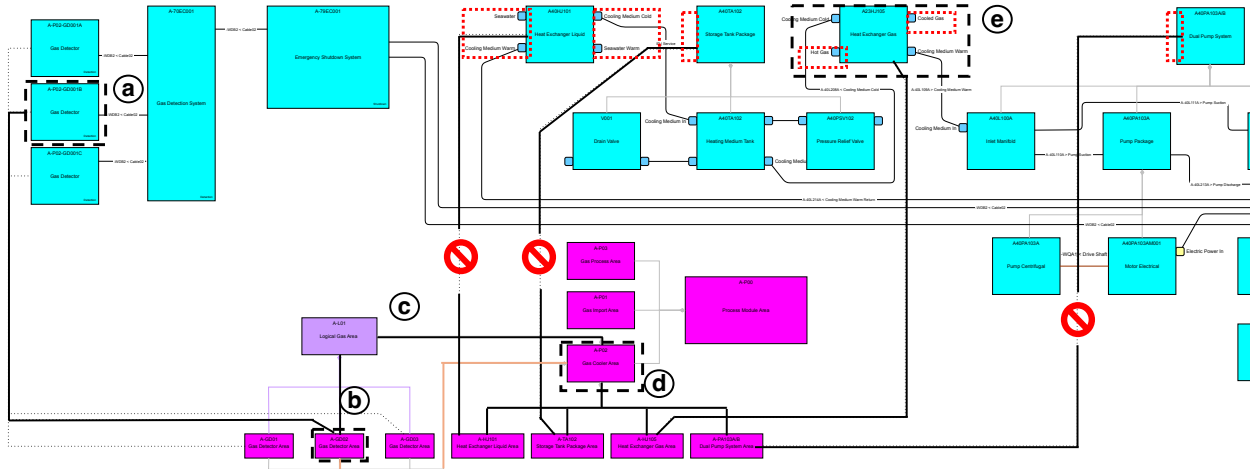


Figure 11.7: Conceptual overview of the cross-aspect query used to identify potential gas leak sources

Answering this question requires reasoning across aspects, linking spatial context to physical implementation. The overall query logic is illustrated in Figure 11.7, after which the individual query steps are explained in detail.

11.4.1 Step 1: Find Gas Leak Area

The first step determines *where* the gas leak is occurring. This step reasons within the *location aspect* of the IMF model. Starting from the alarming gas detector, the goal is to identify the physical area in which the detector is installed. This establishes the spatial context that will be used in the next step to locate potentially leaking equipment.

Conceptually, the reasoning can be summarised as: “Given a detector that has raised an alarm, find the area to which it belongs.”

SPARQL Query 1 (Find Gas Leak area)

```
PREFIX imf: <http://ns.imfid.org/imf#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>

SELECT ?locationTag ?locationLabel
WHERE {
  [ imf:tagID "A-P02-GD001B" ;
    (imf:proxy | ^imf:proxy) [
      imf:hasAspect imf:locationAspect ;
      (imf:partOf | ^imf:hasPart) [
        imf:tagID ?locationTag ;
        skos:prefLabel ?locationLabel ] ] ]
}
ORDER BY ?locationTag
```

Listing 11.1: SPARQL Query 1 (Find Gas Leak area)

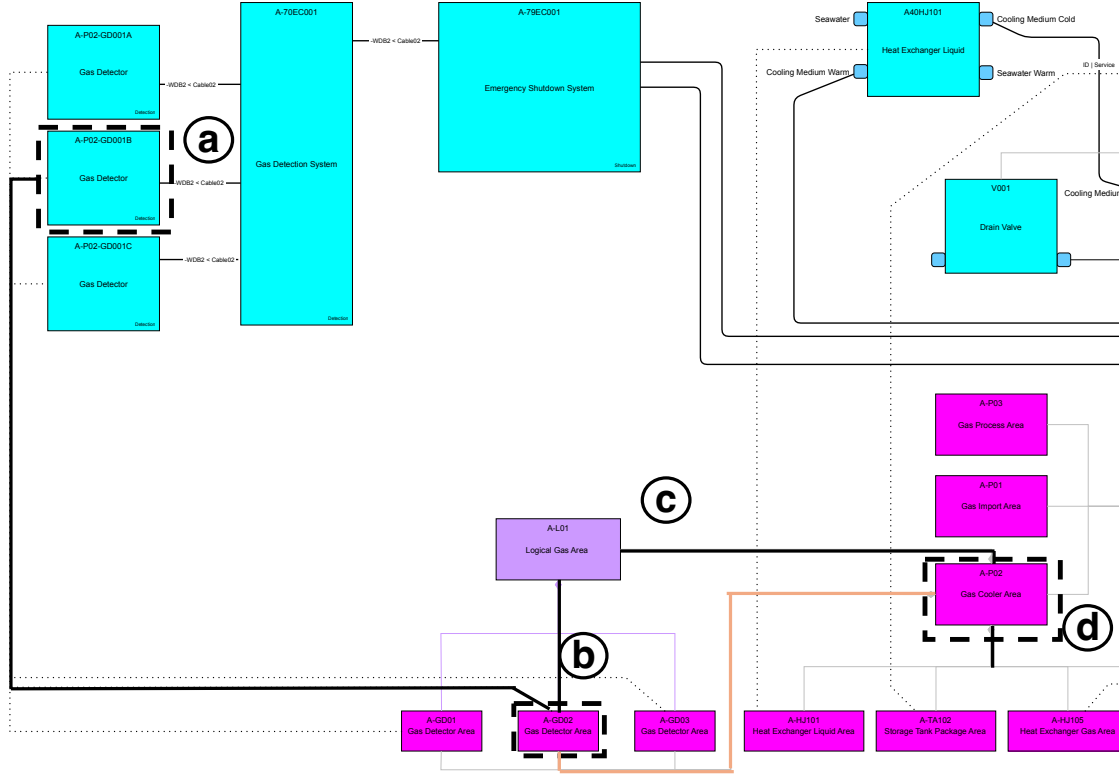


Figure 11.8: Query Step 1: Traversing the location aspect to identify the gas leak area from the alarming detector

Although expressed in SPARQL, the query [Listing 11.1](#) follows a simple and intuitive traversal pattern that mirrors how engineers reason about the system. The traversal logic of this query is illustrated in [Figure 11.8](#), highlighting how the detector is linked to its enclosing area through the location aspect of the IMF model.

The query proceeds as follows:

- **Identify the alarming detector.** Start from the gas detector with tag A-P02-GD001B (node a in the query path visualisation).
- **Switch to the location aspect.** Follow the proxy relation to reach the detector's corresponding system element in the *location aspect* (a → b).
- **Navigate the area hierarchy.** Traverse the part-whole relationship to find the enclosing location to which the detector belongs (b → d).
- **Return the area identifier.** Retrieve the tag ID and label of the identified location.

The result of the query is given in [Listing 11.2](#).

locationTag	locationLabel
"A-P02"	"Gas Cooler Area"

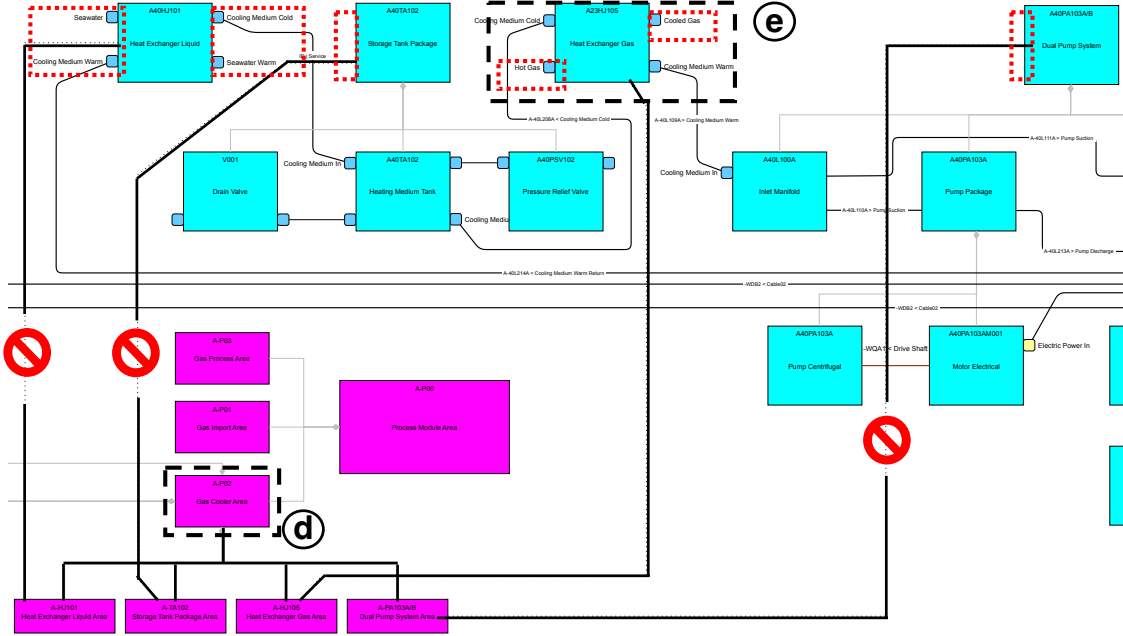


Figure 11.9: Query Step 2: Traversing the product aspect to find the gas handling equipment from the gas leak area

Listing 11.2: SPARQL Query 1 Result

The result shows that the alarming detector is located in the Gas Cooler Area, identified by the physical asset tag A-P02.

11.4.2 Step 2: Find Gas Handling Equipment in the Gas Leak Area

Once the leakage area has been identified, the next step is to determine *which physical components within that area could be responsible for the leak*. This step performs explicit *cross-aspect reasoning*, combining information from the *location aspect* and the *product aspect* of the IMF model.

Conceptually, the reasoning can be summarised as: “*Within the identified leak area, find all physical components that handle gas.*”

This step builds directly on the result of Step 1. The query starts from the same alarming detector and reuses the traversal logic to reach all system locations contained within the leak area.

SPARQL Query 2 (Find Equipment)

The following SPARQL query identifies all products located in the leakage area that handle the relevant medium (Gas). For each such product, the query returns its tag, label, and the specific terminals through which gas is handled.

```

PREFIX imf: <http://ns.imfid.org/imf#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>

SELECT DISTINCT ?productTag ?productLabel ?terminalTag
WHERE {
  [ imf:tagID "A-P02-GD001B" ;
    (imf:proxy | ^imf:proxy) [
      imf:hasAspect imf:locationAspect ;
      (imf:partOf | ^imf:hasPart) /
      (imf:hasPart | ^imf:partOf)* /
      (imf:proxy | ^imf:proxy) ?product ] ] .

  ?product
    imf:hasAspect imf:productAspect ;
    imf:tagID ?productTag ;
    skos:prefLabel ?productLabel ;
    imf:hasTerminal [
      imf:medium imf:gasMaterialMedium ;
      imf:tagID ?terminalTag ] .
}
ORDER BY ?productTag ?terminalTag

```

Listing 11.3: SPARQL Query 2 (Find equipment)

Although the query expression is compact in [Listing 11.3](#), it follows a clear logical sequence that mirrors the intended engineering reasoning [Figure 11.9](#):

- **Reuse the leakage-area traversal.** Starting from the alarming detector, follow proxy relations to the *location aspect* and identify the leakage area as in Step 1.
- **Enumerate contained locations.** Traverse the part-whole hierarchy to include all system locations that are part of the identified leakage area.
- **Switch from location to product aspect.** Use proxy relations to reach the corresponding physical products associated with each relevant system location.
- **Select gas-handling interfaces.** For each product, retrieve its terminals and filter those whose medium is specified as Gas.
- **Return actionable identifiers.** Output the product tags, labels, and terminal identifiers required for targeted operational response.

11.4.3 Final Result: Critical Equipment Identified

By combining information from the *location aspect* (where the leak is detected) and the *product aspect* (which components physically handle gas), the querying process isolates a small and actionable set of candidate leak sources. This demonstrates how IMF enables engineers to move efficiently from an alarm event to concrete, equipment-level insight.

```

-----
| productTag | productLabel          | terminalTag |
=====
| "A23HJ105" | "Heat Exchanger Gas" | "N3"       |
| "A23HJ105" | "Heat Exchanger Gas" | "N4"       |
-----

```

Listing 11.4: SPARQL Query 2 Result

The query result, given in [Listing 11.4](#), indicates that the Heat Exchanger Gas, identified by tag A23HJ105, is the only product within the affected area that exposes gas-handling interfaces. Specifically, its terminals N3 and N4 are identified as the relevant gas connection points.

From an operational perspective, this significantly narrows the scope of investigation. Instead of inspecting all equipment within the Gas Cooler Area, engineers can immediately focus on a single physical asset and its gas interfaces.

11.5 Using Query Results to Extract System Attributes

The examples in the previous section demonstrated how IMF models can be queried to identify *which* physical asset and interface are most likely involved in a gas leak. Once this identification is made, the same identifiers can be used to retrieve detailed engineering attributes from external IT systems that remain the authoritative sources for such data.

This section illustrates this integration principle. Rather than duplicating or re-modelling all detailed attributes inside IMF, the framework acts as a *semantic navigation and identification layer*. IMF provides precise, context-aware identifiers (e.g., equipment tag and terminal) that enable engineers and digital tools to access relevant information across connected systems in a consistent and automated manner.

In the example in [Listing 11.5](#), the query result identifies the gas-handling interface A23HJ105, terminal N3. Using this information, a RESTful API call is issued to an imagined engineering information system to retrieve detailed component data. The returned data is shown *as-is* and is intentionally not reformatted into IMF structures, reflecting a common industrial setup where detailed design, vendor, or inspection data is managed in specialised systems.

```
{
  "flange_id": "A23HJ105",
  "nozzle": "N3",
  "size": "14 inch",
  "nominal_diameter_mm": 355,
  "rating_class": "ASME Class 1500",
  "pressure_range_psi": 1500,
  "material": {
    "body": "Carbon Steel A105",
    "face": "RTJ",
    "finish": "Shot blasted"
  },
  "surface_treatment": {
    "coating": "Epoxy",
    "coating_thickness_microns": 120
  },
  "gasket": {
    "type": "Ring Type Joint",
    "rtj_profile": "R59",
    "material": "Inconel 625",
    "inner_diameter_mm": 396.9,
    "outer_diameter_mm": 499.5,
  },
  "bolt_pattern": {
    "bolt_circle_diameter_mm": 635,
    "bolt_length_mm": 425,
    "bolt_size_mm": 57.15,
    "num_bolts": 16
  },
  "operating_conditions": {
    "temperature_max_c": 427,
    "temperature_min_c": -29,
```

```

    "fluid": "Natural Gas",
    "approval_codes": ["API 6A", "NACE MR0175/ISO 15156"]
  },
  "manufacturer": {
    "name": "Stavanger Flanges",
    "location": "Stavanger Forus 123",
    "part_number": "SF-14-1500-RTJ60"
  },
  "traceability": {
    "lot_number": "LT-2024-11-07",
    "inspection_date": "2024-12-15",
    "certificates": ["WPS_2024-12-15", "QA-9151"]
  },
  "notes": "RTJ seating surface inspected for wear; gasket seating surface within spec."
}

```

Listing 11.5: Response for A23HJ105, N3

11.6 IMF RDF Graph Representation

IMF models are not limited to graphical representations as shown in the preceding sections. In addition to visual modelling, every IMF model can be represented as a structured *RDF knowledge graph*. This is enabled by formally defining the IMF graphical language and specifying a precise translation from that language into RDF.

This RDF representation is a key enabler for many of the capabilities demonstrated in this chapter. In particular, it allows IMF models to be queried, validated, and reasoned over using standard semantic technologies. The SPARQL queries shown earlier in this chapter operate directly on this RDF graph representation.

The RDF graph uses the vocabulary defined in the IMF OWL ontology and is constrained by the IMF SHACL shapes, both available at <http://ns.imfid.org>. Together, the OWL ontology and SHACL shapes define the IMF metamodel for practical use with semantic technologies: OWL provides formal semantics and inference rules, while SHACL specifies structural and data-quality constraints for valid IMF models.

11.6.1 IMF RDF Example

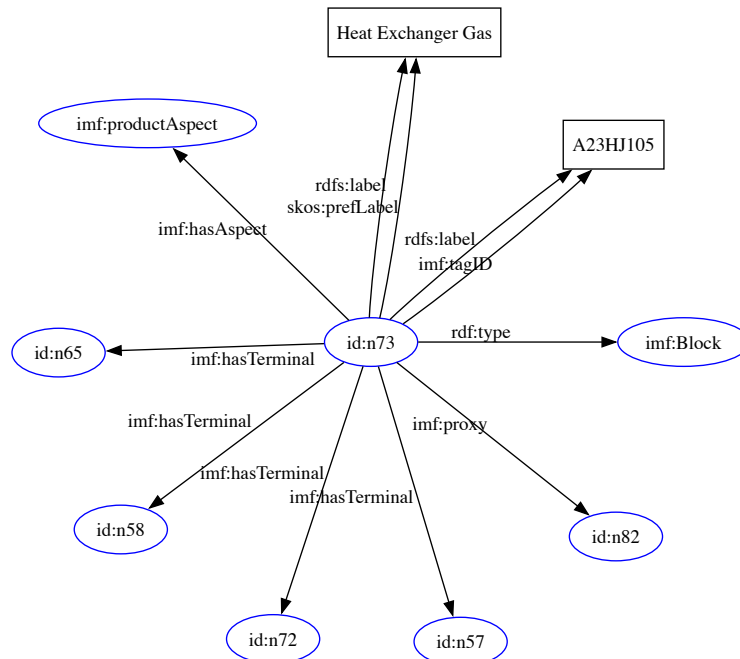
To illustrate how IMF models are represented in RDF, Listing 11.6 shows a Turtle serialisation of the product block with tag A23HJ105 (*Heat Exchanger Gas*) from the use case model.

```

PREFIX dc: <http://purl.org/dc/terms/>
PREFIX id: <http://example.com/id/>
PREFIX imf: <http://ns.imfid.org/imf#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdl: <http://example.org/rdl#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema>

id:n73  rdf:type          imf:Block;
        rdfs:label       "A23HJ105" , "Heat Exchanger Gas";
        imf:hasAspect    imf:productAspect;
        imf:hasTerminal  id:n65 , id:n58 , id:n72 , id:n57;
        imf:proxy        id:n82;
        imf:tagID        "A23HJ105";

```



Model:
(Unknown)

Namespaces:
 dc: <http://purl.org/dc/terms/>
 id: <http://example.com/id/>
 imf: <http://ns.imfid.org/imf#>
 owl: <http://www.w3.org/2002/07/owl#>
 rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
 rdfs: <http://www.w3.org/2000/01/rdf-schema#>
 rdl: <http://example.org/rdl#>
 skos: <http://www.w3.org/2004/02/skos/core#>
 xsd: <http://www.w3.org/2001/XMLSchema>

Figure 11.10: Graphical visualisation of the RDF representation of block A23HJ105.

```
skos:prefLabel "Heat Exchanger Gas" .
```

Listing 11.6: RDF representation of block A23HJ105

The RDF fragment in [Figure 11.10](#) encodes the same information shown earlier in the graphical IMF model: the block is typed as an `imf:Block`, assigned to the product aspect, connected to its terminals, identified by a tag ID, and linked to corresponding elements in other aspects via the proxy relation.

The complete IMF RDF graph for the gas cooling system use case is available at: imfid.org/2025-cooling-system-case/model/CoolingSystem.ttl.

Figure 11.11 shows a visualisation of the complete RDF graph. While such a graph is typically too dense for direct manual inspection, it provides a precise and machine-readable representation of all system aspects and their interconnections.

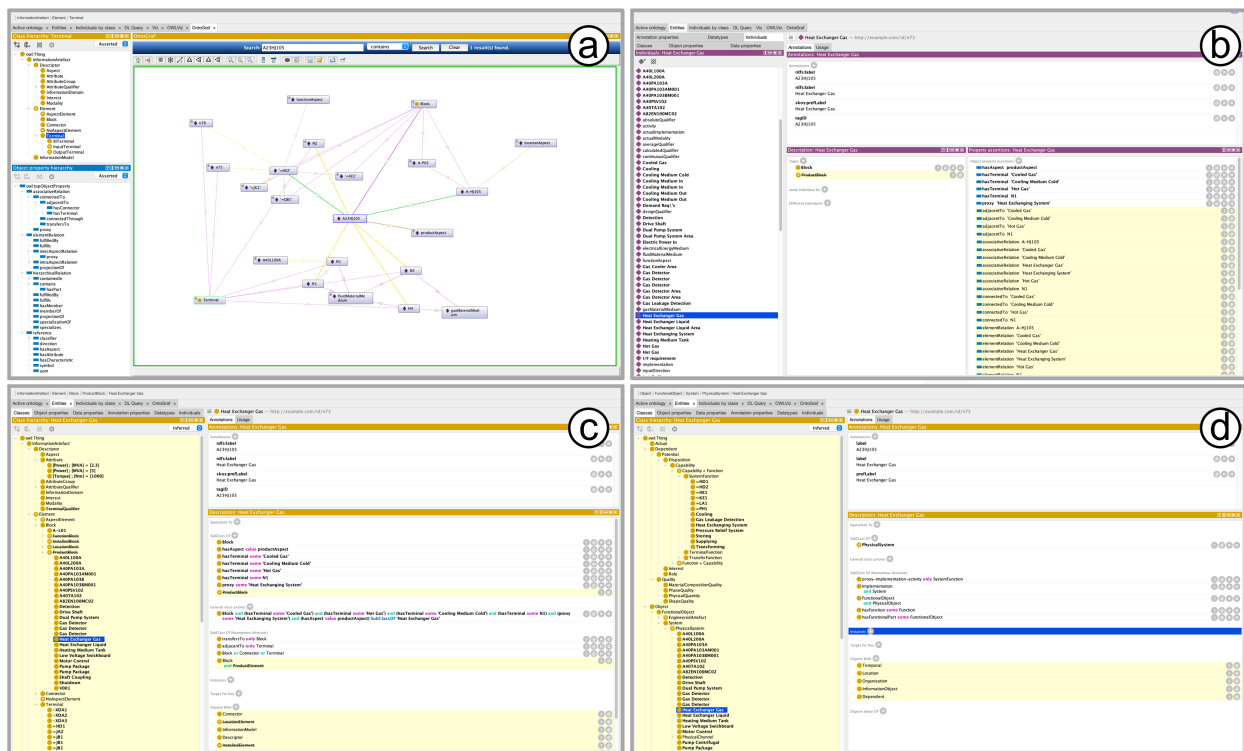


Figure 11.12: **a:** Protégé visualisation of the IMF ontology and the RDF instance data surrounding A23HJ105. **b:** OWL reasoning results in Protégé, showing inferred facts for block A23HJ105. **c:** OWL class-based representation of A23HJ105 in Protégé, showing inferred classification relationships produced by OWL reasoning. **d:** Protégé visualisation of the IMF model translated into an IDO-compliant OWL ontology, including inferred facts.

In addition to this instance-oriented representation, the same IMF model information can also be expressed using an alternative *class-based OWL representation*. In this form, specific system elements are modelled as OWL classes rather than individuals. For example, the element A23HJ105 is represented as a subclass of `imf:Block`, with its defining characteristics encoded as class axioms.

This class-based representation enables the use of OWL *classification reasoning*. Through automated reasoning, more specific block descriptions can be inferred as subclasses of more generic block descriptions based on their properties, constraints, and relationships. This capability is particularly useful for:

- Revealing unintended or implicit relationships between model element descriptions.
- Detecting inconsistencies or overlaps between functional, product, and spatial specifications.
- Identifying cases where product implementations are intended to fulfil specific functional or location-based requirements, but where this alignment is only implicitly stated.

By supporting both instance-based and class-based OWL representations, IMF enables complementary analysis modes: operational querying on concrete assets and design-time reasoning over system descriptions and specifications. Figure 11.12c shows a Protégé view of the OWL class representation of A23HJ105, including inferred superclass relationships derived through OWL reasoning. The OWL class-based

representation of the IMF use case model is available at: imfid.org/2025-cooling-system-case/model/CoolingSystem.rdf+owltype.rdf.

11.8 IMF Model IDO Representation

An initial mapping of the IMF language to the Industrial Data Ontology (IDO) is currently under development. This mapping enables IMF RDF models to be translated into an OWL ontology that conforms to the IDO OWL specification, thereby aligning IMF-based engineering models with a broader, cross-industry semantic framework.

Through this translation, IMF concepts such as blocks, aspects, terminals, and relations are expressed using IDO-compliant classes and properties. This allows IMF models to interoperate with other IDO-aligned data sources and to benefit from the generic reasoning capabilities and integration patterns supported by IDO-based tooling.

The IDO representation of the IMF use case model is available at: imfid.org/2025-cooling-system-case/model/CoolingSystem.ttl-ido.owl.

Figure 11.12d shows a Protégé view of the IDO-based representation of the gas cooling system IMF model. The screenshot illustrates how IMF instance data is embedded within the IDO ontology structure and how inferred knowledge can be derived using OWL reasoning at the IDO level.

11.9 IMF Model AAS Representation

In addition to ontology-based representations, an experimental translation of IMF models into the Asset Administration Shell (AAS) framework has been implemented. This work explores how IMF can serve as a semantic backbone for asset-centric digital twins commonly used in industrial automation and Industry 4.0 environments.

In the AAS representation, selected IMF elements—such as physical products, identifiers, and key attributes—are mapped to AAS shells, submodels, and properties. While this translation does not yet cover the full expressiveness of IMF (e.g., multi-aspect reasoning and rich inter-aspect relations), it demonstrates a viable pathway for integrating IMF-based system models with AAS-based asset ecosystems.

The AAS representation of the IMF use case model is available at: imfid.org/2025-cooling-system-case/model/CoolingSystem.ttl.aasx.

Figure 11.13 shows the AAS shell corresponding to the A23HJ105 component as viewed in the AAS Explorer. This illustrates how IMF-derived system knowledge can be consumed by AAS tools for visualisation, integration with operational systems, and lifecycle management.

11.10 Scope, Limitations, and Common Pitfalls

The use case presented in this chapter demonstrates how IMF can be applied to integrate functional, physical, and spatial system information and enable powerful cross-domain reasoning. However, it is equally important to clarify what IMF is *not* intended to do, where its boundaries lie, and which pitfalls practitioners should be aware of when applying the framework in real projects.

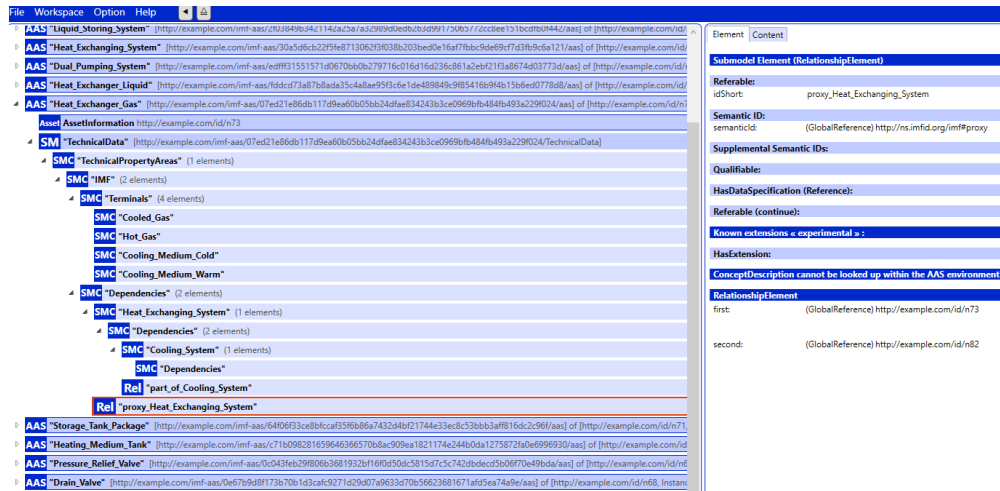


Figure 11.13: Asset Storing Administration Shell (AAS) representation of the A23HJ105 component derived from the IMF model.

11.10.1 What IMF Does Not Answer

IMF is an information modelling framework, not a replacement for domain-specific engineering tools or analyses. In particular, IMF does *not* aim to:

- **Perform numerical simulation or optimisation.** IMF does not calculate heat transfer, pressure drops, structural stresses, or control responses. Such analyses remain the responsibility of specialised simulation and calculation tools.
- **Replace discipline-specific data models in detail.** IMF does not attempt to fully replicate the internal data structures of CAD, CAE, PLM, or maintenance systems. Instead, it provides a semantic layer that links and contextualises information managed by those systems.
- **Define engineering correctness by itself.** IMF can validate structural consistency and semantic completeness (e.g., required attributes exist, relations are well-formed), but it does not determine whether a design is technically correct, safe, or optimal.
- **Prescribe a single modelling style for all projects.** IMF defines core concepts and relations, but it deliberately leaves room for project-specific conventions, naming schemes, and levels of detail.

Understanding these boundaries helps avoid unrealistic expectations and positions IMF correctly as an enabling framework rather than a monolithic solution.

11.10.2 Common Modelling Pitfalls

Based on early applications and experiments, several recurring pitfalls have been observed:

- **Collapsing aspects prematurely.** A frequent mistake is to merge functional, product, and location concerns into a single model structure. This undermines one of IMF's core strengths: the explicit separation of concerns and the ability to reason across aspects using well-defined relations.
- **Over-modelling too early.** Attempting to capture every conceivable component, attribute, or relation from the outset can lead to unnecessary complexity. IMF models are most effective when they grow

incrementally, driven by concrete use cases and questions to be answered.

- **Treating IMF as a data repository.** IMF should not be used as a dumping ground for all available data. Its value lies in structuring, linking, and contextualising information, not in duplicating large volumes of detailed records already managed elsewhere.
- **Ignoring modelling rationale.** Without documenting why certain aspect splits, breakdowns, or proxy relations were chosen, models can become difficult to interpret and evolve. Explicit modelling intent is essential for long-term maintainability.

11.10.3 Practical Guidance for Adoption

To apply IMF effectively in practice, the following principles are recommended:

- **Start from concrete questions.** Define the engineering or operational questions the model should answer (e.g., impact analysis, fault tracing, compliance checking) and let these guide the modelling scope.
- **Model just enough structure.** Introduce elements, relations, and attributes only when they support a clear use case. IMF models can always be extended as new needs arise.
- **Leverage standards-based tooling.** Use existing RDF, OWL, SHACL, and SPARQL tools where possible, rather than building bespoke solutions. This reinforces interoperability and sustainability.
- **Treat IMF as a shared language.** IMF is most valuable when used as a communication and integration layer across disciplines, organisations, and lifecycle phases, rather than as a discipline-specific artefact.

11.10.4 Closing Remarks

This chapter has shown how IMF moves from abstract concepts to practical application through a realistic engineering use case. By combining disciplined modelling, explicit separation of aspects, and semantic technologies, IMF enables engineers and organisations to reason across complex systems in ways that are difficult or impossible with document-centric approaches.

At the same time, IMF remains intentionally focused: it structures and connects information, but relies on existing tools, analyses, and engineering judgement to provide domain-specific depth. Used with this understanding, IMF can serve as a robust foundation for integrated engineering, digital twins, and data-driven decision support across the system lifecycle.